

Name:Srushti Rajendra Mulik
RollNO:2010

Home Work

Advertising Dataset

1.Load using pandas

```
In [1]: import pandas as pd

df=pd.read_csv("D:/AI Notes/All Datasets (1)/Advertising.csv")
df.head()
```

```
Out[1]:
```

	TV	radio	newspaper	sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5
4	180.8	10.8	58.4	12.9

2.perform Data Cleaning

```
In [2]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype  
---  -
0   TV           200 non-null   float64
1   radio        200 non-null   float64
2   newspaper    200 non-null   float64
3   sales        200 non-null   float64
dtypes: float64(4)
memory usage: 6.4 KB
```

```
In [3]: df.isnull().sum()
```

```
Out[3]: TV          0
        radio       0
        newspaper   0
        sales       0
        dtype: int64
```

```
In [4]: df.duplicated().sum()
```

```
Out[4]: np.int64(0)
```

```
In [5]: df.dtypes
```

```
Out[5]: TV          float64
        radio       float64
        newspaper   float64
        sales       float64
        dtype: object
```

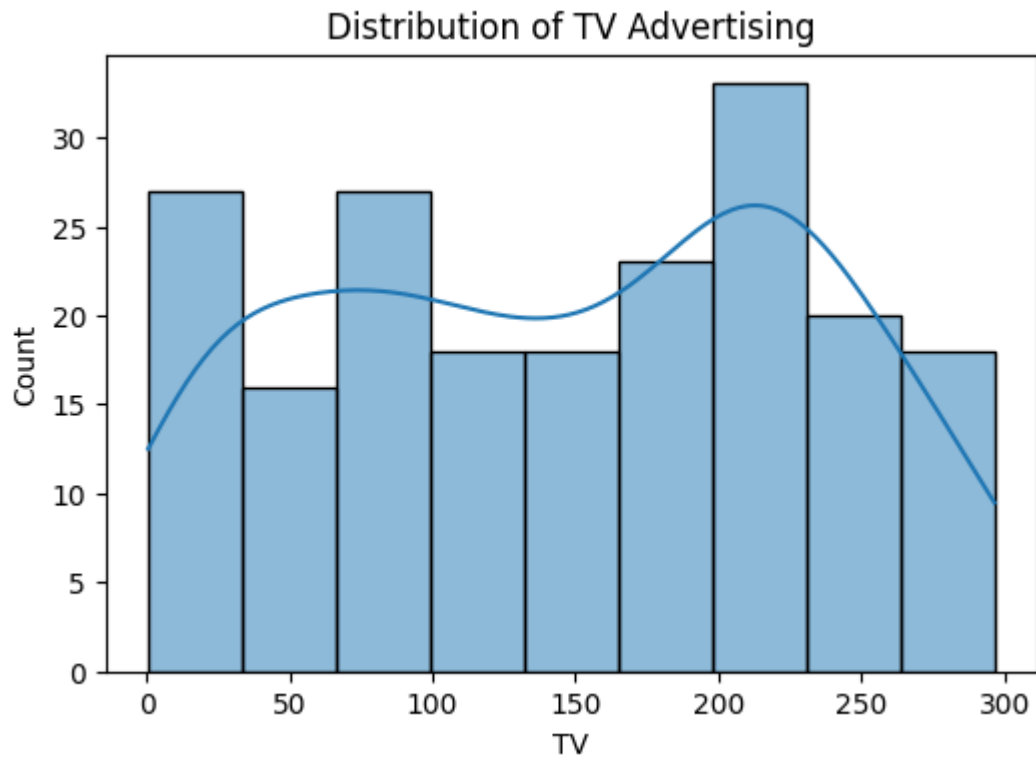
3.perform Exploratory Data Analysis(EDA)

```
In [6]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

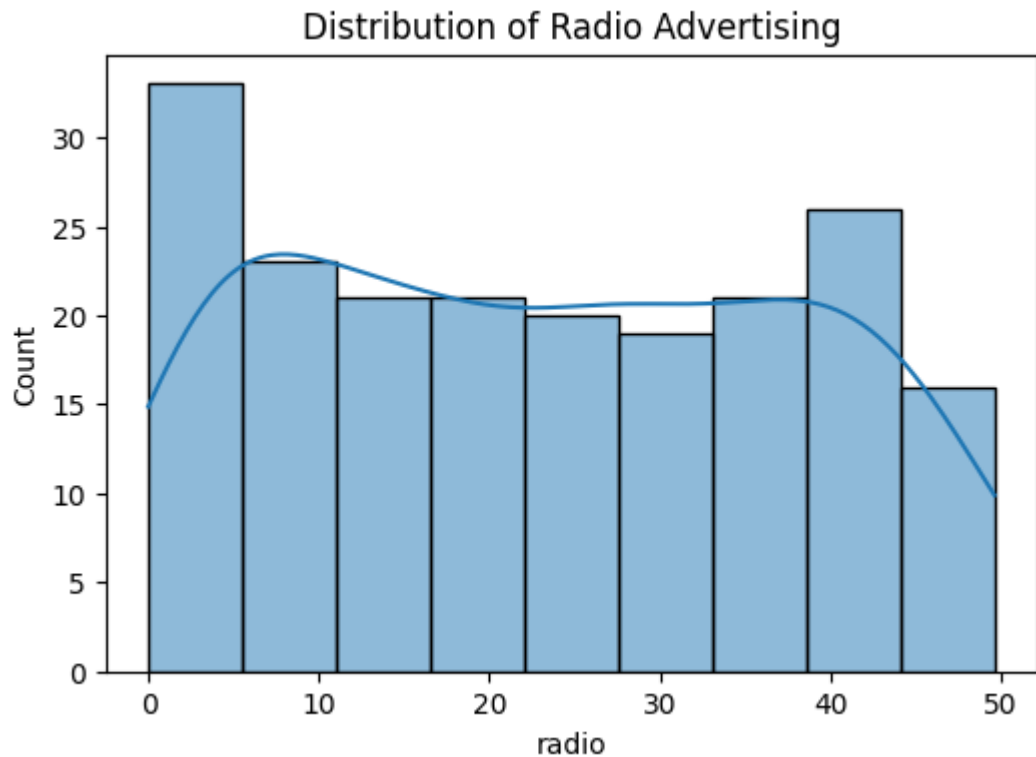
df=pd.read_csv("D:/AI Notes/All Datasets (1)/Advertising.csv")

if "Unnamed: 0" in df.columns:
    df.drop("Unnamed: 0",axis=1,inplace=True)

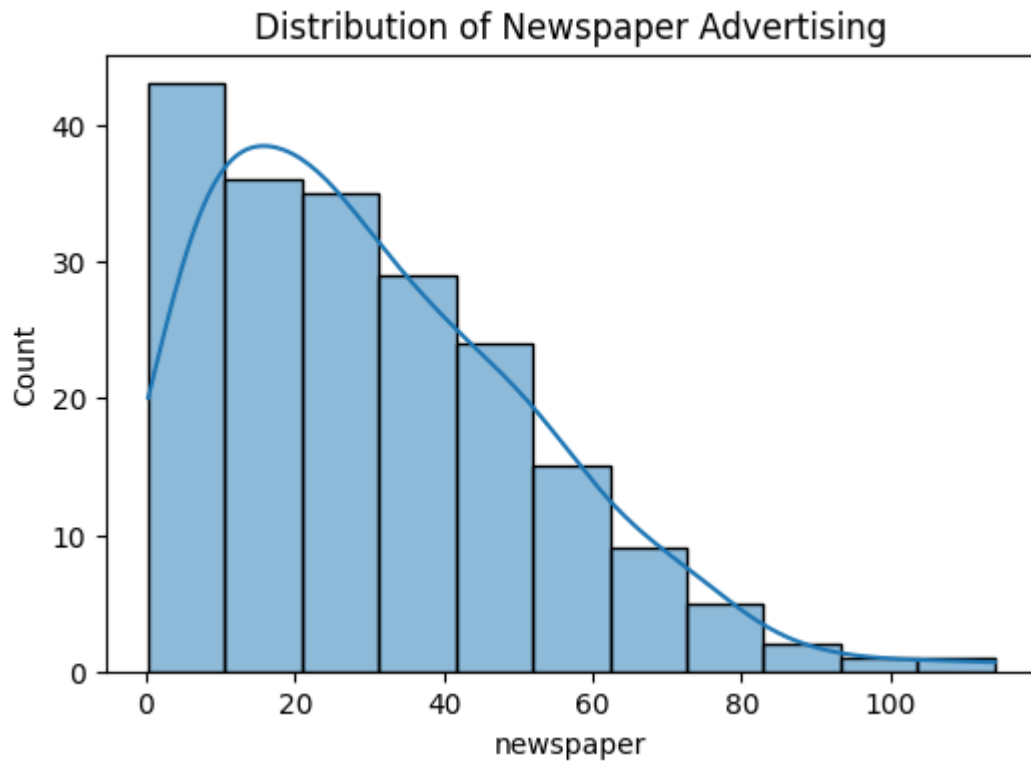
#1.Histogram-Tv
plt.figure(figsize=(6,4))
sns.histplot(df["TV"],kde=True)
plt.title("Distribution of TV Advertising")
plt.show()
```



```
In [7]: #2.Histogram-Radio
plt.figure(figsize=(6,4))
sns.histplot(df["radio"],kde=True)
plt.title("Distribution of Radio Advertising")
plt.show()
```



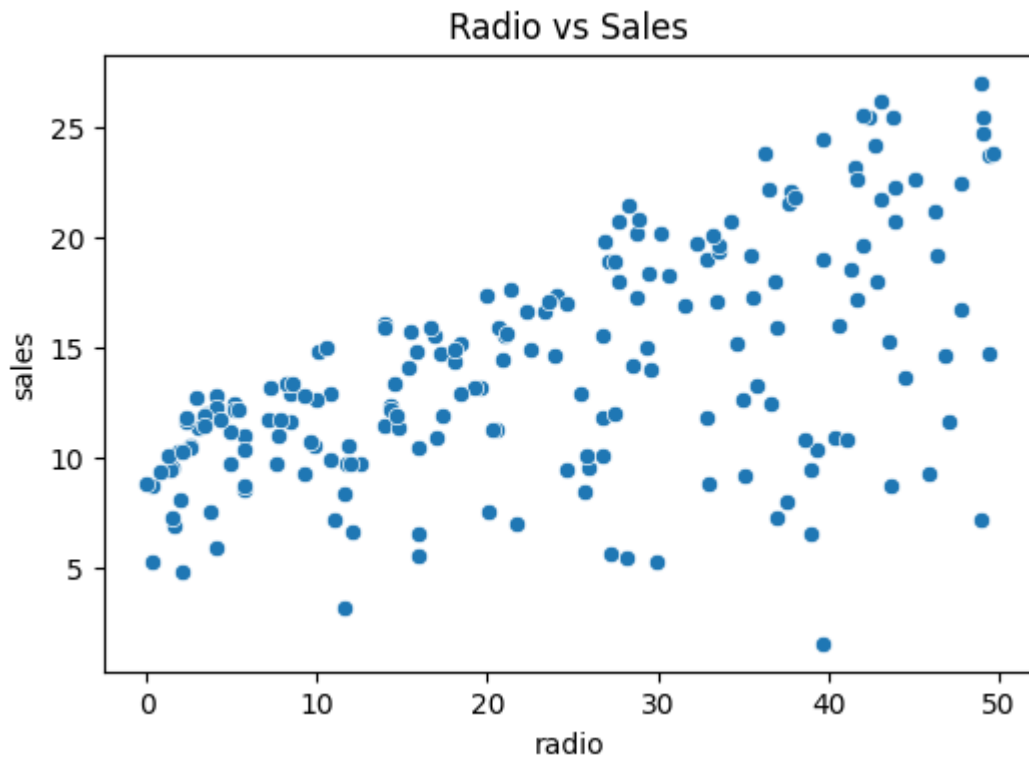
```
In [8]: #3.Histogram-Newspaper
plt.figure(figsize=(6,4))
sns.histplot(df["newspaper"],kde=True)
plt.title("Distribution of Newspaper Advertising")
plt.show()
```



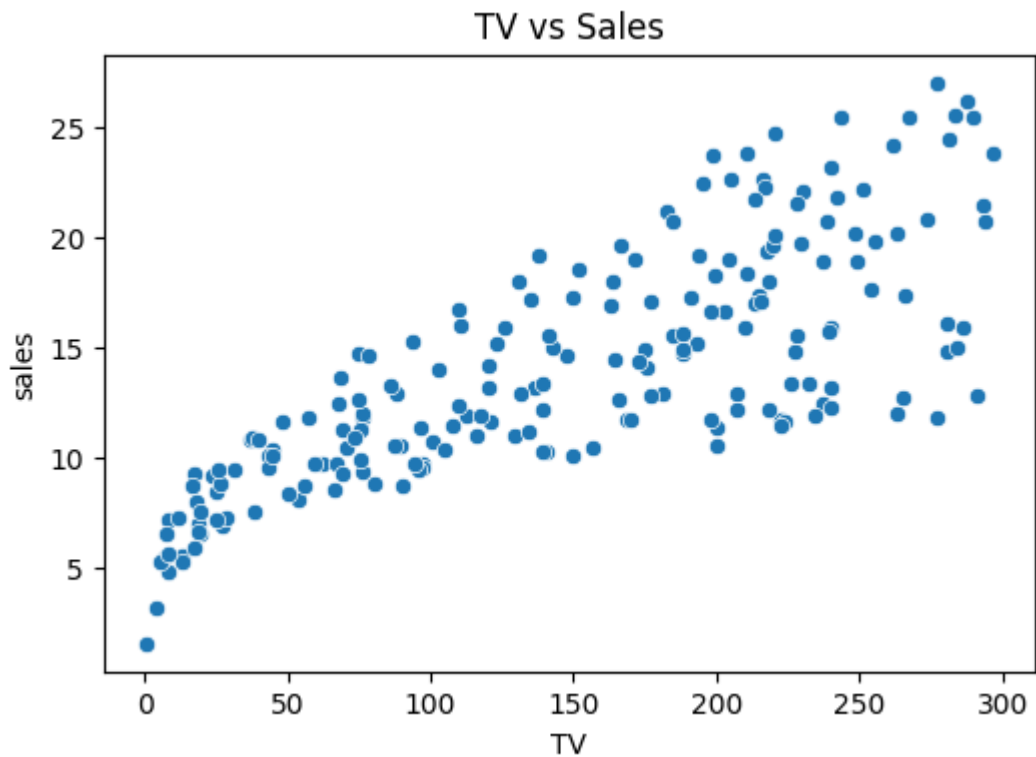
```
In [9]: #4.Histogarm-Sales
plt.figure(figsize=(6,4))
sns.histplot(df["sales"],kde=True)
plt.title("Distribution of Sales Advertising")
plt.show()
```



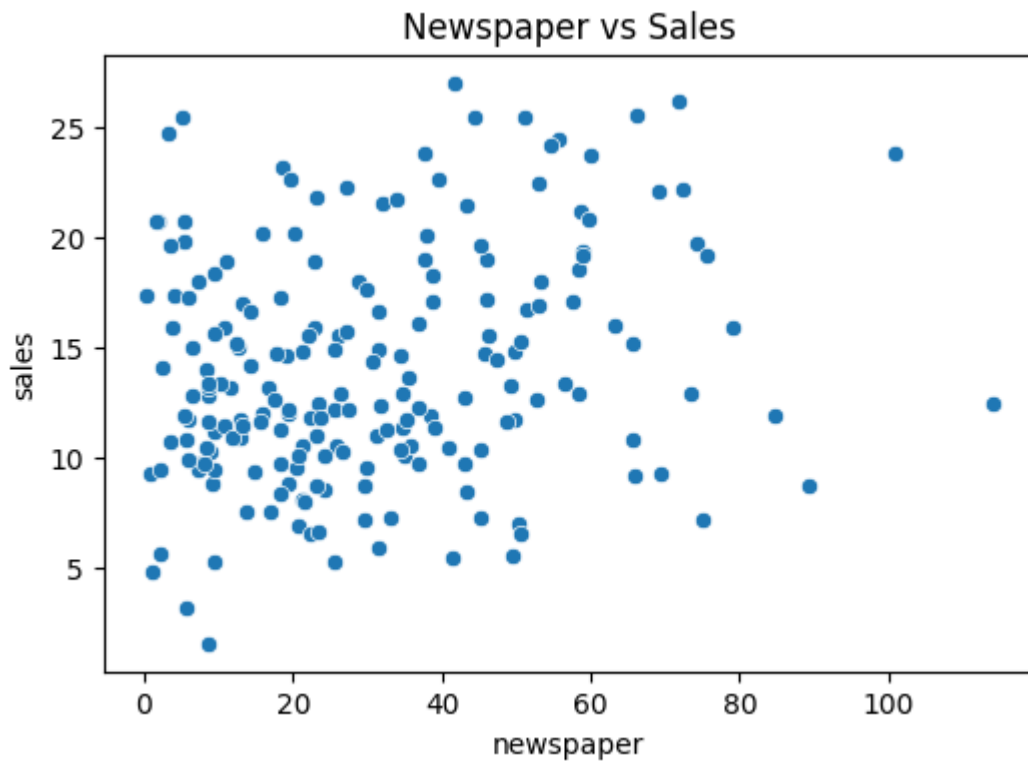
```
In [10]: #5.Scatter plot-Radio vs Sales
plt.figure(figsize=(6,4))
sns.scatterplot(x="radio",y="sales",data=df)
plt.title("Radio vs Sales")
plt.show()
```



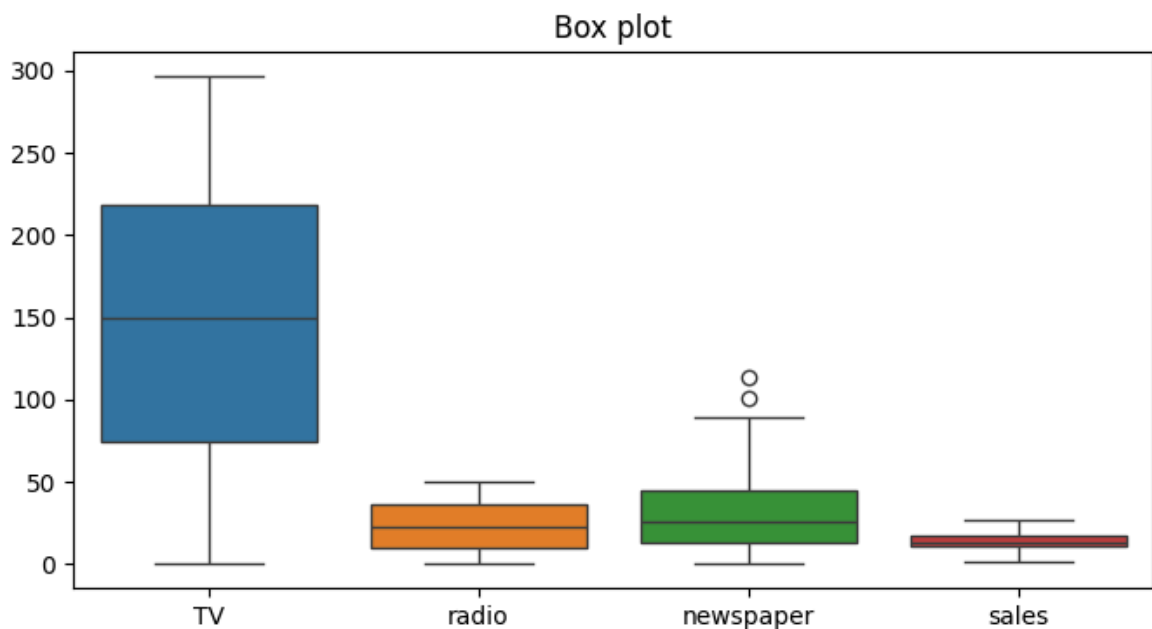
```
In [11]: #6.Scatter plot-TV vs Sales
plt.figure(figsize=(6,4))
sns.scatterplot(x="TV",y="sales",data=df)
plt.title("TV vs Sales")
plt.show()
```



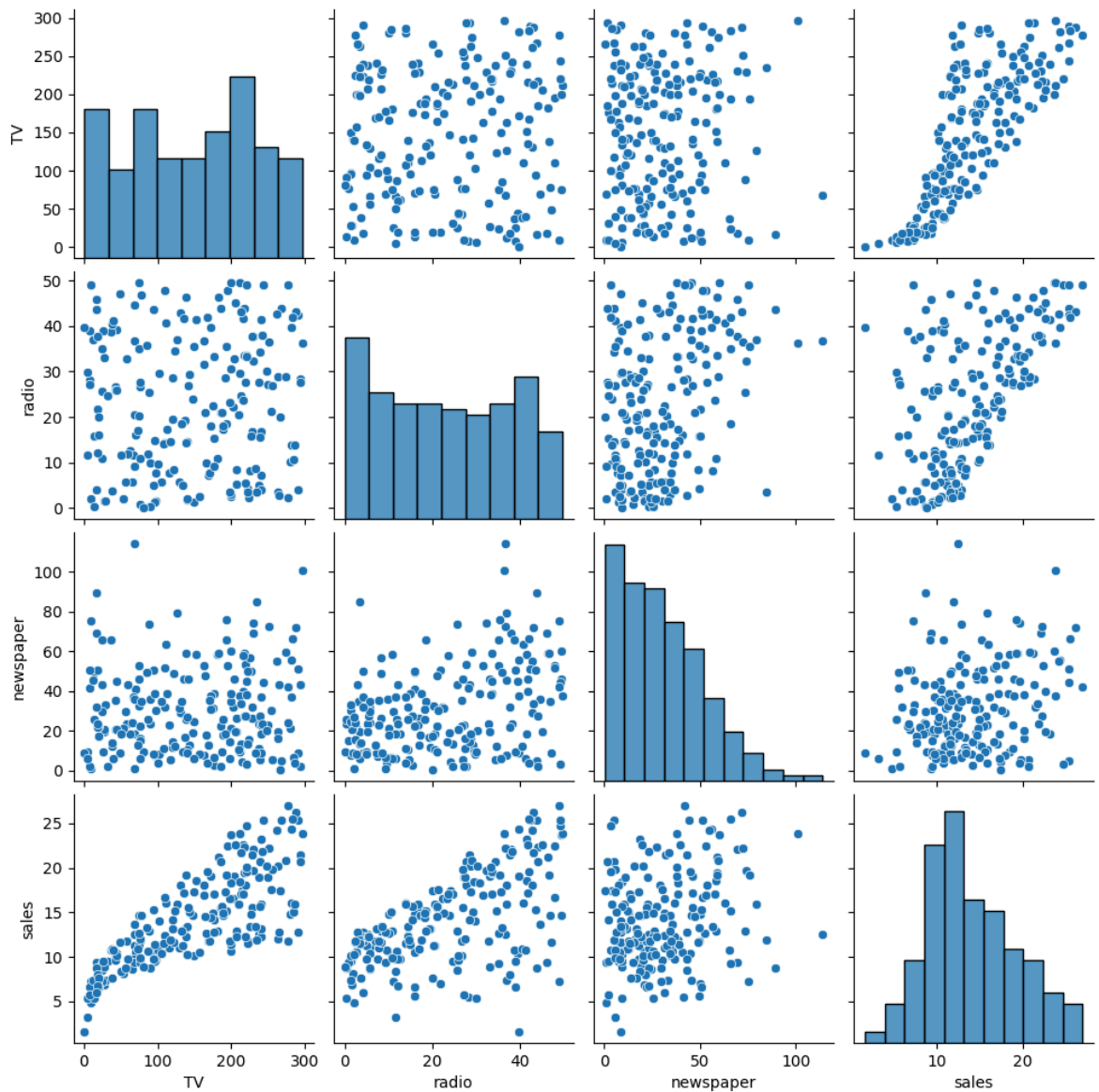
```
In [12]: #7.Scatter plot-Newspaper vs Sales
plt.figure(figsize=(6,4))
sns.scatterplot(x="newspaper",y="sales",data=df)
plt.title("Newspaper vs Sales")
plt.show()
```



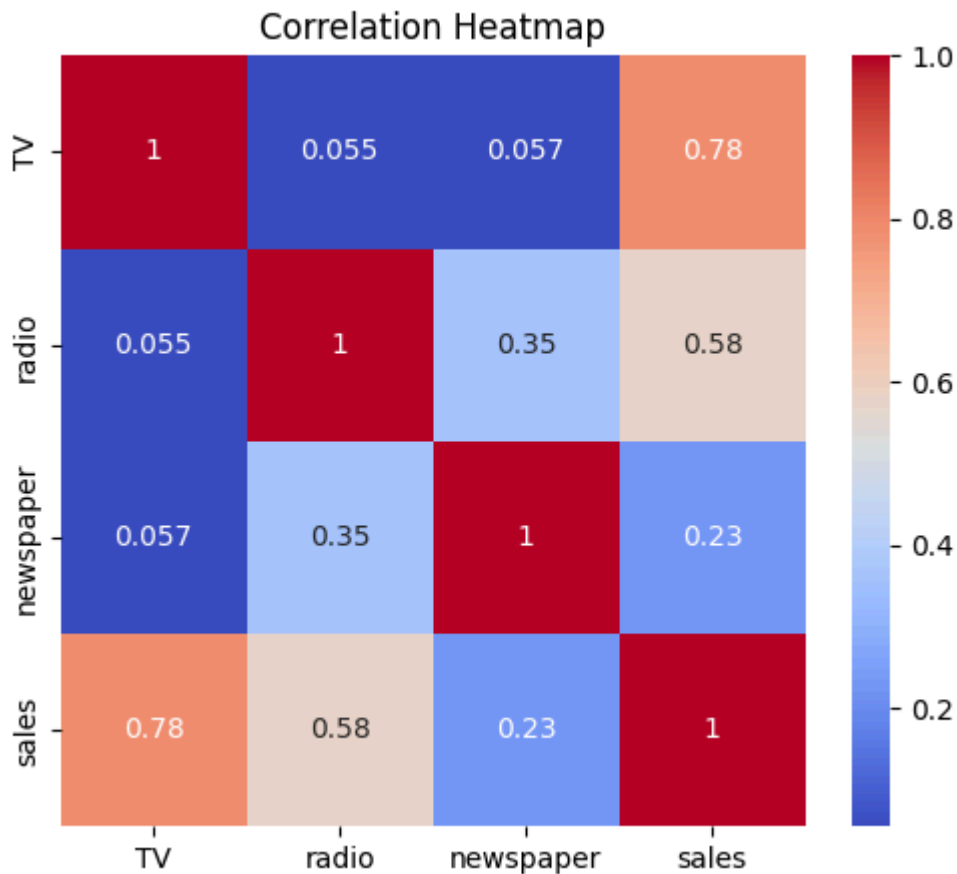
```
In [13]: #8.Box plot
plt.figure(figsize=(8,4))
sns.boxplot(data=df)
plt.title("Box plot")
plt.show()
```



```
In [14]: #9.Pair plot
sns.pairplot(df)
plt.show()
```



```
In [15]: #10.Heatmap
plt.figure(figsize=(6,5))
sns.heatmap(df.corr(),annot=True,cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



4.perform Feature Engineering

Encoding categorical features (if any)

```
In [27]: df.columns
```

```
Out[27]: Index(['TV', 'radio', 'newspaper', 'sales', 'Total_Advertising'], dtype='object')
```

```
In [32]: from sklearn.preprocessing import LabelEncoder, StandardScaler
```

```
categorical_cols=df.select_dtypes(include=['object']).columns
```

```
le=LabelEncoder()
```

```
for col in categorical_cols:
```

```
    df[col]=le.fit_transform(df[col])
```

```
df.head()
```


Out[32]:

	TV	radio	newspaper	sales	Total_Advertising
0	0.969852	0.981522	1.778945	22.1	337.1
1	-1.197376	1.082808	0.669579	10.4	128.9
2	-1.516155	1.528463	1.783549	9.3	132.4
3	0.052050	1.217855	1.286405	18.5	251.3
4	0.394182	-0.841614	1.281802	12.9	250.0

Feature Scaling

```
In [33]: from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()

numeric_cols=["TV", "radio", "newspaper"]
df[numeric_cols]=scaler.fit_transform(df[numeric_cols])
df.head()
```

Out[33]:

	TV	radio	newspaper	sales	Total_Advertising
0	0.969852	0.981522	1.778945	22.1	337.1
1	-1.197376	1.082808	0.669579	10.4	128.9
2	-1.516155	1.528463	1.783549	9.3	132.4
3	0.052050	1.217855	1.286405	18.5	251.3
4	0.394182	-0.841614	1.281802	12.9	250.0

Create New Feature

```
In [21]: df["Total_Advertising"]=(df["TV"]+df["radio"]+df["newspaper"])
df.head()
```

Out[21]:

	TV	radio	newspaper	sales	Total_Advertising
0	230.1	37.8	69.2	22.1	337.1
1	44.5	39.3	45.1	10.4	128.9
2	17.2	45.9	69.3	9.3	132.4
3	151.5	41.3	58.5	18.5	251.3
4	180.8	10.8	58.4	12.9	250.0

In []:

5.Traning & Testing

```
In [34]: x=df.drop("sales",axis=1)
         y=df["sales"]
```

```
In [35]: from sklearn.model_selection import train_test_split

         x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)

         print("x_train",x_train.shape)
         print("x_test",x_test.shape)
         print("y_train",y_train.shape)
         print("y_test",y_test.shape)
```

```
x_train (160, 4)
x_test (40, 4)
y_train (160,)
y_test (40,)
```

Fabric Dataset

1.Load using pandas

```
In [36]: import pandas as pd

         df=pd.read_excel("D:/AI Notes/All Datasets (1)/Fabric data.xlsx")
         df.head()
```

```
Out[36]:
```

	Fabric_length
0	151.2
1	160.3
2	147.5
3	149.2
4	159.2

2.perform Data Cleaning

```
In [37]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25 entries, 0 to 24
Data columns (total 1 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Fabric_length  25 non-null    float64
dtypes: float64(1)
memory usage: 332.0 bytes
```

```
In [38]: df.isnull().sum()
```

```
Out[38]: Fabric_length    0  
dtype: int64
```

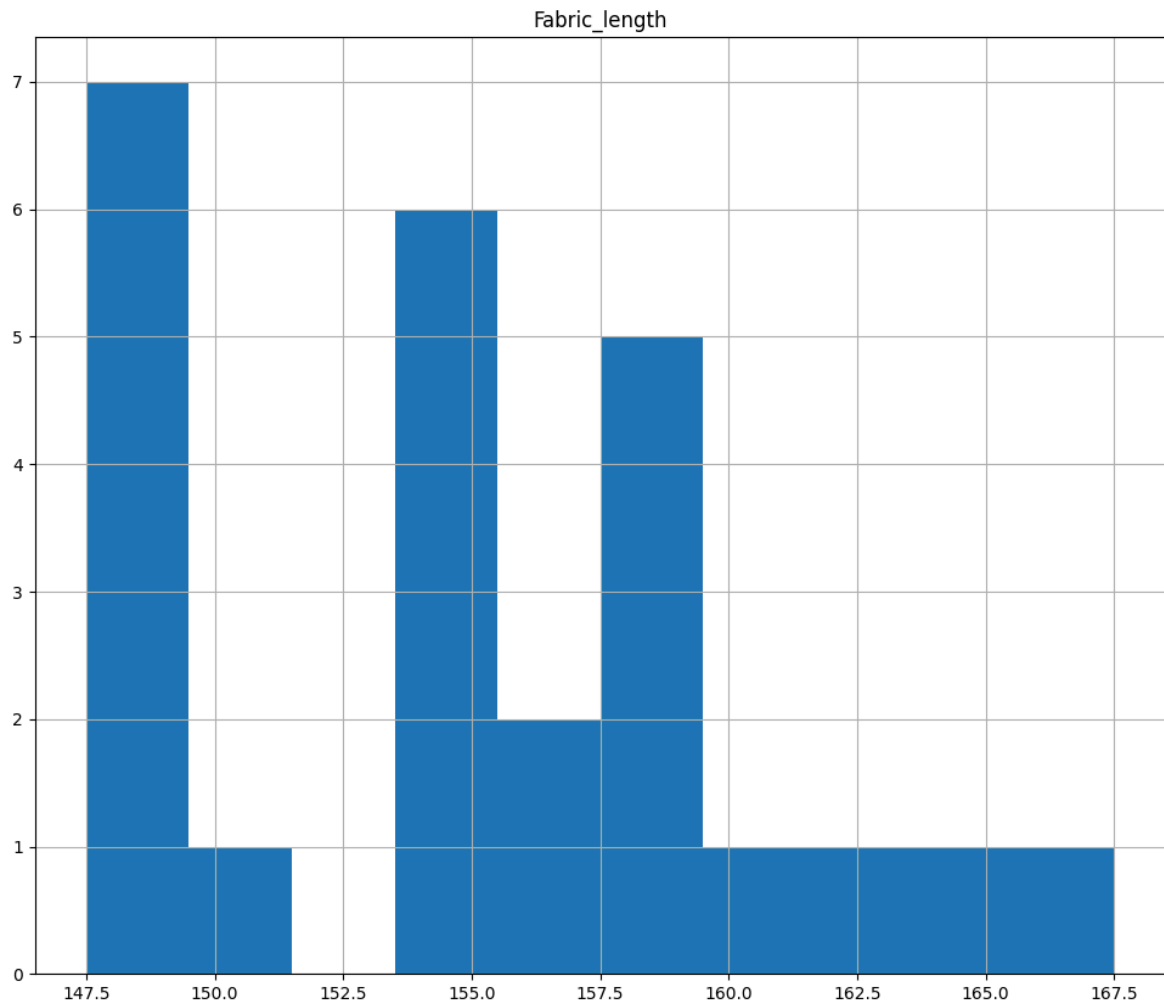
```
In [39]: df.describe()
```

```
Out[39]:
```

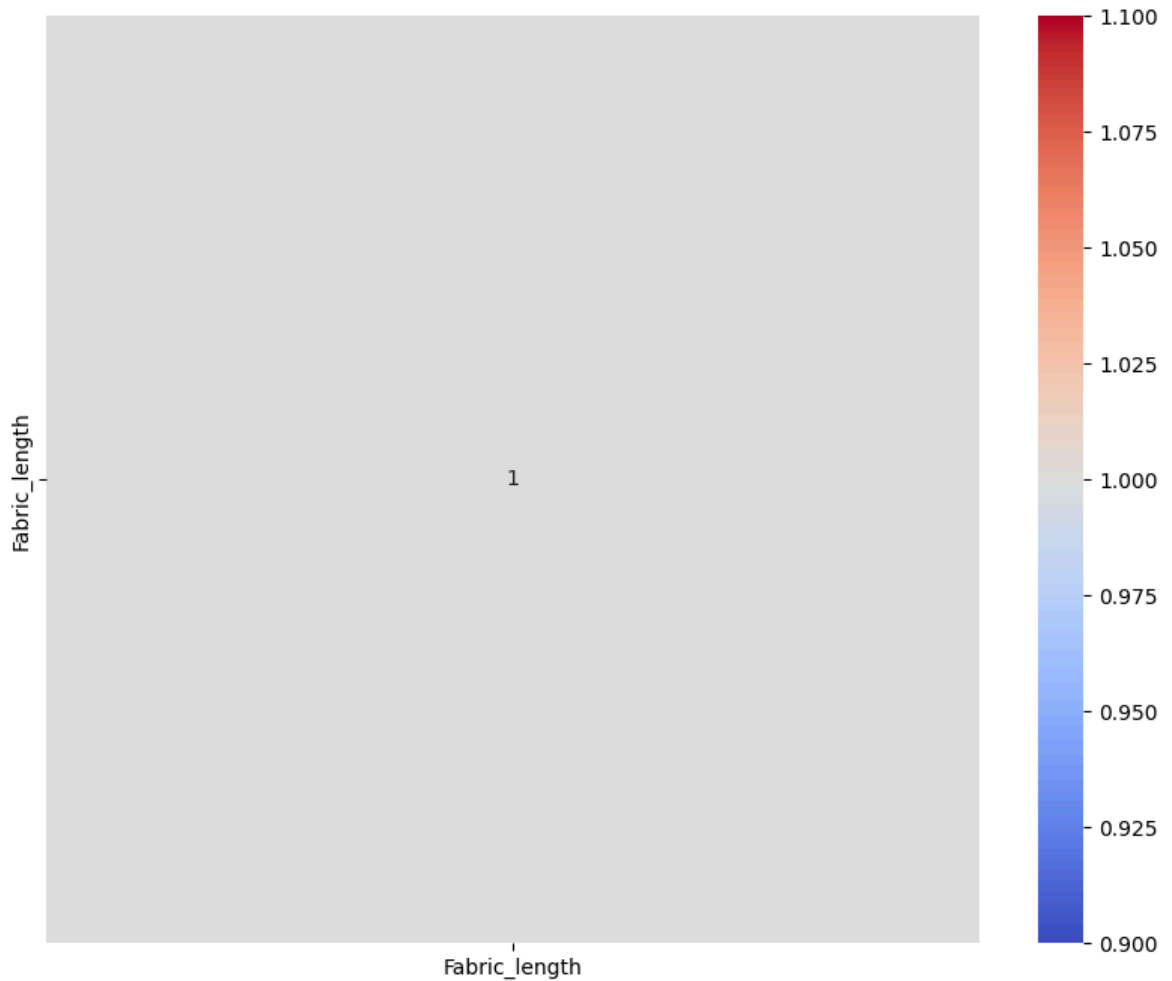
	Fabric_length
count	25.00000
mean	155.06400
std	5.64047
min	147.50000
25%	149.20000
50%	155.20000
75%	158.40000
max	167.50000

3.perform Exploratory Data Analysis(EDA)

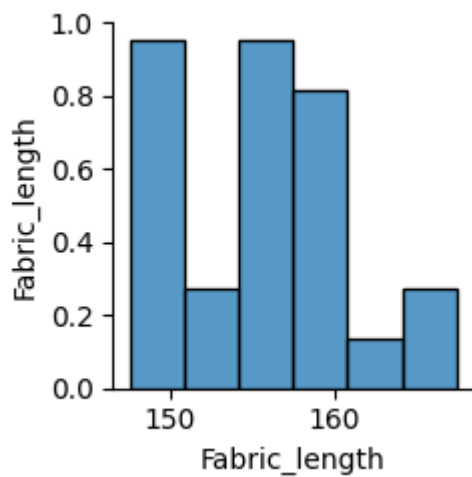
```
In [40]: import seaborn as sns  
import matplotlib.pyplot as plt  
  
#1.Histogram  
df.hist(figsize=(12,10))  
plt.show()
```



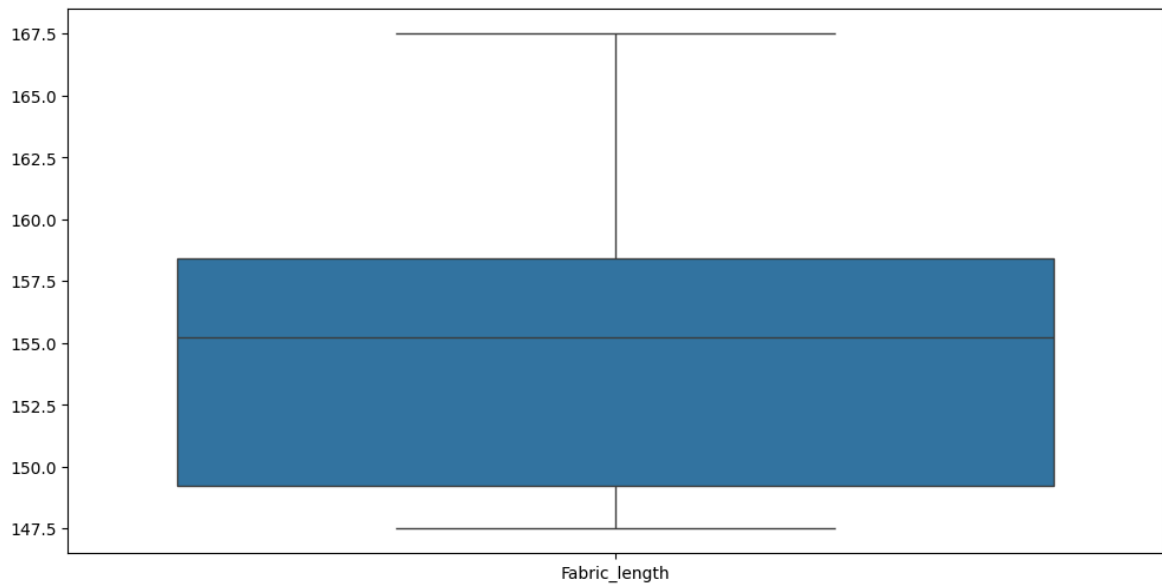
```
In [41]: #2. Correlation Heatmap
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.show()
```



```
In [42]: #3.Pair Plot  
sns.pairplot(df)  
plt.show()
```

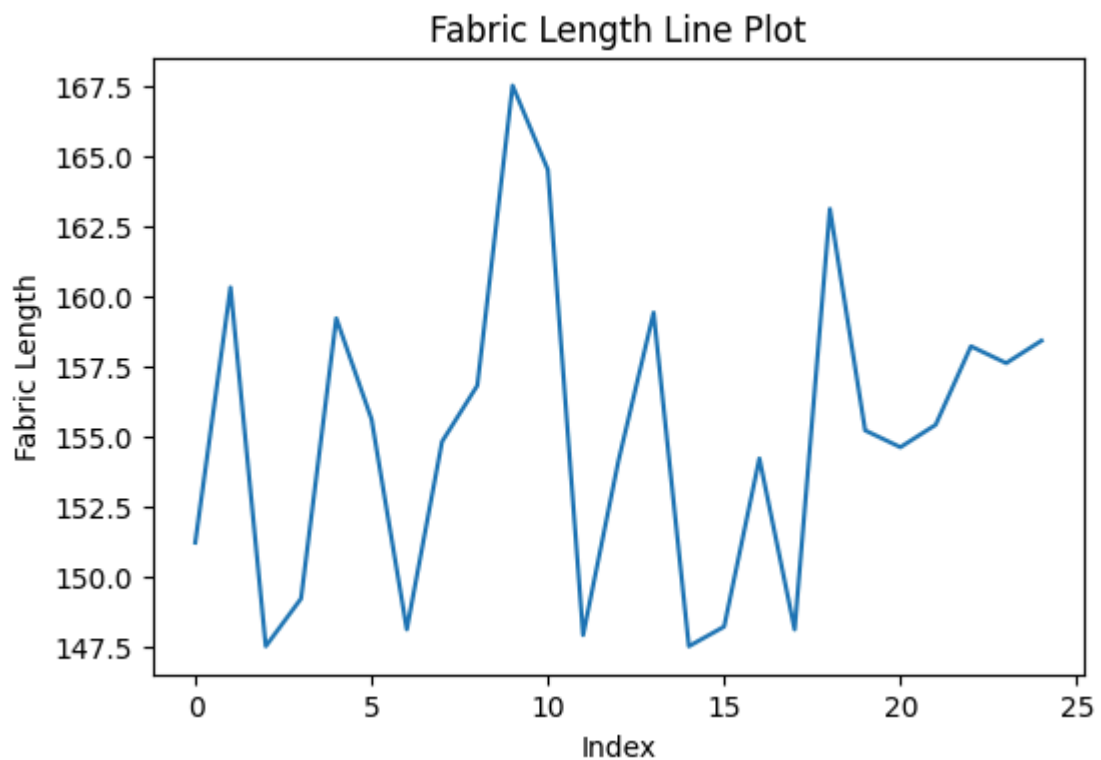


```
In [43]: #4.Box Plot  
plt.figure(figsize=(12,6))  
sns.boxplot(data=df.select_dtypes(include=['number']))  
plt.show()
```



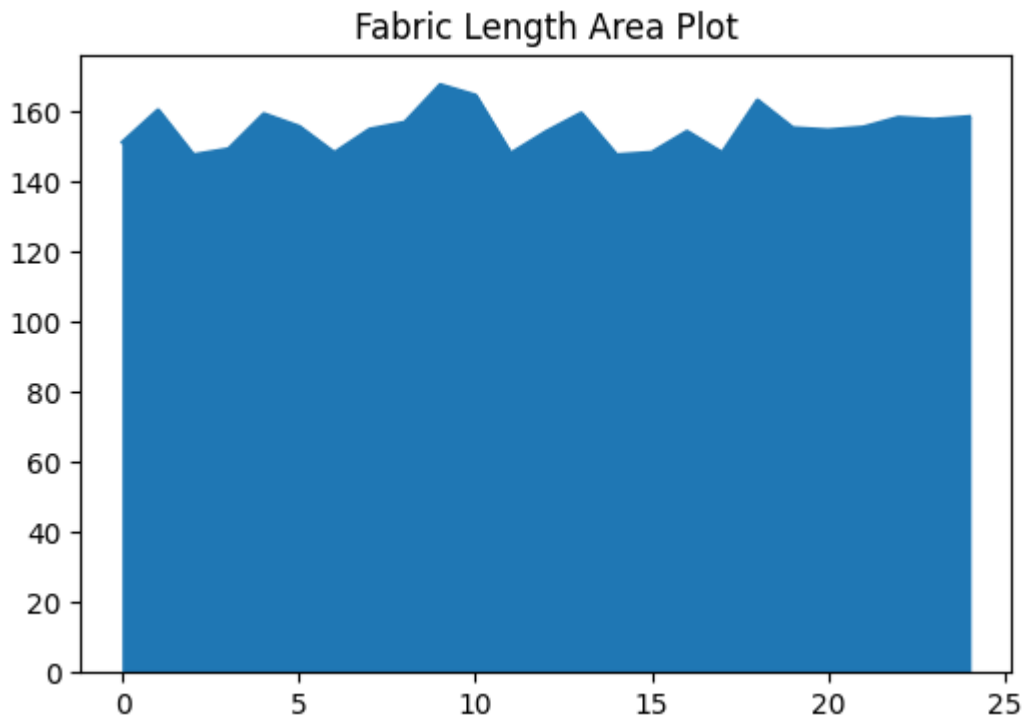
In [48]: *#5.Line plot*

```
plt.figure(figsize=(6,4))
plt.plot(df["Fabric_length"])
plt.title("Fabric Length Line Plot")
plt.xlabel("Index")
plt.ylabel("Fabric Length")
plt.show()
```

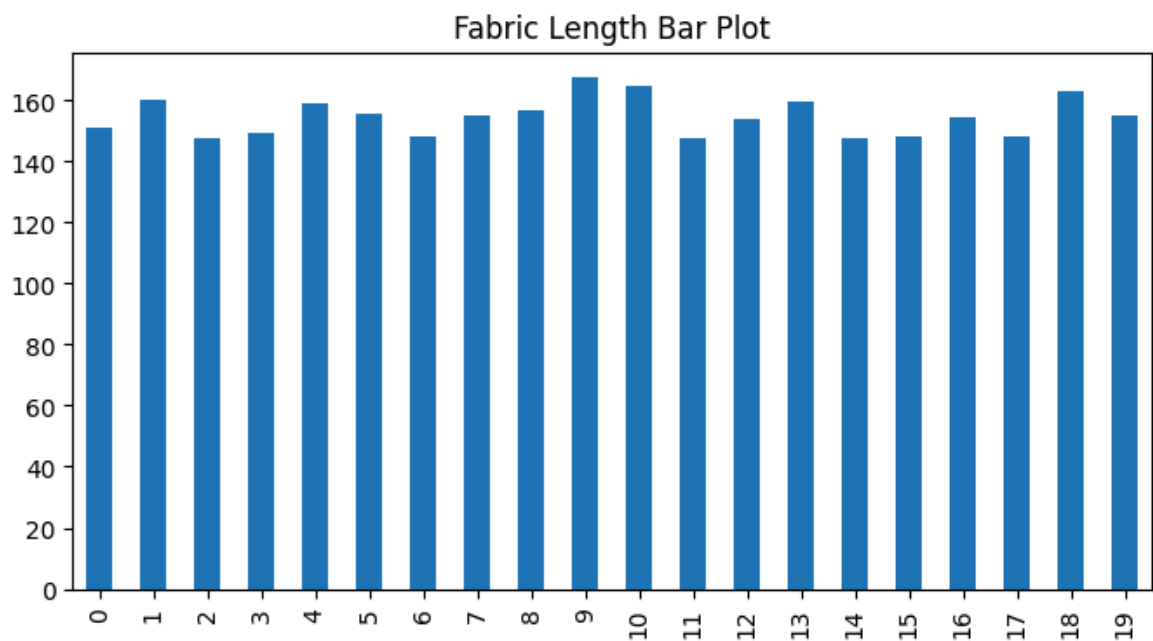


In [49]: *#6.Area plot*

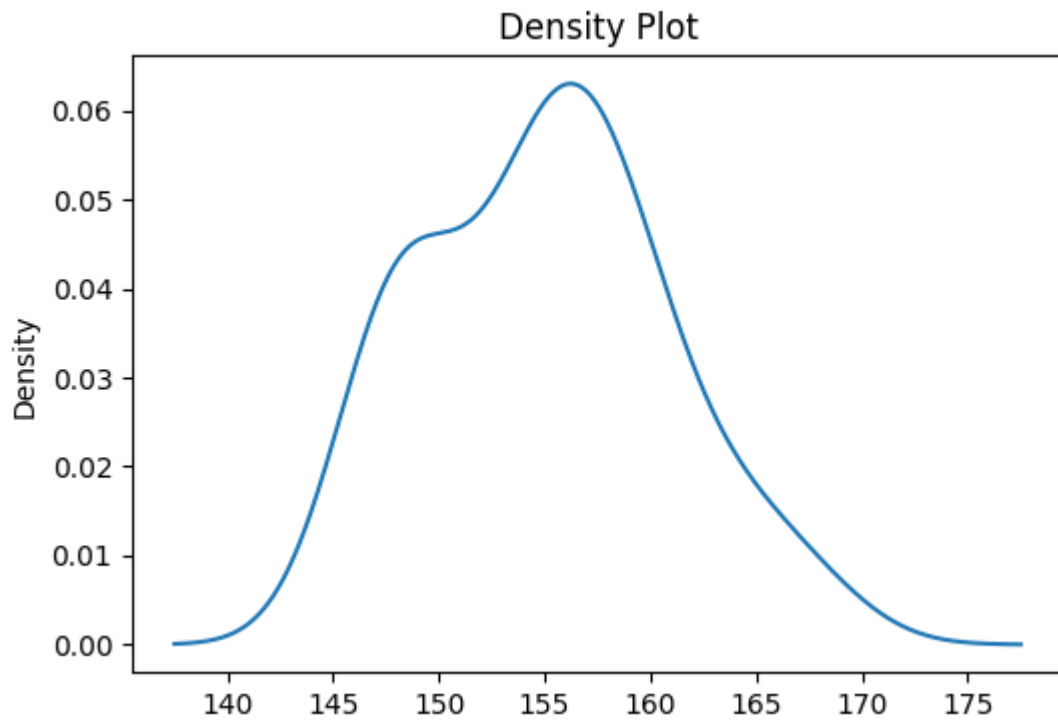
```
plt.figure(figsize=(6,4))
df["Fabric_length"].plot(kind="area")
plt.title("Fabric Length Area Plot")
plt.show()
```



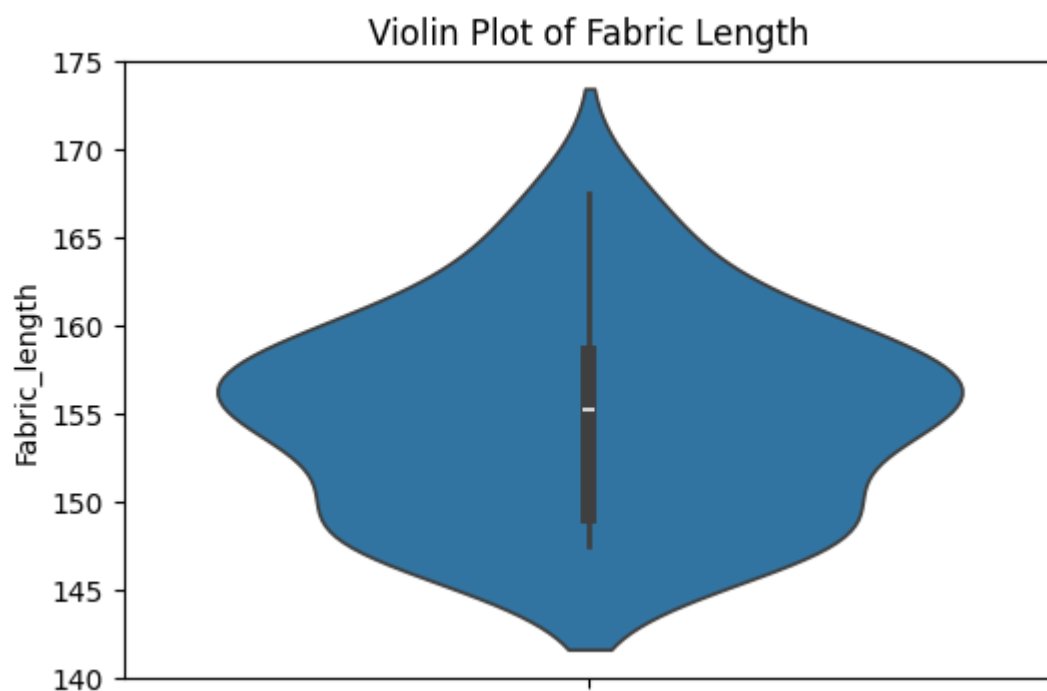
```
In [50]: #7.Bar Plot
plt.figure(figsize=(8,4))
df["Fabric_length"].head(20).plot(kind="bar")
plt.title("Fabric Length Bar Plot")
plt.show()
```



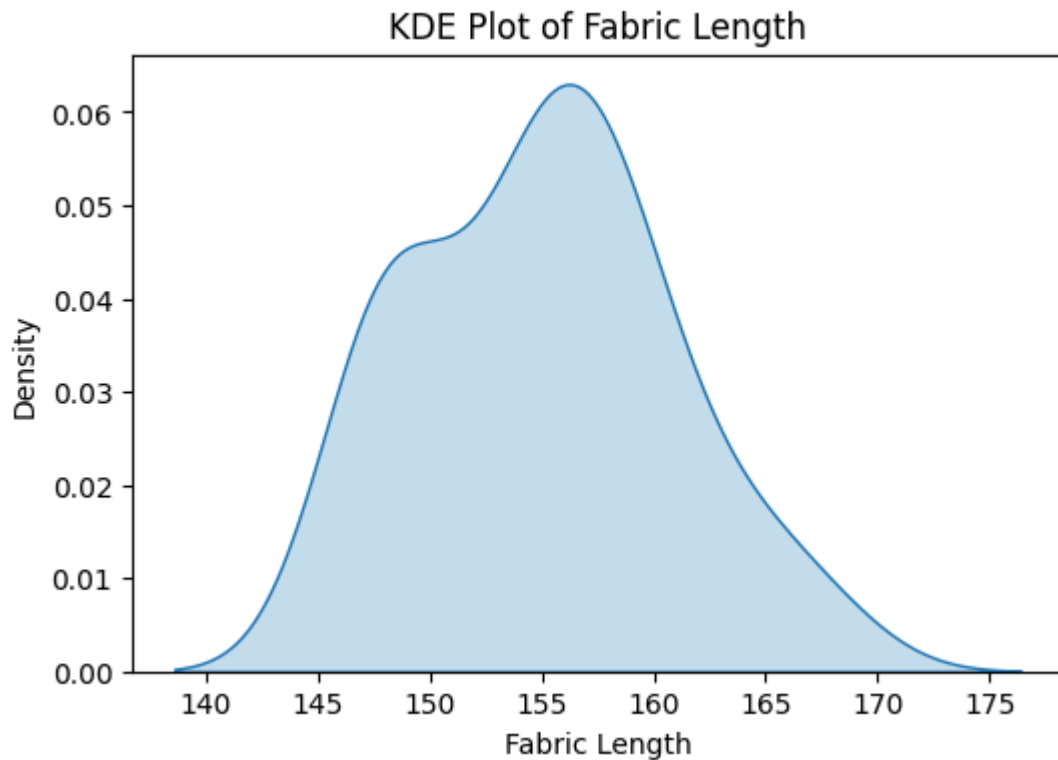
```
In [52]: #8.Density Plot
plt.figure(figsize=(6,4))
df["Fabric_length"].plot(kind="density")
plt.title("Density Plot")
plt.show()
```



```
In [53]: #9.violin Plot
plt.figure(figsize=(6,4))
sns.violinplot(y=df["Fabric_length"])
plt.title("Violin Plot of Fabric Length")
plt.show()
```



```
In [54]: #10.KDE Plot
plt.figure(figsize=(6,4))
sns.kdeplot(data=df,x="Fabric_length",fill=True)
plt.title("KDE Plot of Fabric Length")
plt.xlabel("Fabric Length")
plt.show()
```

4.perform Feature Engineering

Encoding categorical features (if any)

```
In [56]: from sklearn.preprocessing import LabelEncoder

categorical_cols=df.select_dtypes(include=['object']).columns

le=LabelEncoder()
for col in categorical_cols:
    df[col]=le.fit_transform(df[col])
df.head()
```

```
Out[56]:
```

	Fabric_length	Index
0	151.2	0
1	160.3	1
2	147.5	2
3	149.2	3
4	159.2	4

Feature Scaling

```
In [58]: from sklearn.preprocessing import StandardScaler

scaler=StandardScaler()
```

```
numeric_cols=df.select_dtypes(include=['number']).columns

df[numeric_cols]=scaler.fit_transform(df[numeric_cols])
df.head()
```

Out[58]:

	Fabric_length	Index
0	-0.699176	-1.664101
1	0.947434	-1.525426
2	-1.368676	-1.386750
3	-1.061068	-1.248075
4	0.748393	-1.109400

Create New Feature

In [59]:

```
df["New_Feature"]=df[numeric_cols[0]]+df[numeric_cols[1]]
df.head()
```

Out[59]:

	Fabric_length	Index	New_Feature
0	-0.699176	-1.664101	-2.363276
1	0.947434	-1.525426	-0.577992
2	-1.368676	-1.386750	-2.755426
3	-1.061068	-1.248075	-2.309143
4	0.748393	-1.109400	-0.361008

5.Traning & Testing

In [55]:

```
import pandas as pd
from sklearn.model_selection import train_test_split

df=pd.read_excel("D:/AI Notes/All Datasets (1)/Fabric data.xlsx")

df["Index"]=range(len(df))

x=df[["Index"]]
y=df["Fabric_length"]

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)

print("x_train",x_train.shape)
print("x_test",x_test.shape)
print("y_train",y_train.shape)
print("y_test",y_test.shape)
```

```
x_train (20, 1)
x_test (5, 1)
y_train (20,)
y_test (5,)
```

Home Prices

1.Load using pandas

```
In [60]: import pandas as pd

df=pd.read_csv("D:/AI Notes/All Datasets (1)/homeprices.csv")
df.head()
```

```
Out[60]:
```

	town	area	price
0	Chennai	2600	5500000
1	Chennai	3000	5650000
2	Chennai	3200	6100000
3	Chennai	3600	6800000
4	Bangalore	2600	5850000

2.perform Data Cleaning

```
In [61]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0   town    12 non-null      object
1   area    12 non-null      int64
2   price   12 non-null      int64
dtypes: int64(2), object(1)
memory usage: 420.0+ bytes
```

```
In [62]: df.describe()
```

Out[62]:

	area	price
count	12.000000	1.200000e+01
mean	3075.000000	6.212500e+06
std	391.094037	5.214163e+05
min	2600.000000	5.500000e+06
25%	2750.000000	5.825000e+06
50%	3050.000000	6.125000e+06
75%	3375.000000	6.575000e+06
max	3600.000000	7.100000e+06

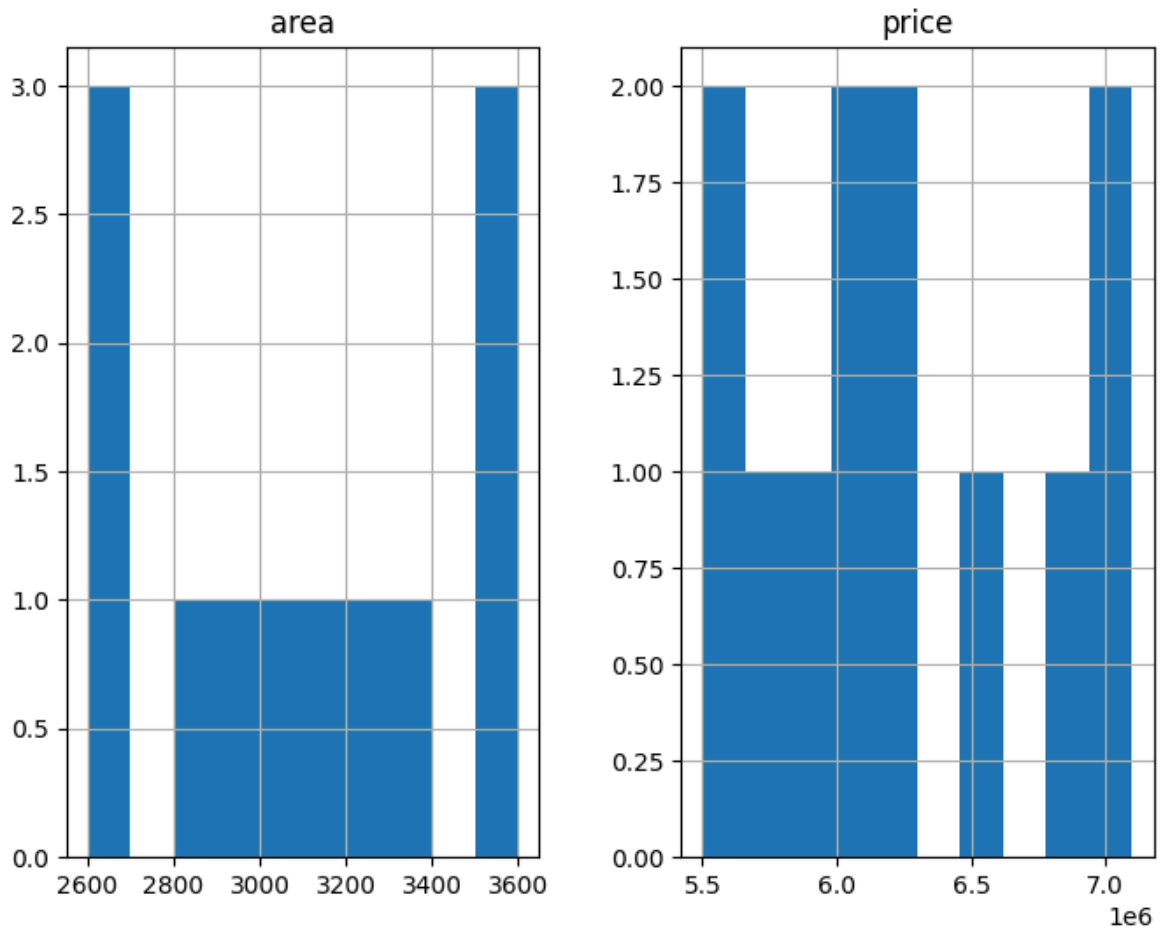
In [63]: `df.isnull().sum()`

```
Out[63]: town      0
         area      0
         price     0
         dtype: int64
```

3.perform Exploratory Data Analysis(EDA)

```
In [65]: import matplotlib.pyplot as plt
import seaborn as sns

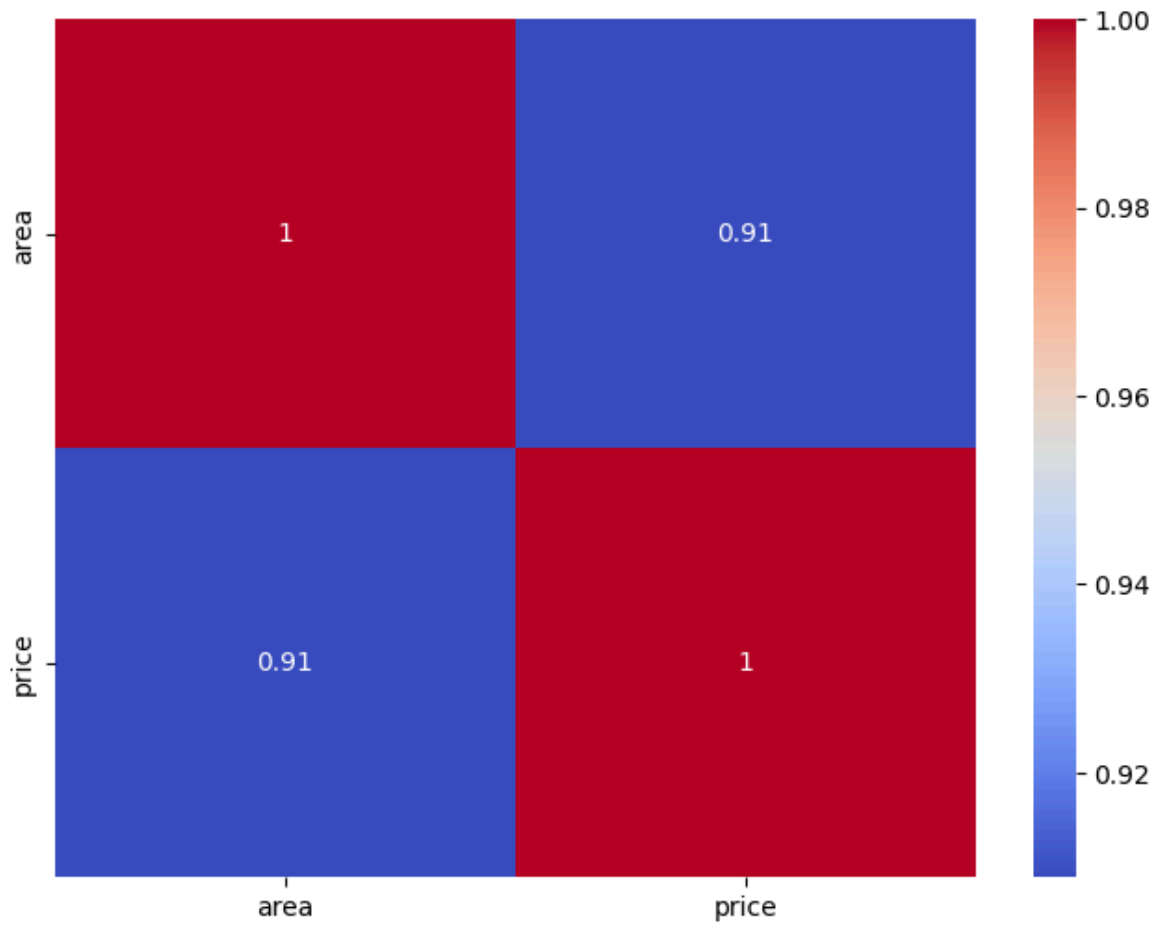
#1.Histogram
df.hist(figsize=(8,6))
plt.show()
```



```
In [66]: #2. Correlation Heatmap

import matplotlib.pyplot as plt
import seaborn as sns

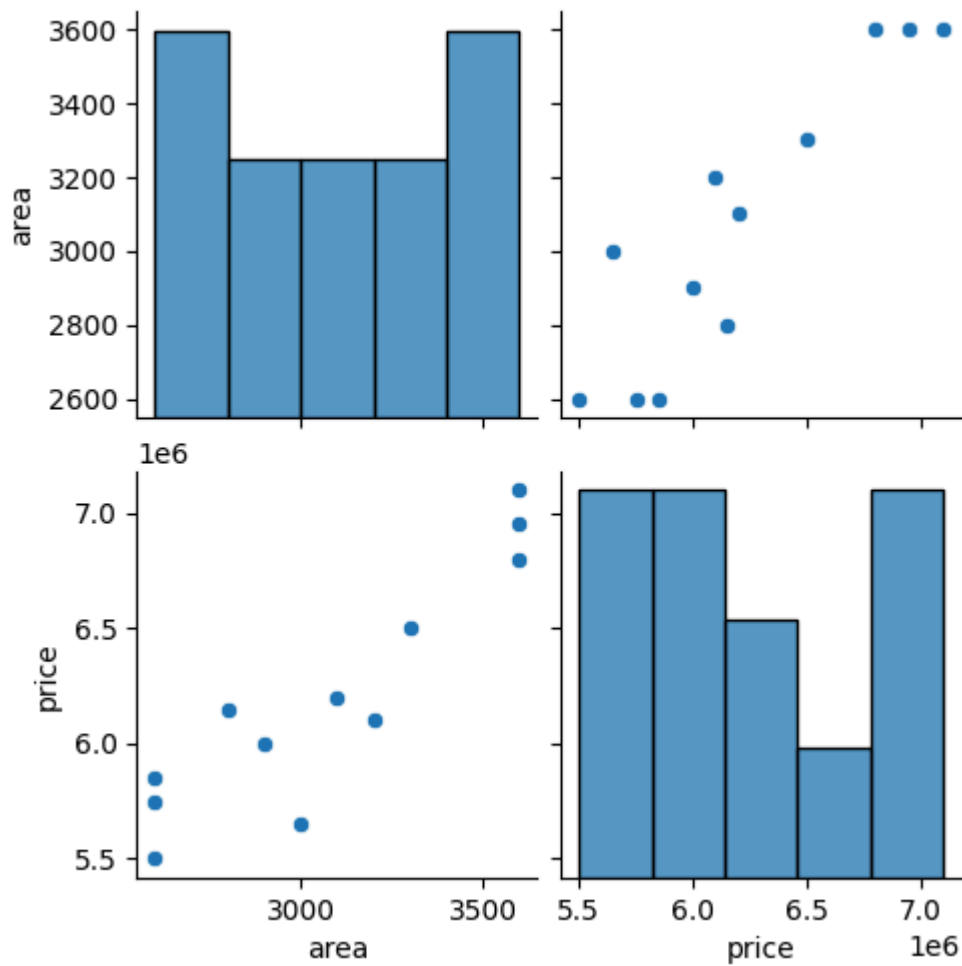
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.show()
```



```
In [67]: #3.Pair Plot

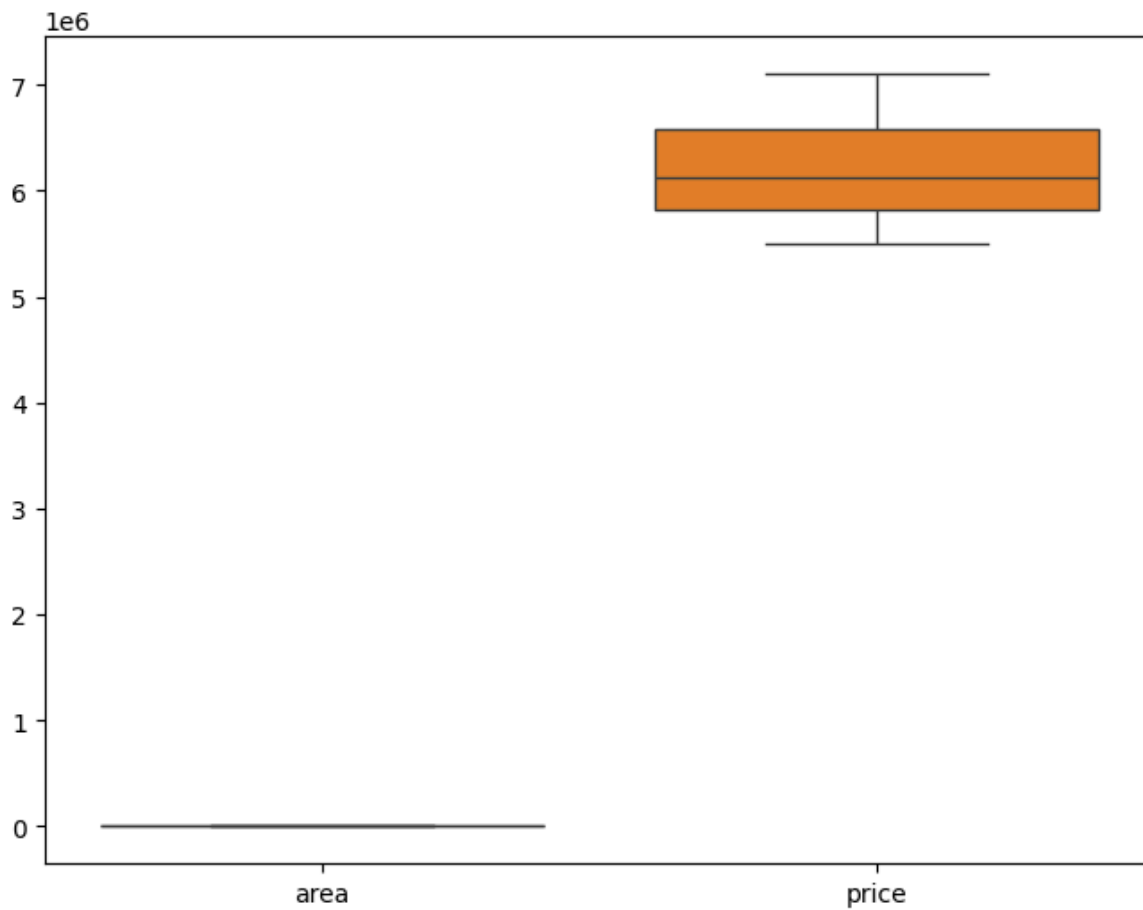
import matplotlib.pyplot as plt
import seaborn as sns

sns.pairplot(df)
plt.show()
```



In [68]: *#4.Box Plot*

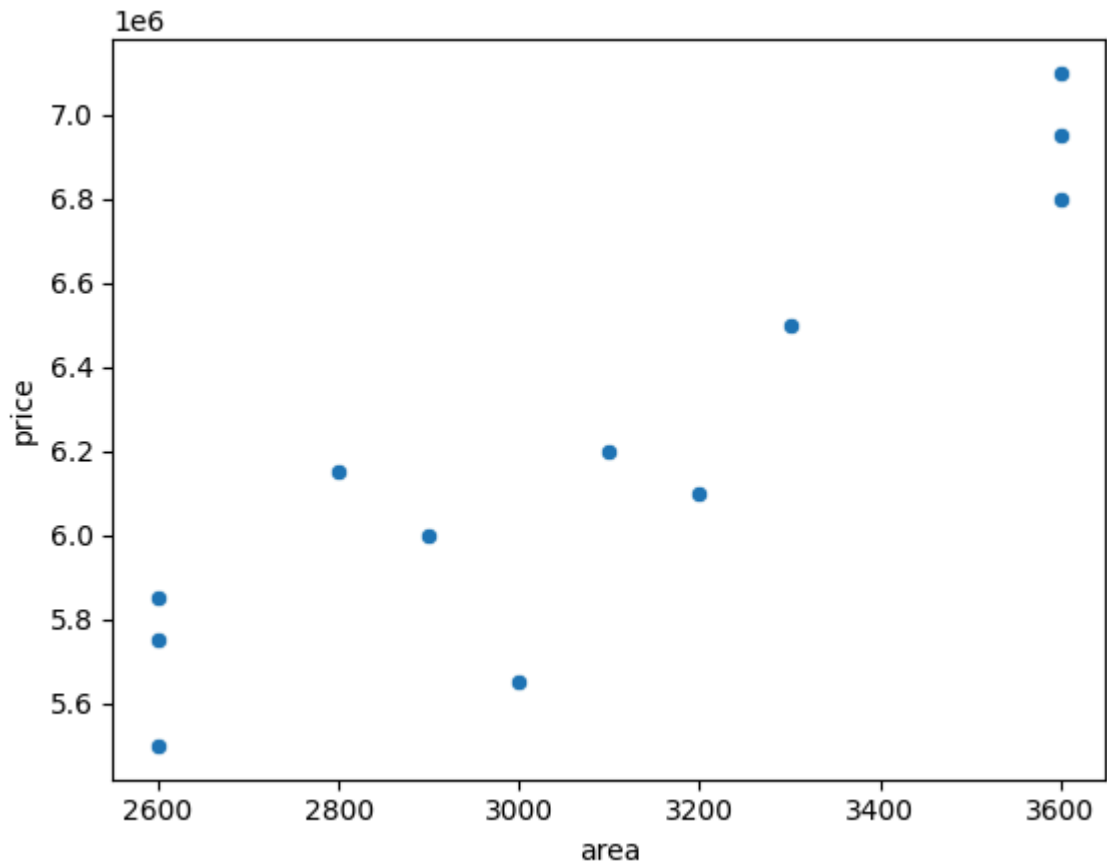
```
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(8,6))
sns.boxplot(data=df.select_dtypes(include=['number']))
plt.show()
```



In [69]: *#5.Scatter Plot*

```
import matplotlib.pyplot as plt
import seaborn as sns

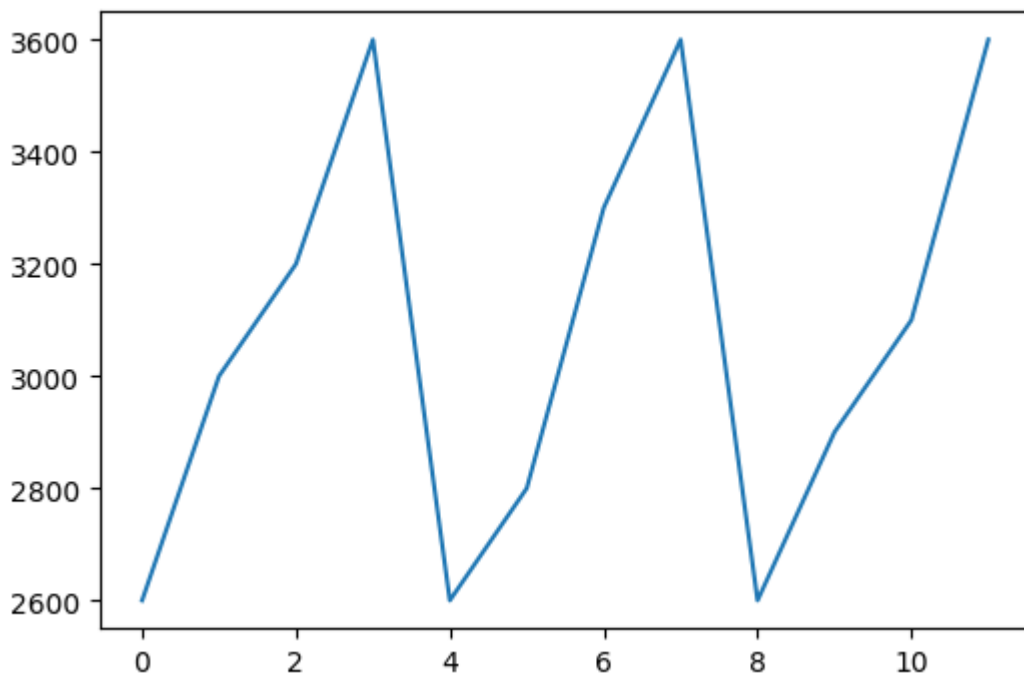
sns.scatterplot(x=df.select_dtypes(include=['number']).columns[0], y=df.select_dtypes(include=['number']).columns[1])
plt.show()
```

In [70]: #6. Line Plot

```
import matplotlib.pyplot as plt
import seaborn as sns

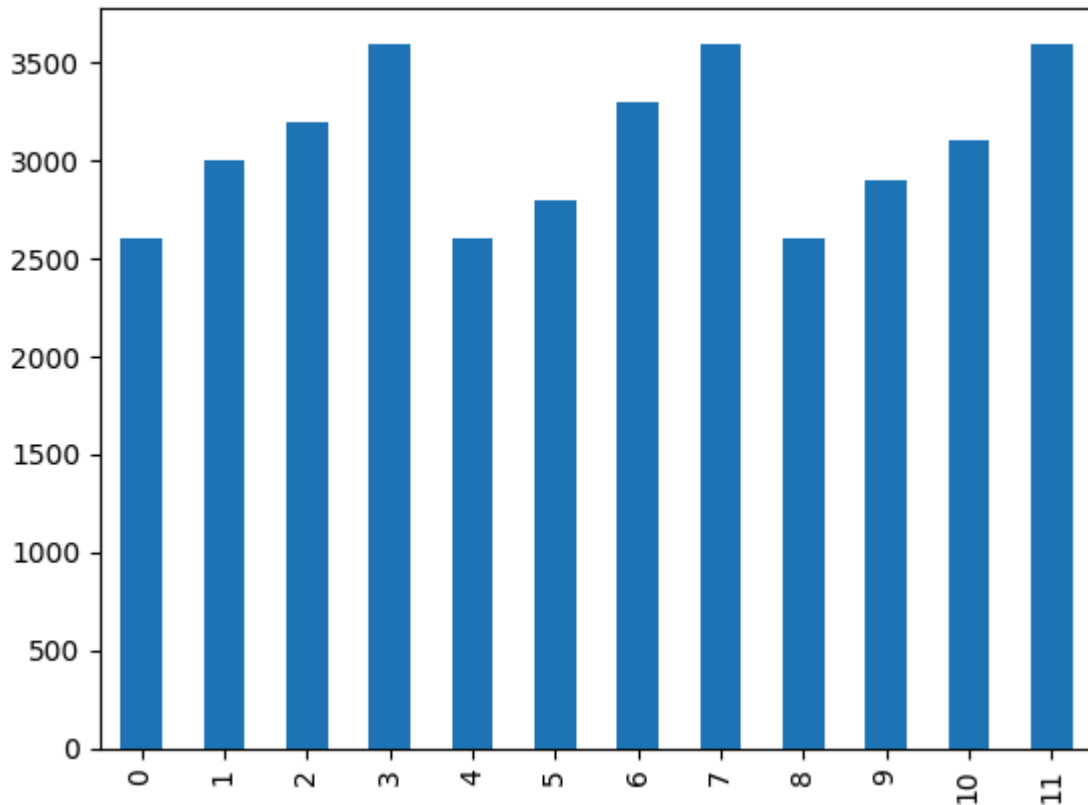
plt.figure(figsize=(6,4))
plt.plot(df.select_dtypes(include=['number']).iloc[:,0])
plt.show()
```



In [71]: #7. Bar Plot

```
import matplotlib.pyplot as plt
import seaborn as sns

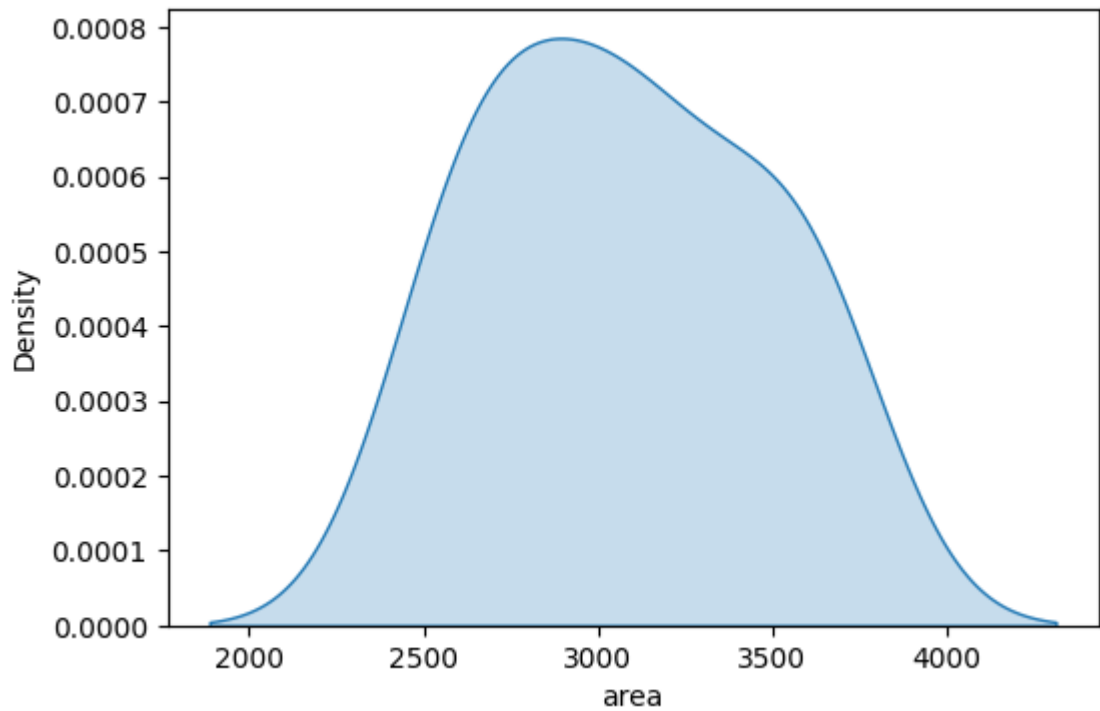
df.select_dtypes(include=['number']).iloc[:20,0].plot(kind='bar')
plt.show()
```



In [72]: #8.KDE plot

```
import matplotlib.pyplot as plt
import seaborn as sns

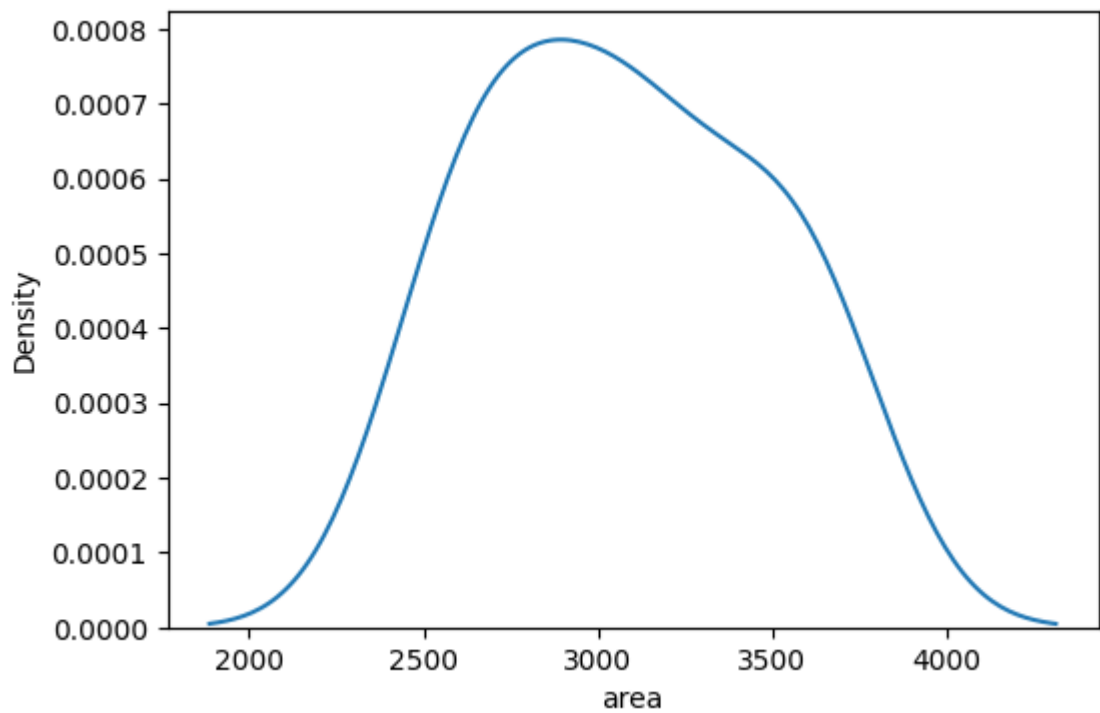
plt.figure(figsize=(6,4))
sns.kdeplot(df.select_dtypes(include=['number']).iloc[:,0],fill=True)
plt.show()
```



In [73]: *#9.Violin Plot*

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6,4))
sns.kdeplot(df.select_dtypes(include=['number']).iloc[:,0])
plt.show()
```

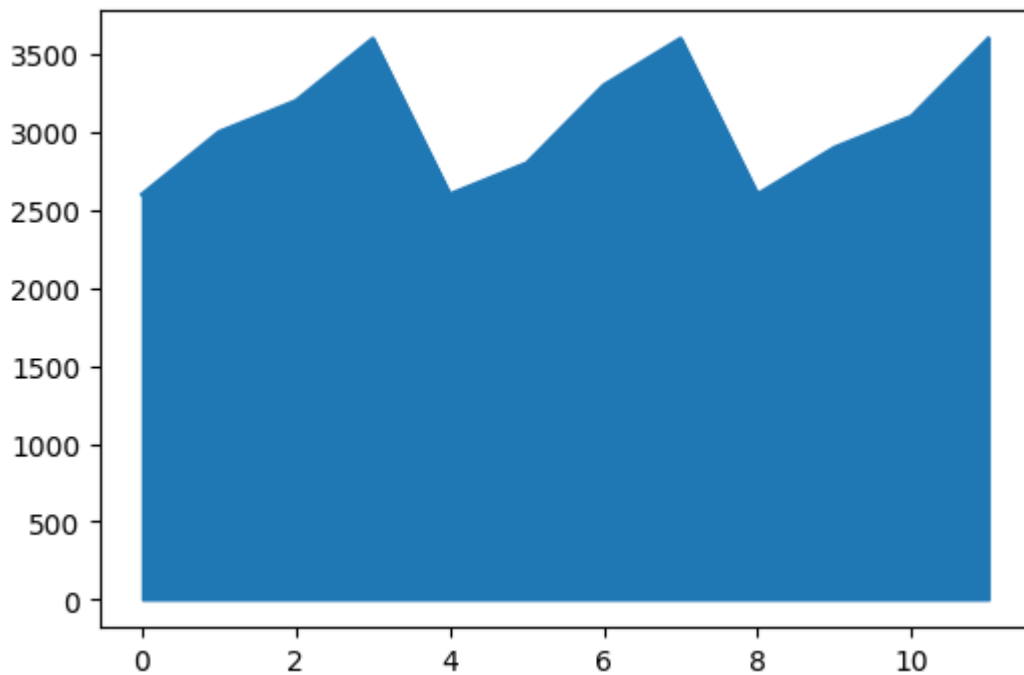


In [74]: *#10.Area Plot*

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6,4))
```

```
df.select_dtypes(include=['number']).iloc[:,0].plot(kind='area')
plt.show()
```



4.perform Feature Engineering

Encoding categorical features (if any)

```
In [75]: from sklearn.preprocessing import LabelEncoder

categorical_cols=df.select_dtypes(include=['object']).columns

le=LabelEncoder()
for col in categorical_cols:
    df[col]=le.fit_transform(df[col])
df.head()
```

```
Out[75]:
```

	town	area	price
0	1	2600	5500000
1	1	3000	5650000
2	1	3200	6100000
3	1	3600	6800000
4	0	2600	5850000

Feature Scaling

```
In [76]: from sklearn.preprocessing import StandardScaler
```

```

scaler=StandardScaler()

numeric_cols=df.select_dtypes(include=['number']).columns

df[numeric_cols]=scaler.fit_transform(df[numeric_cols])
df.head()

```

Out[76]:

	town	area	price
0	0.000000	-1.268547	-1.427232
1	0.000000	-0.200297	-1.126762
2	0.000000	0.333828	-0.225352
3	0.000000	1.402079	1.176840
4	-1.224745	-1.268547	-0.726135

Create New Feature

```

In [79]: df["arice_per_area"]=df["price"]/df["area"]
df.head()

```

Out[79]:

	town	area	price	arice_per_area
0	0.000000	-1.268547	-1.427232	1.125091
1	0.000000	-0.200297	-1.126762	5.625457
2	0.000000	0.333828	-0.225352	-0.675055
3	0.000000	1.402079	1.176840	0.839354
4	-1.224745	-1.268547	-0.726135	0.572415

5.Traning & Testing

```

In [80]: from sklearn.model_selection import train_test_split

x=df.drop("price",axis=1)
y=df["price"]

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)

print("x_train:",x_train.shape)
print("x_test:",x_test.shape)
print("y_train:",y_train.shape)
print("y_test:",y_test.shape)

```

```

x_train: (9, 3)
x_test: (3, 3)
y_train: (9,)
y_test: (3,)

```

Mushrooms

1.Load using pandas

```
In [81]: import pandas as pd

df=pd.read_csv("D:/AI Notes/All Datasets (1)/mushrooms.csv")
df.head()
```

Out[81]:

	class	cap- shape	cap- surface	cap- color	bruises	odor	gill- attachment	gill- spacing	gill- size	gill- color	...	su t
0	p	x	s	n	t	p	f	c	n	k	...	
1	e	x	s	y	t	a	f	c	b	k	...	
2	e	b	s	w	t	l	f	c	b	n	...	
3	p	x	y	w	t	p	f	c	n	n	...	
4	e	x	s	g	f	n	f	w	b	k	...	

5 rows × 23 columns



2.perform Data Cleaning

```
In [82]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8124 entries, 0 to 8123
Data columns (total 23 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   class                                8124 non-null   object
1   cap-shape                            8124 non-null   object
2   cap-surface                          8124 non-null   object
3   cap-color                           8124 non-null   object
4   bruises                             8124 non-null   object
5   odor                                8124 non-null   object
6   gill-attachment                     8124 non-null   object
7   gill-spacing                        8124 non-null   object
8   gill-size                           8124 non-null   object
9   gill-color                          8124 non-null   object
10  stalk-shape                         8124 non-null   object
11  stalk-root                          8124 non-null   object
12  stalk-surface-above-ring            8124 non-null   object
13  stalk-surface-below-ring            8124 non-null   object
14  stalk-color-above-ring              8124 non-null   object
15  stalk-color-below-ring              8124 non-null   object
16  veil-type                           8124 non-null   object
17  veil-color                          8124 non-null   object
18  ring-number                         8124 non-null   object
19  ring-type                           8124 non-null   object
20  spore-print-color                   8124 non-null   object
21  population                          8124 non-null   object
22  habitat                             8124 non-null   object
dtypes: object(23)
memory usage: 1.4+ MB

```

In [83]: `df.describe(include='all')`

Out[83]:

	class	cap- shape	cap- surface	cap- color	bruises	odor	gill- attachment	gill- spacing	gill- size	gill- color
count	8124	8124	8124	8124	8124	8124	8124	8124	8124	8124
unique	2	6	4	10	2	9	2	2	2	12
top	e	x	y	n	f	n	f	c	b	b
freq	4208	3656	3244	2284	4748	3528	7914	6812	5612	1728

4 rows × 23 columns



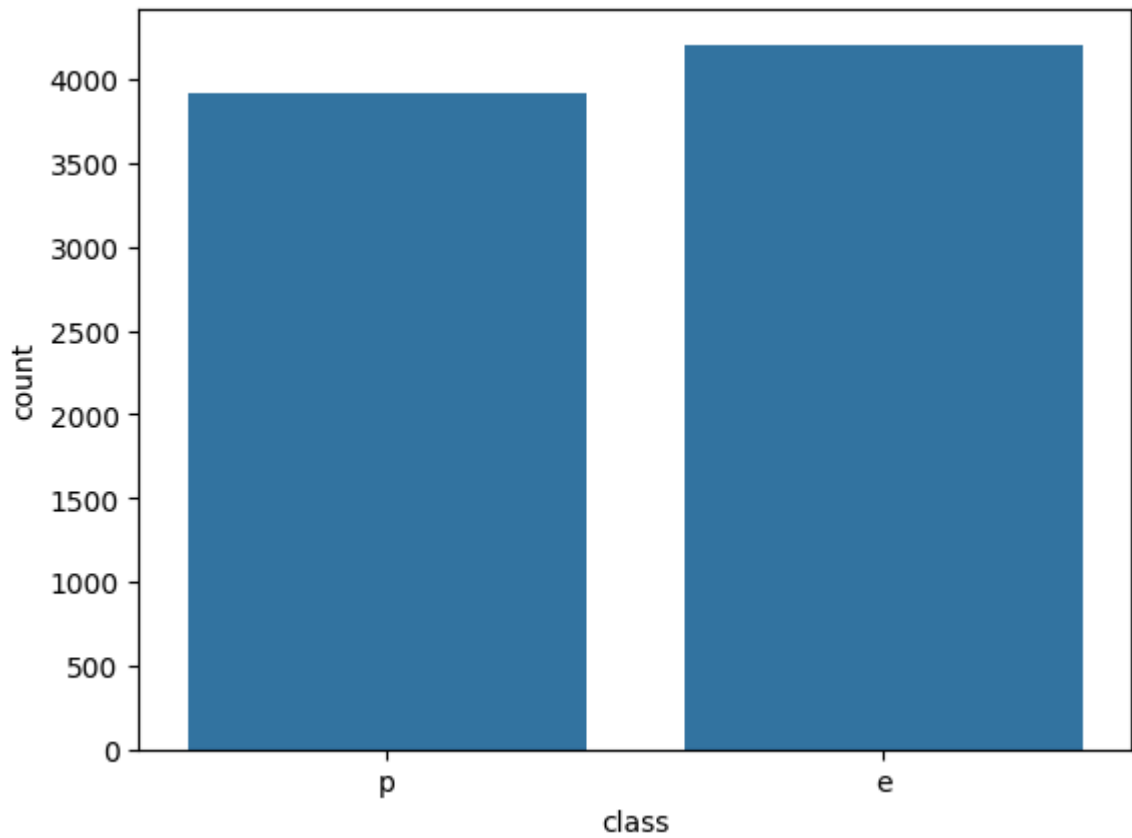
In [84]: `df.isnull().sum()`

```
Out[84]: class 0
         cap-shape 0
         cap-surface 0
         cap-color 0
         bruises 0
         odor 0
         gill-attachment 0
         gill-spacing 0
         gill-size 0
         gill-color 0
         stalk-shape 0
         stalk-root 0
         stalk-surface-above-ring 0
         stalk-surface-below-ring 0
         stalk-color-above-ring 0
         stalk-color-below-ring 0
         veil-type 0
         veil-color 0
         ring-number 0
         ring-type 0
         spore-print-color 0
         population 0
         habitat 0
         dtype: int64
```

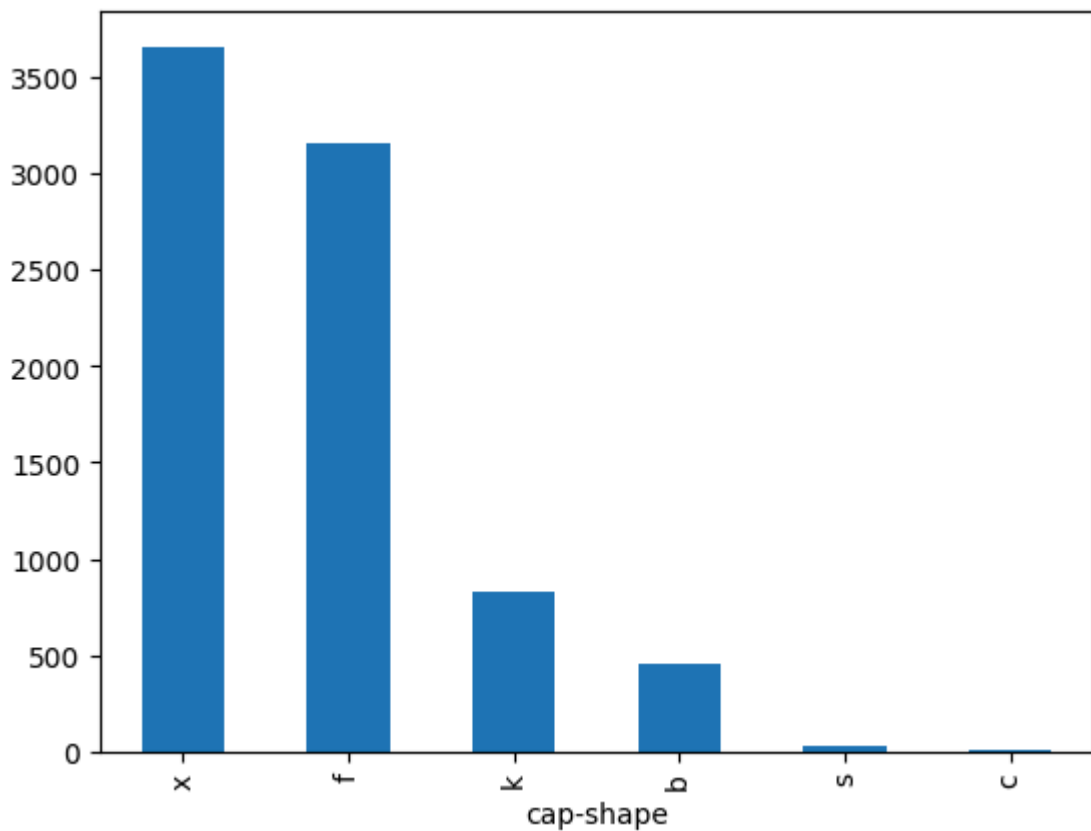
3.perform Exploratory Data Analysis(EDA)

```
In [85]: import matplotlib.pyplot as plt
         import seaborn as sns

         #1.Count Plot
         sns.countplot(x=df["class"])
         plt.show()
```

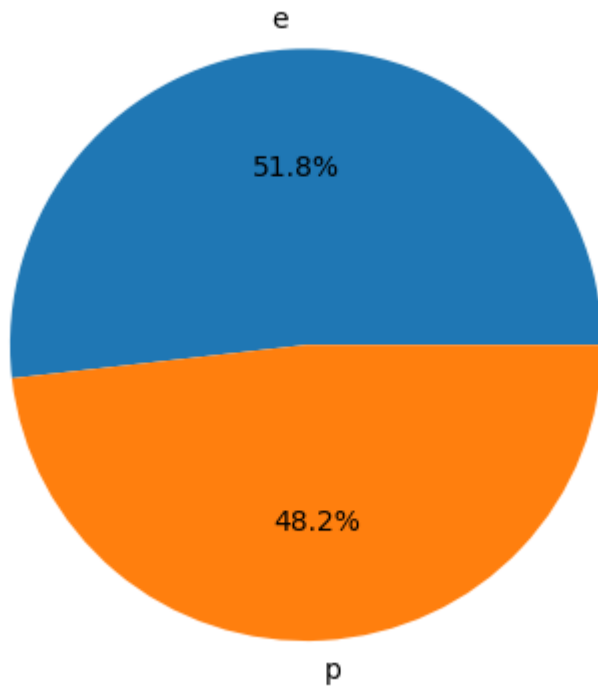



```
In [86]: #2.Bar Plot
import matplotlib.pyplot as plt
import seaborn as sns
df["cap-shape"].value_counts().plot(kind="bar")
plt.show()
```



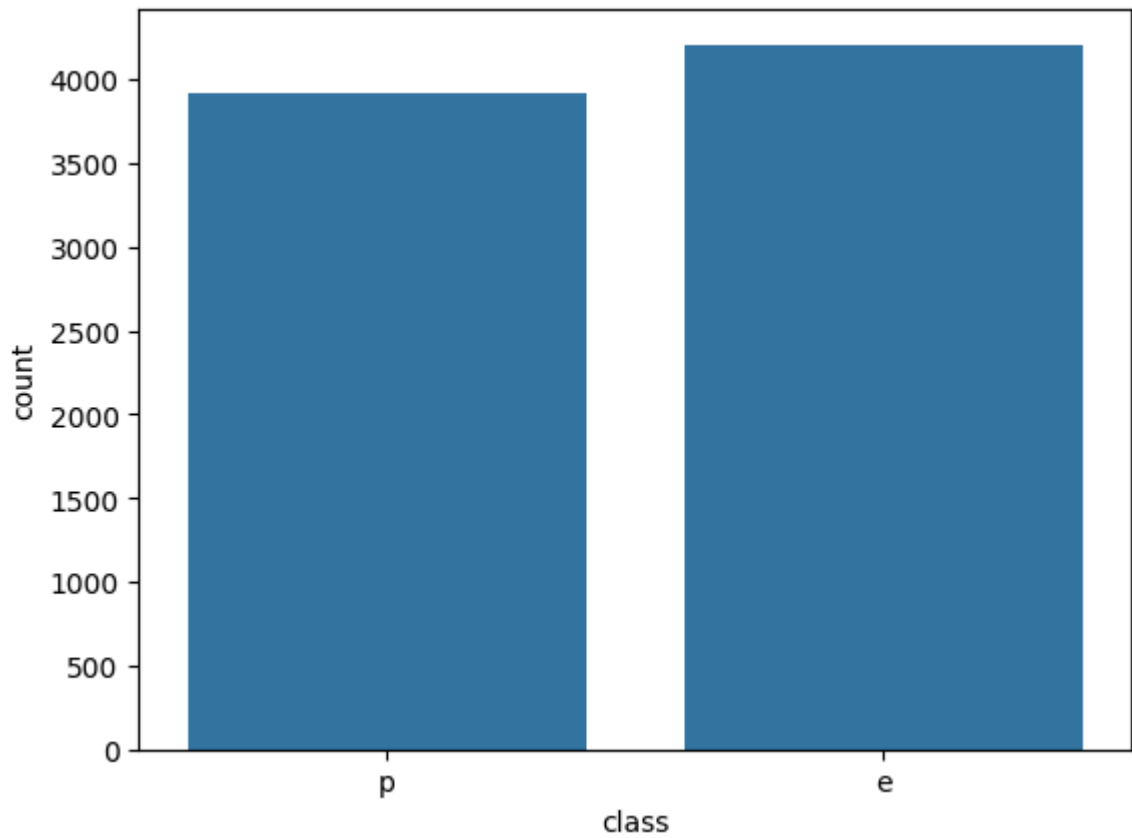
```
In [87]: #3.Pie Chart
import matplotlib.pyplot as plt
import seaborn as sns

df["class"].value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.ylabel("")
plt.show()
```

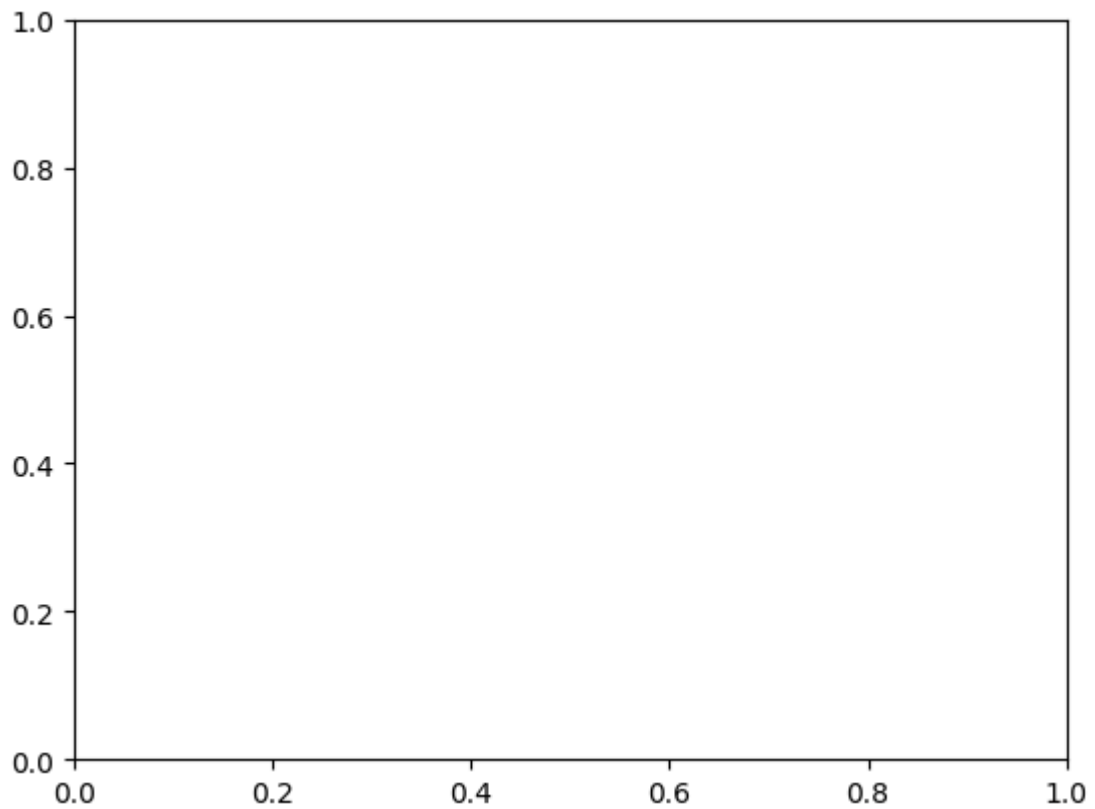


```
In [90]: #4.Histogram

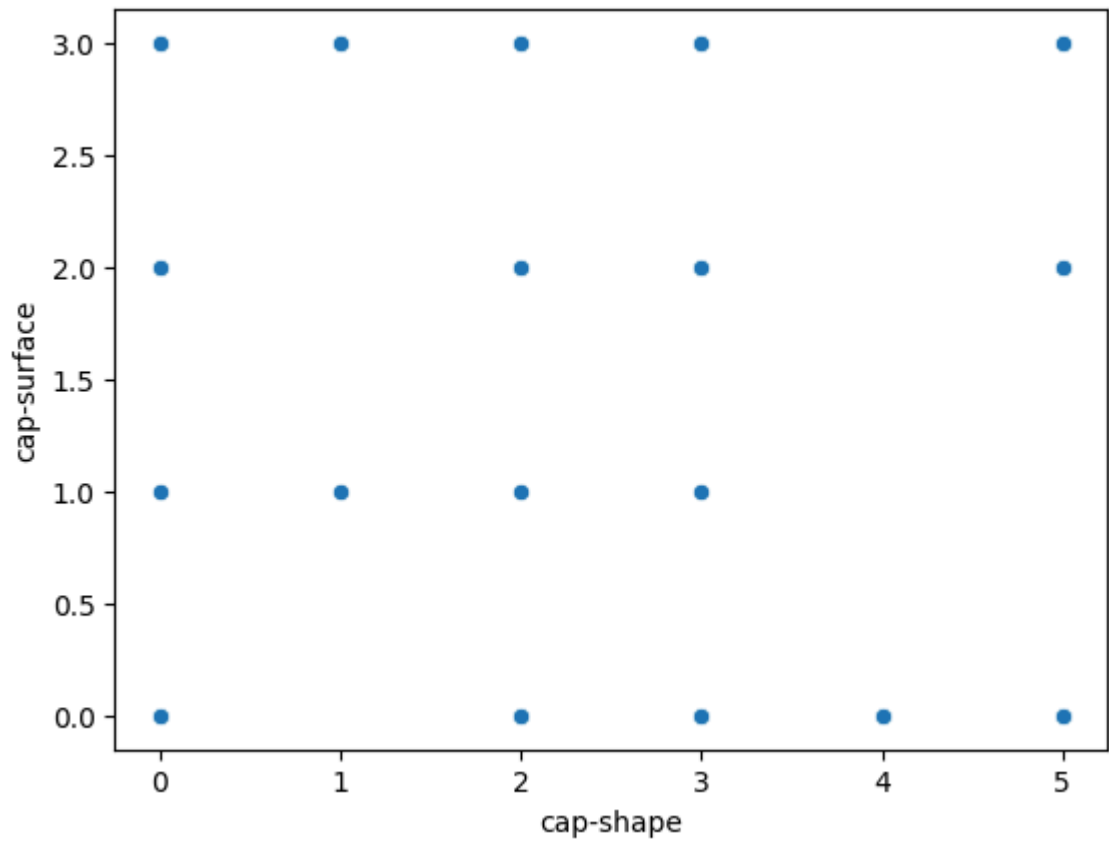
import matplotlib.pyplot as plt
import seaborn as sns
sns.countplot(x=df["class"])
plt.show()
```



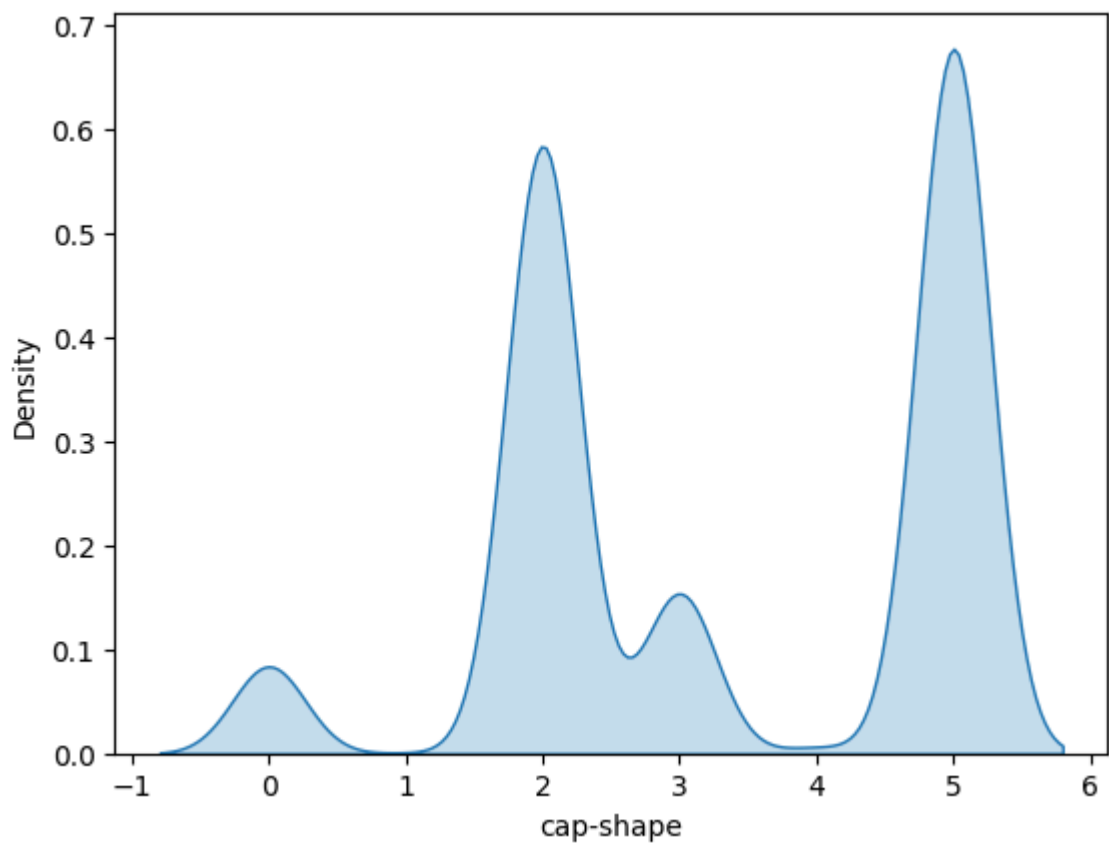
```
In [91]: #5.Box Plot
sns.boxplot(data=df.select_dtypes(include=["number"]))
plt.show()
```



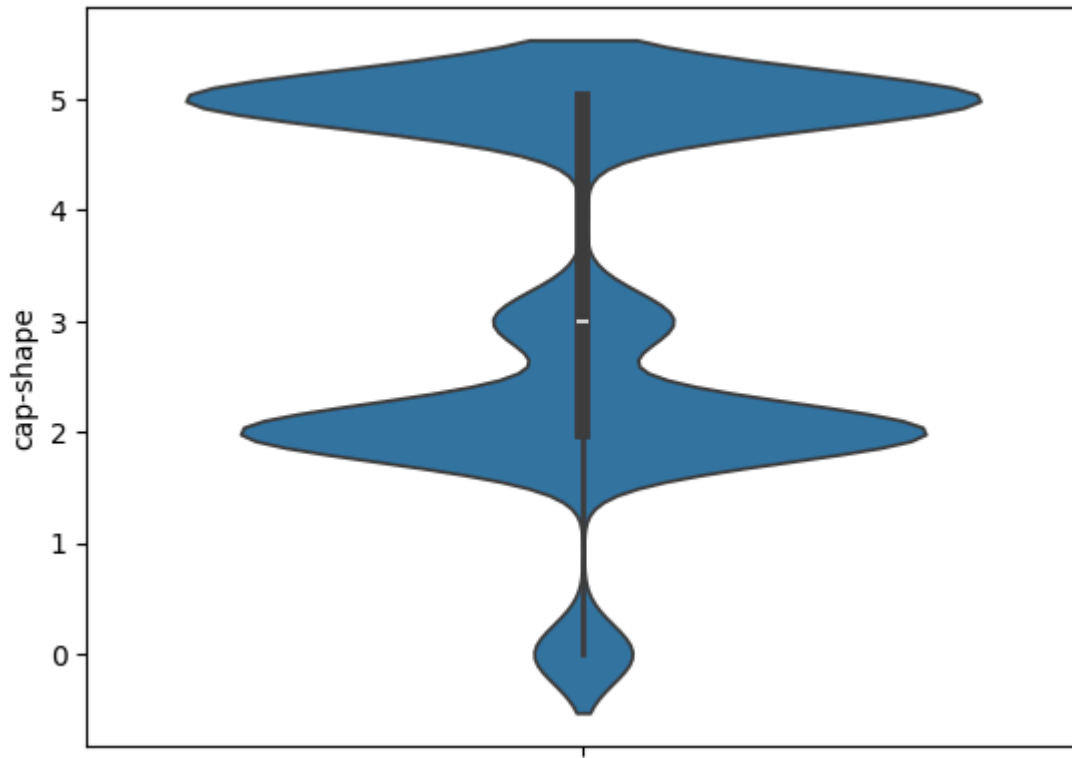
```
In [97]: #6.Scatter Plot
sns.scatterplot(x=df.iloc[:,1],y=df.iloc[:,2])
plt.show()
```



```
In [98]: #7.KDE plot  
sns.kdeplot(df.iloc[:,1],fill=True)  
plt.show()
```

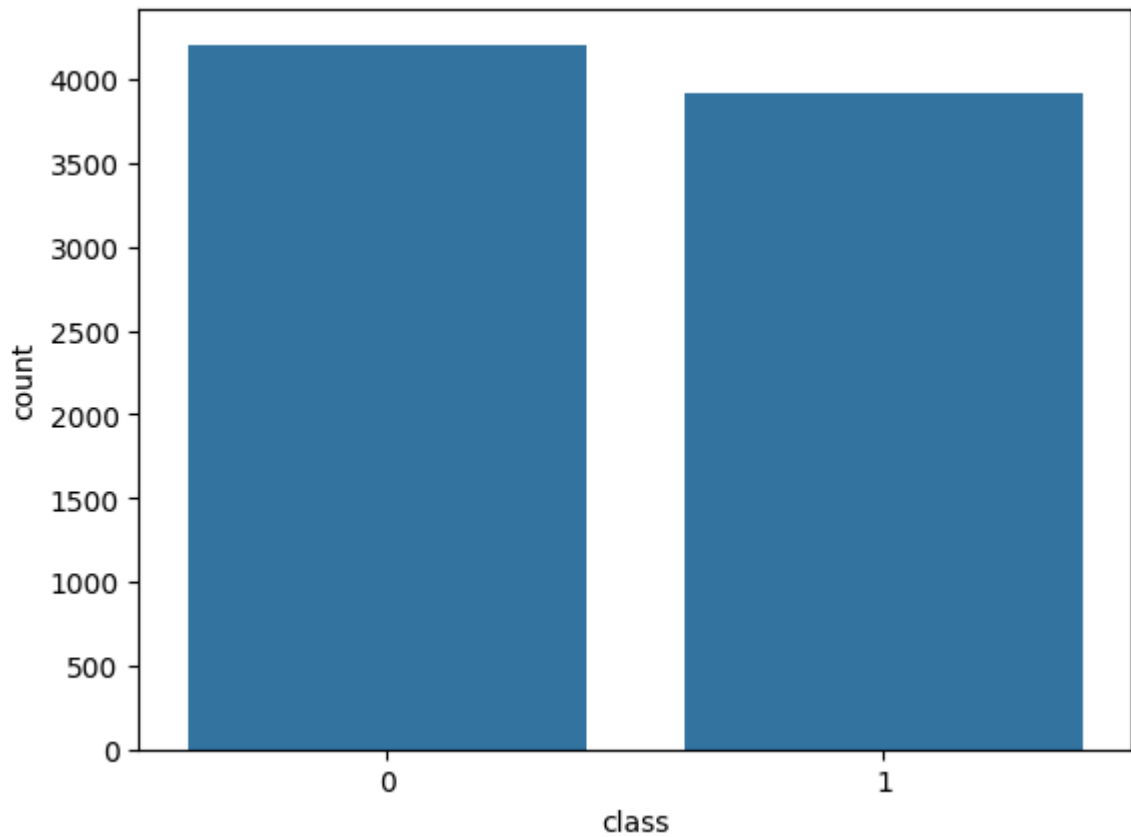


```
In [99]: #8.Violin Plot  
sns.violinplot(y=df.iloc[:,1])  
plt.show()
```



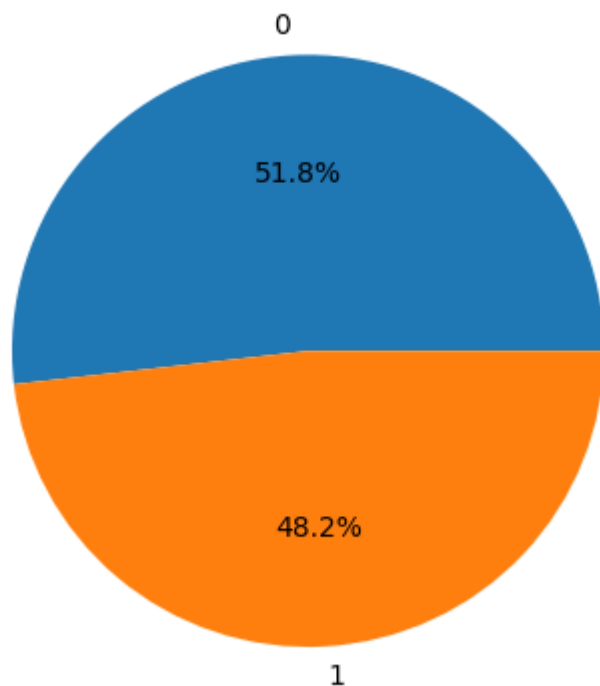
In [100...

```
#9.Count Plot  
sns.countplot(x=df.iloc[:,0])  
plt.show()
```



In [101...

```
#10.Pie Chart  
df.iloc[:,0].value_counts().plot(kind="pie", autopct="%1.1f%%")  
plt.ylabel("")  
plt.show()
```



4.perform Feature Engineering

Encoding categorical features (if any)

In [102...

```
from sklearn.preprocessing import LabelEncoder

le=LabelEncoder()

for col in df.columns:
    df[col]=le.fit_transform(df[col])
df.head()
```

Out[102...

	class	cap- shape	cap- surface	cap- color	bruises	odor	gill- attachment	gill- spacing	gill- size	gill- color	...	su t
0	1	5	2	4	1	6	1	0	1	4	...	
1	0	5	2	9	1	0	1	0	0	4	...	
2	0	0	2	8	1	3	1	0	0	5	...	
3	1	5	3	8	1	6	1	0	1	5	...	
4	0	5	2	3	0	5	1	1	0	4	...	

5 rows × 23 columns



Feature Scaling

```
In [106... scaler=StandardScaler()
df[df.columns]=scaler.fit_transform(df[df.columns])
df.head()
```

Out[106...

	class	cap- shape	cap- surface	cap- color	bruises	odor	gill- attachment	gill spacing
0	1.036613	1.029712	0.140128	-0.198250	1.185917	0.881938	0.162896	-0.43886
1	-0.964680	1.029712	0.140128	1.765874	1.185917	-1.970316	0.162896	-0.43886
2	-0.964680	-2.087047	0.140128	1.373049	1.185917	-0.544189	0.162896	-0.43886
3	1.036613	1.029712	0.953270	1.373049	1.185917	0.881938	0.162896	-0.43886
4	-0.964680	1.029712	0.140128	-0.591075	-0.843230	0.406562	0.162896	2.27861

5 rows × 23 columns



Create New Feature

```
In [107... df["New_Feature"]=df.iloc[:,1]+df.iloc[:,2]
df.head()
```

Out[107...

	class	cap- shape	cap- surface	cap- color	bruises	odor	gill- attachment	gill spacing
0	1.036613	1.029712	0.140128	-0.198250	1.185917	0.881938	0.162896	-0.43886
1	-0.964680	1.029712	0.140128	1.765874	1.185917	-1.970316	0.162896	-0.43886
2	-0.964680	-2.087047	0.140128	1.373049	1.185917	-0.544189	0.162896	-0.43886
3	1.036613	1.029712	0.953270	1.373049	1.185917	0.881938	0.162896	-0.43886
4	-0.964680	1.029712	0.140128	-0.591075	-0.843230	0.406562	0.162896	2.27861

5 rows × 24 columns



5.Traning & Testing

```
In [ ]: from sklearn.model_selection import train_test_split

x=df.drop("class",axis=1)
y=df["class"]
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)

print("x_train:",x_train.shape)
print("x_test:",x_test.shape)
print("y_train:",y_train.shape)
print("y_test:",y_test.shape)
```

Penguins Size Dataset

1.Load using pandas

In [108... `import pandas as pd`

```
df=pd.read_csv("D:/AI Notes/All Datasets (1)/penguins_size.csv")
df.head()
```

Out[108...

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_
0	Adelie	Torgersen	39.1	18.7	181.0	
1	Adelie	Torgersen	39.5	17.4	186.0	
2	Adelie	Torgersen	40.3	18.0	195.0	
3	Adelie	Torgersen	NaN	NaN	NaN	
4	Adelie	Torgersen	36.7	19.3	193.0	

perform Data Cleaning

In [109... `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   species               344 non-null    object
1   island                344 non-null    object
2   culmen_length_mm      342 non-null    float64
3   culmen_depth_mm       342 non-null    float64
4   flipper_length_mm     342 non-null    float64
5   body_mass_g           342 non-null    float64
6   sex                   334 non-null    object
dtypes: float64(4), object(3)
memory usage: 18.9+ KB
```

In [110... `df.describe(include='all')`

Out[110...

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	boc
count	344	344	342.000000	342.000000	342.000000	3
unique	3	3	NaN	NaN	NaN	
top	Adelie	Biscoe	NaN	NaN	NaN	
freq	152	168	NaN	NaN	NaN	
mean	NaN	NaN	43.921930	17.151170	200.915205	42
std	NaN	NaN	5.459584	1.974793	14.061714	8
min	NaN	NaN	32.100000	13.100000	172.000000	27
25%	NaN	NaN	39.225000	15.600000	190.000000	35
50%	NaN	NaN	44.450000	17.300000	197.000000	40
75%	NaN	NaN	48.500000	18.700000	213.000000	47
max	NaN	NaN	59.600000	21.500000	231.000000	63

In [111...

```
df.isnull().sum()
```

Out[111...

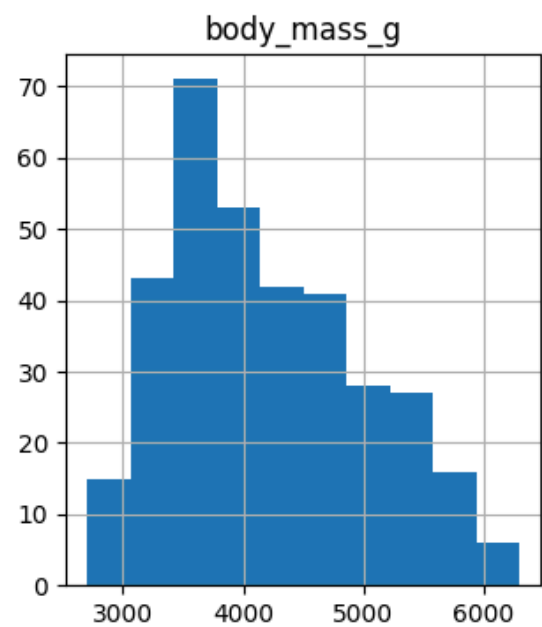
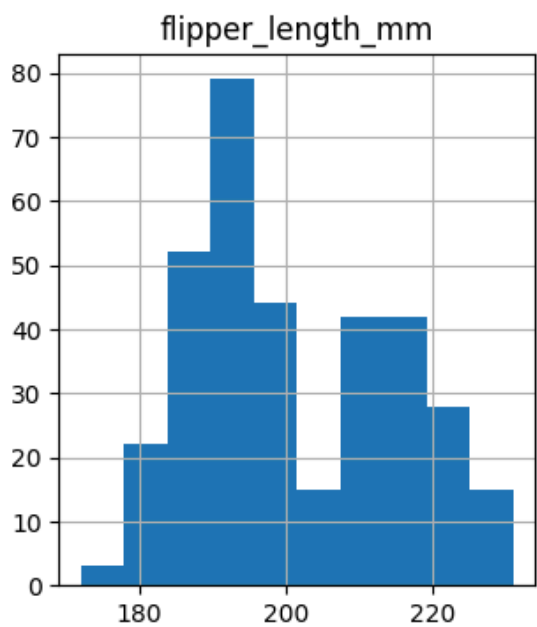
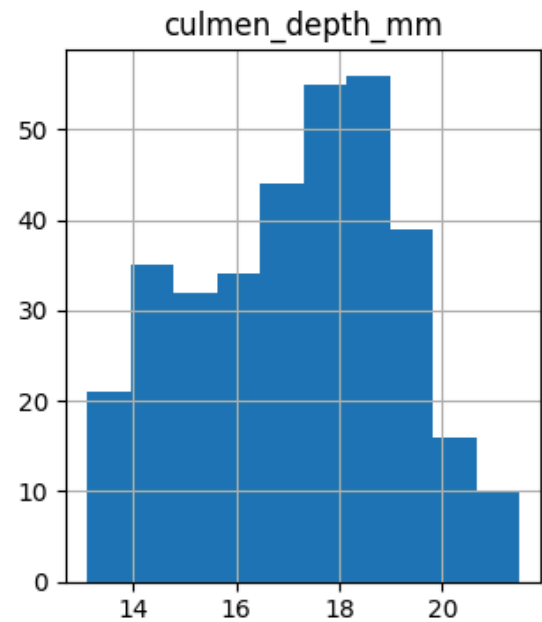
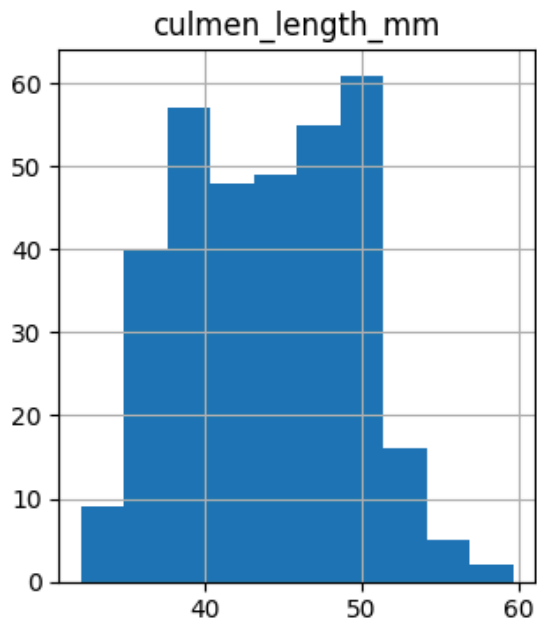
```
species      0
island       0
culmen_length_mm    2
culmen_depth_mm    2
flipper_length_mm    2
body_mass_g      2
sex           10
dtype: int64
```

3.perform Exploratory Data Analysis(EDA)

In [112...

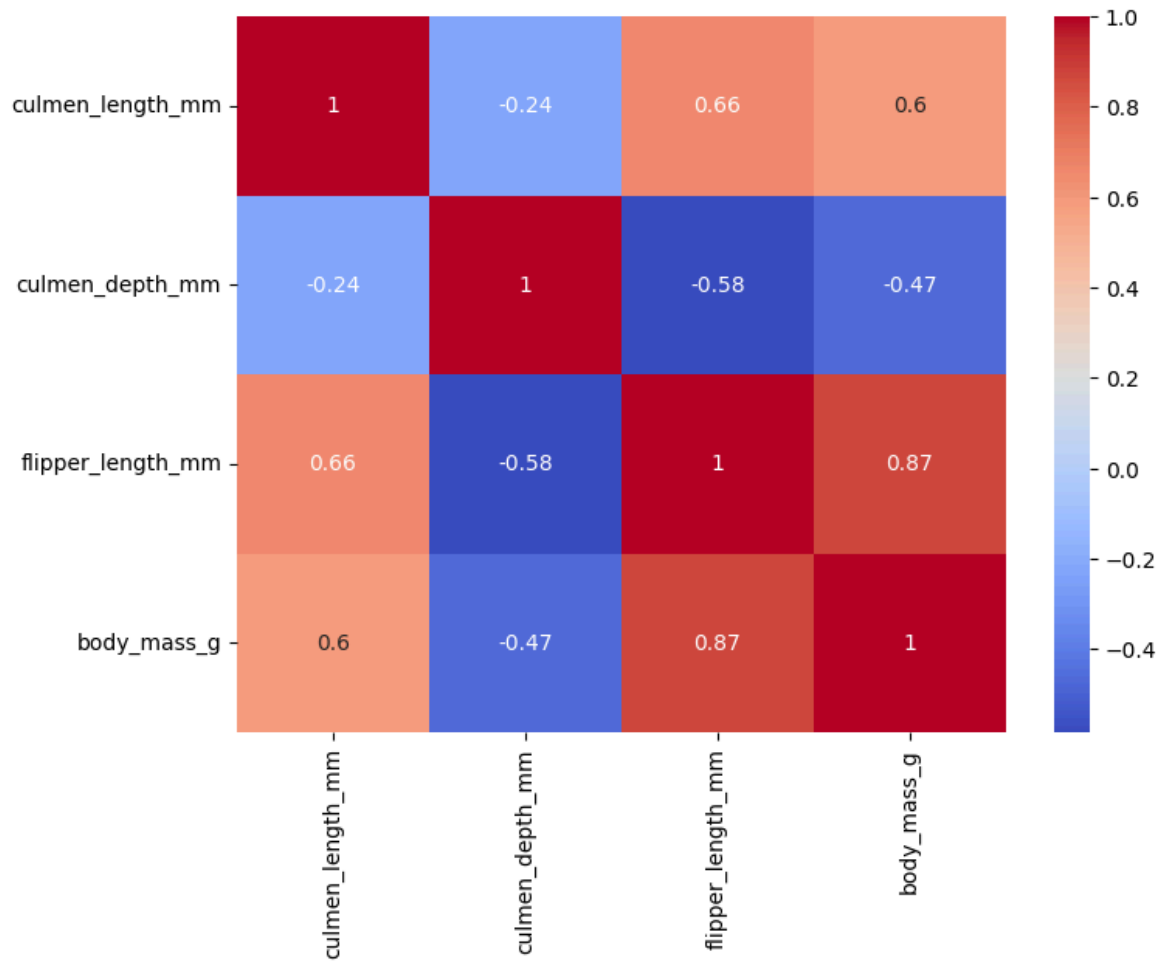
```
import matplotlib.pyplot as plt
import seaborn as sns

#1.Histogram
df.hist(figsize=(8,9))
plt.show()
```



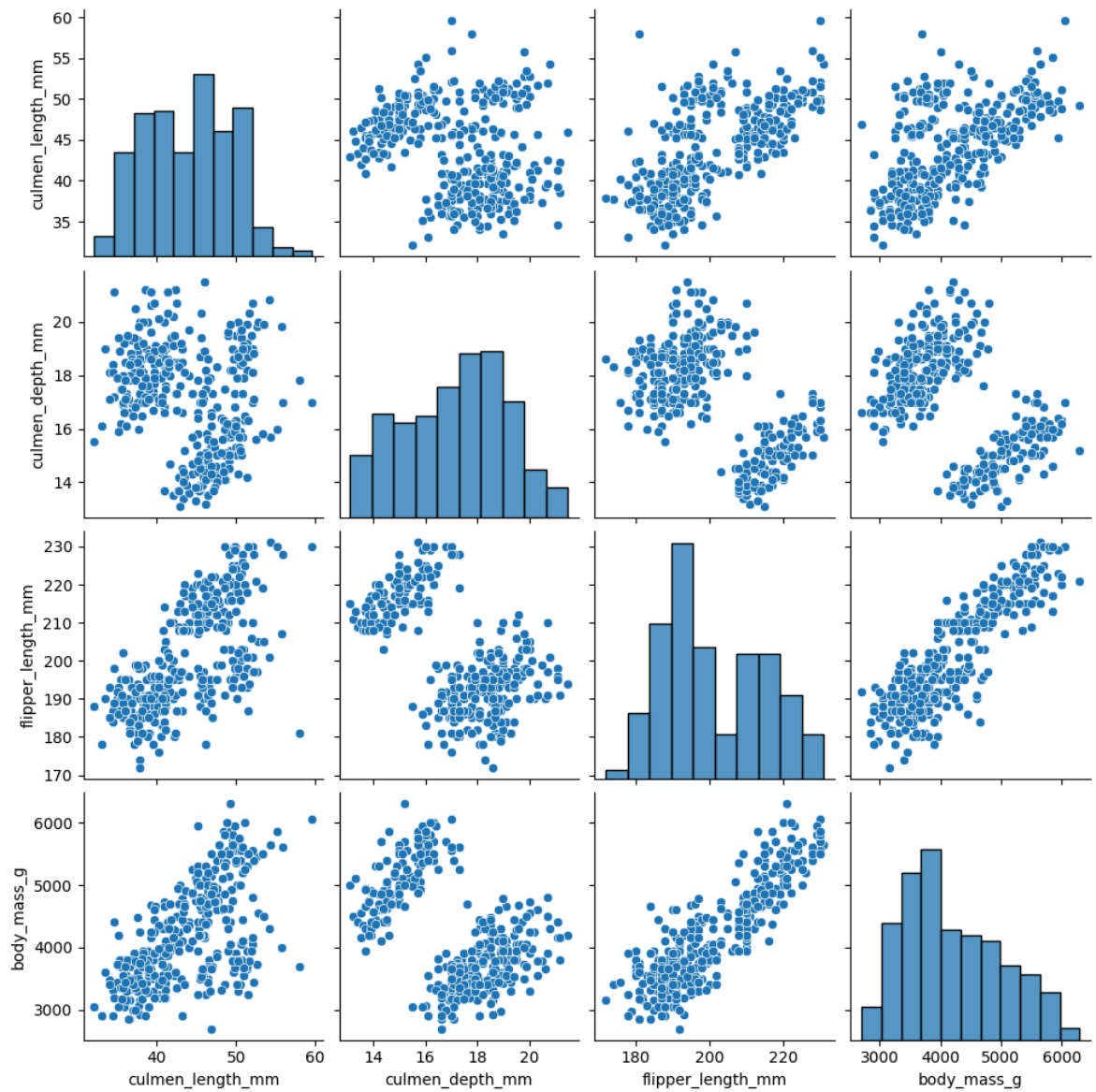
In [113...

```
#2.Heatmap
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(numeric_only=True),annot=True,cmap="coolwarm")
plt.show()
```



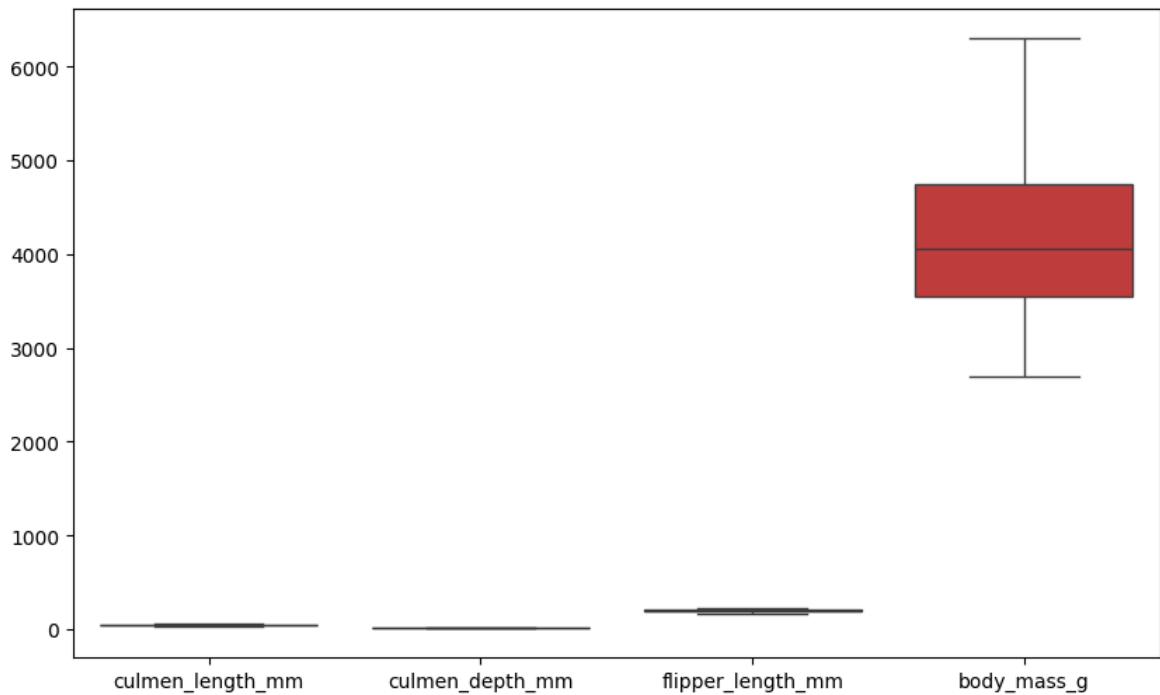
In [114...

```
#3. Pair Plot  
sns.pairplot(df)  
plt.show()
```

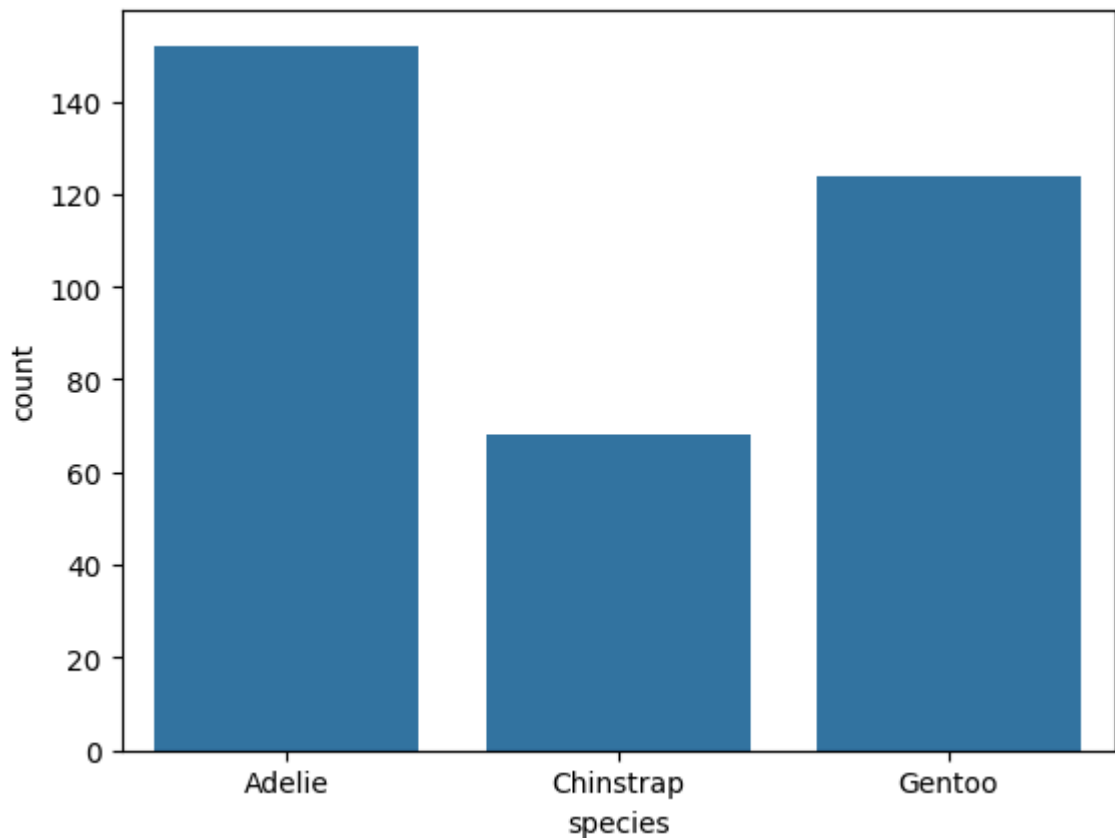


In [115...

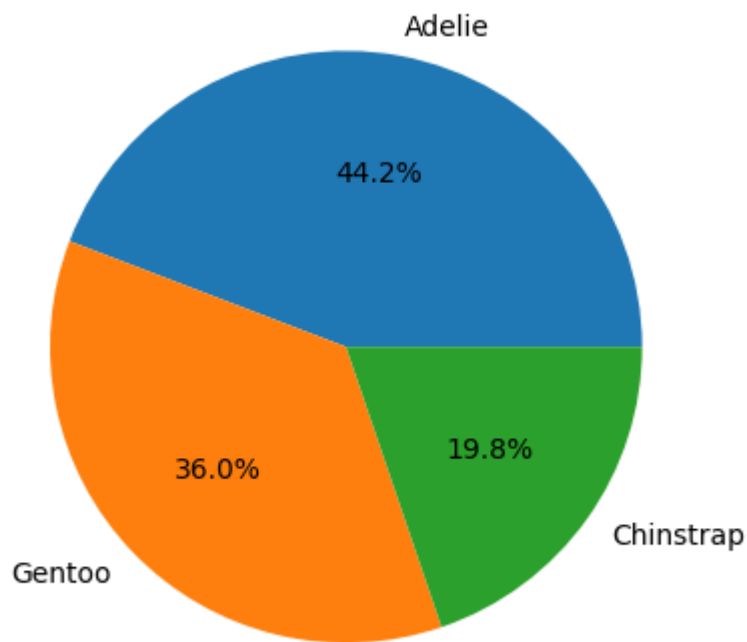
```
#4.Box plot
plt.figure(figsize=(10,6))
sns.boxplot(data=df.select_dtypes(include=['number']))
plt.show()
```



In [116... *#5.Count plot*
`sns.countplot(x=df["species"])`
`plt.show()`

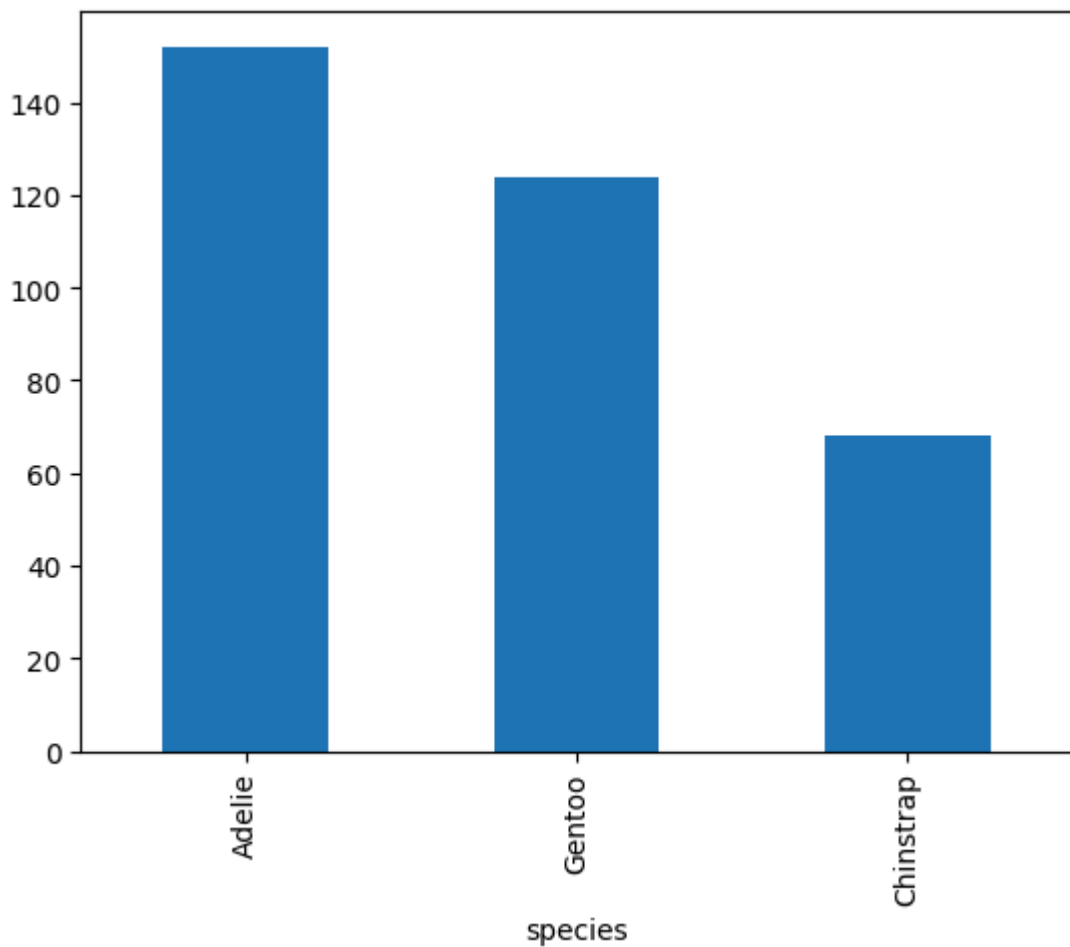


In [117... *#6.pie Chart*
`df["species"].value_counts().plot(kind="pie", autopct="%1.1f%%")`
`plt.ylabel("")`
`plt.show()`



In [118...

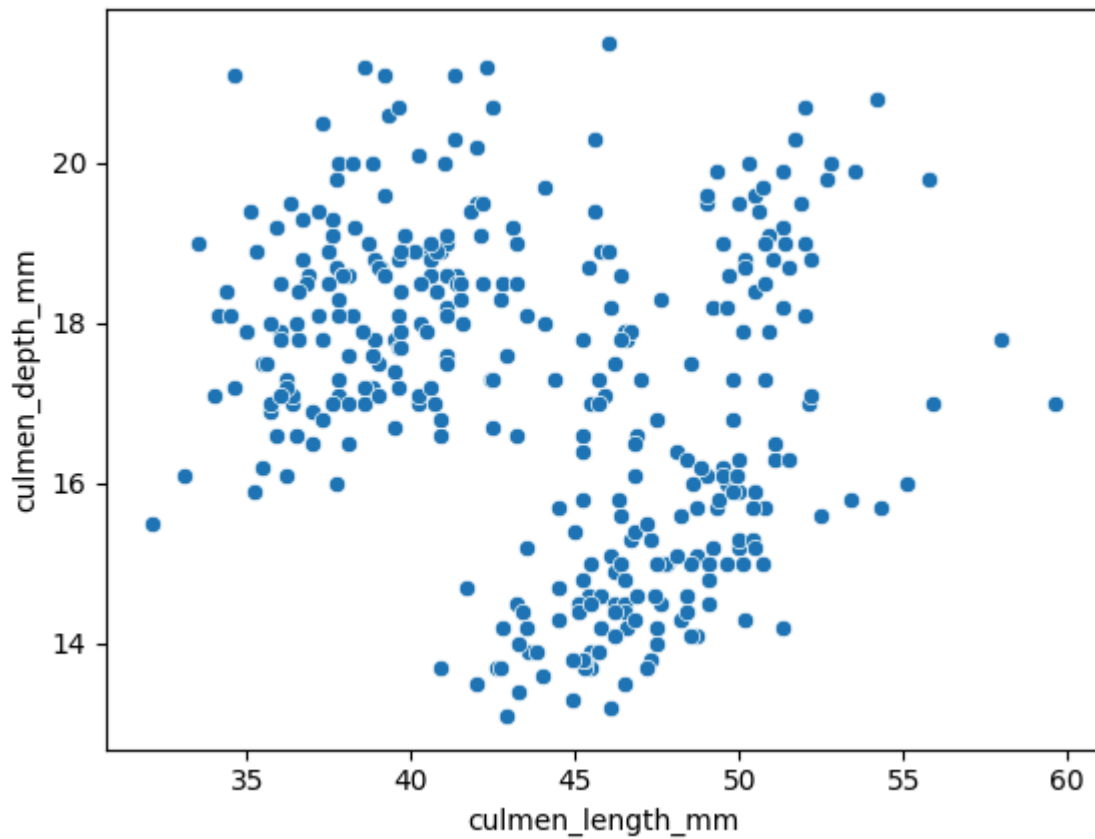
```
#7.Bar Plot  
df["species"].value_counts().plot(kind="bar")  
plt.show()
```



In [122...

```
num_cols=df.select_dtypes(include=['number']).columns  
sns.scatterplot(x=df[num_cols[0]],y=df[num_cols[1]])
```

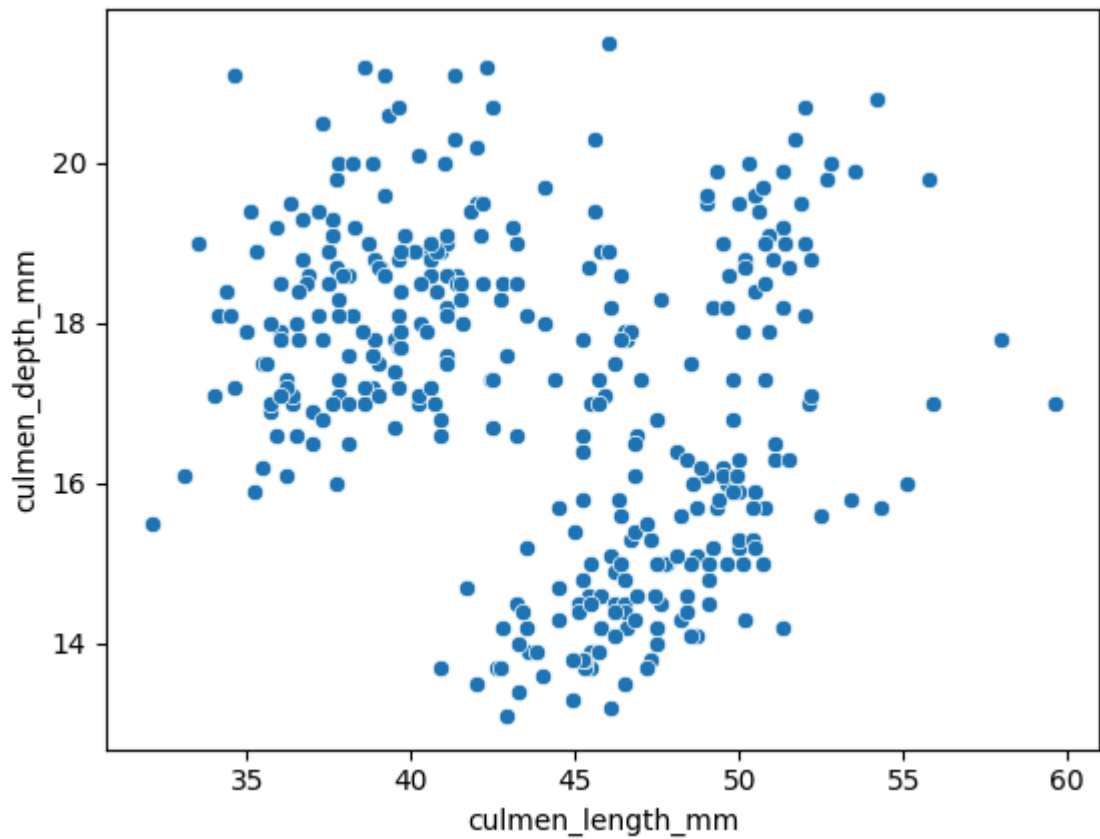
```
plt.show()
```



```
In [124...] df.columns
```

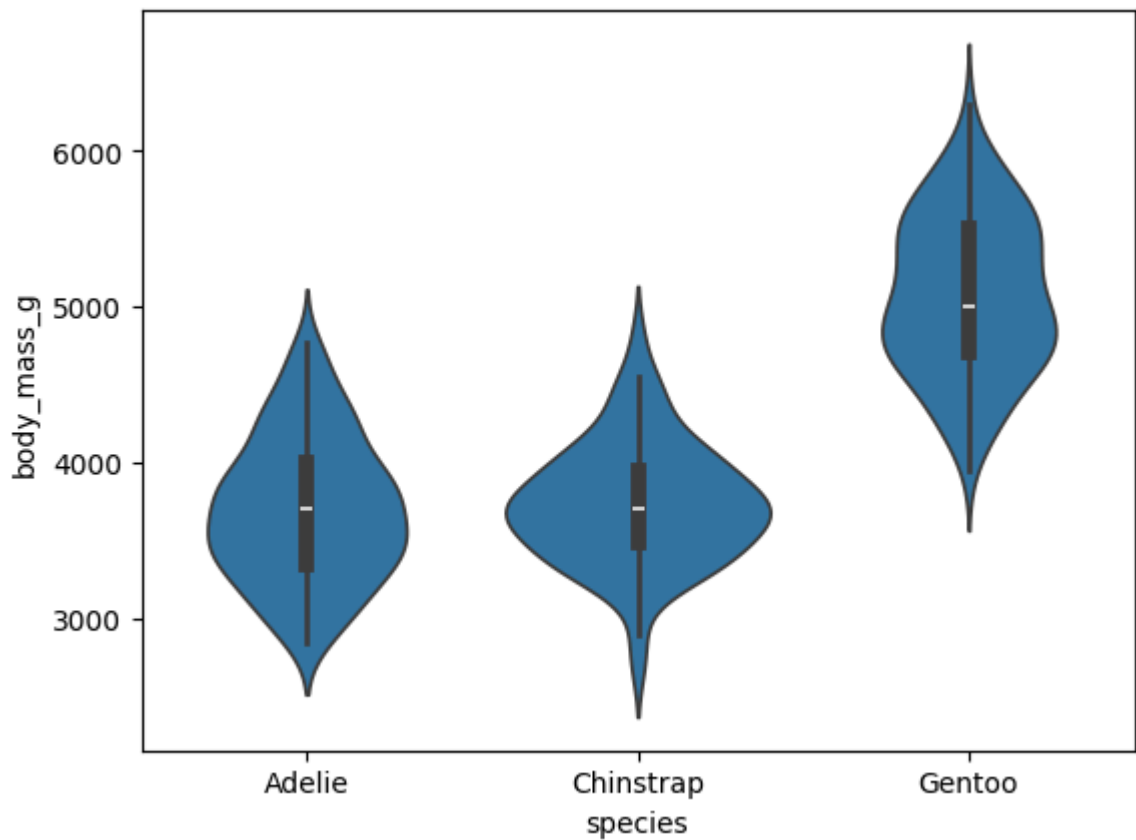
```
Out[124...] Index(['species', 'island', 'culmen_length_mm', 'culmen_depth_mm',  
                'flipper_length_mm', 'body_mass_g', 'sex'],  
                dtype='object')
```

```
In [127...] #9.KDE plot  
num_cols=df.select_dtypes(include=['number']).columns  
sns.scatterplot(x=df[num_cols[0]],y=df[num_cols[1]])  
plt.show()
```



In [126...

```
#10.Violin Plot  
sns.violinplot(x="species",y="body_mass_g",data=df)  
plt.show()
```



4.perform Feature Engineering

Encoding categorical features (if any)

```
In [128... from sklearn.preprocessing import LabelEncoder

le=LabelEncoder()

for col in df.select_dtypes(include=['object']).columns:
    df[col]=le.fit_transform(df[col])
df.head()
```

```
Out[128... species  island  culmen_length_mm  culmen_depth_mm  flipper_length_mm  body_ma
```

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_ma
0	0	2	39.1	18.7	181.0	37
1	0	2	39.5	17.4	186.0	38
2	0	2	40.3	18.0	195.0	32
3	0	2	NaN	NaN	NaN	
4	0	2	36.7	19.3	193.0	34

Feature Scaling

```
In [130... from sklearn.preprocessing import StandardScaler

scaler=StandardScaler()

numeric_cols=df.select_dtypes(include=["number"]).columns

df[numeric_cols]=scaler.fit_transform(df[numeric_cols])
df.head()
```

```
Out[130... species  island  culmen_length_mm  culmen_depth_mm  flipper_length_mm  body
```

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body
0	-1.029802	1.844076	-0.884499	0.785449	-1.418347	-
1	-1.029802	1.844076	-0.811126	0.126188	-1.062250	-
2	-1.029802	1.844076	-0.664380	0.430462	-0.421277	-
3	-1.029802	1.844076	NaN	NaN	NaN	
4	-1.029802	1.844076	-1.324737	1.089724	-0.563715	-

Create New Feature

```
In [131... df["Body_Ratio"]=df["body_mass_g"]/df["flipper_length_mm"]
df.head()
```

Out[131...

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body
0	-1.029802	1.844076	-0.884499	0.785449	-1.418347	-
1	-1.029802	1.844076	-0.811126	0.126188	-1.062250	-
2	-1.029802	1.844076	-0.664380	0.430462	-0.421277	-
3	-1.029802	1.844076	NaN	NaN	NaN	-
4	-1.029802	1.844076	-1.324737	1.089724	-0.563715	-

5. Training & Testing

In [132...

```

from sklearn.model_selection import train_test_split

x=df.drop("species",axis=1)
y=df["species"]

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)

print("x_train",x_train.shape)
print("x_test",x_test.shape)
print("y_train",y_train.shape)
print("y_test",y_test.shape)

```

```

x_train (275, 7)
x_test (69, 7)
y_train (275,)
y_test (69,)

```

Stroke Prediction

1. Load using pandas

In [133...

```

import pandas as pd

df=pd.read_csv("D:/AI Notes/All Datasets (1)/stroke prediction.csv")
df.head()

```

Out[133...

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Resid
0	30669	Male	3.0	0	0	No	children	
1	30468	Male	58.0	1	0	Yes	Private	
2	16523	Female	8.0	0	0	No	Private	
3	56543	Female	70.0	0	0	Yes	Private	
4	46136	Male	14.0	0	0	No	Never_worked	

2.perform Data Cleaning

In [134...

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 43400 entries, 0 to 43399
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   id                     43400 non-null  int64  
1   gender                 43400 non-null  object  
2   age                    43400 non-null  float64 
3   hypertension           43400 non-null  int64  
4   heart_disease         43400 non-null  int64  
5   ever_married           43400 non-null  object  
6   work_type              43400 non-null  object  
7   Residence_type         43400 non-null  object  
8   avg_glucose_level     43400 non-null  float64 
9   bmi                    41938 non-null  float64 
10  smoking_status         30108 non-null  object  
11  stroke                 43400 non-null  int64  
dtypes: float64(3), int64(4), object(5)
memory usage: 4.0+ MB
```

In [135...

df.describe(include='all')

Out[135...

	id	gender	age	hypertension	heart_disease	ever_married
count	43400.000000	43400	43400.000000	43400.000000	43400.000000	43400
unique	NaN	3	NaN	NaN	NaN	2
top	NaN	Female	NaN	NaN	NaN	Yes
freq	NaN	25665	NaN	NaN	NaN	27938
mean	36326.142350	NaN	42.217894	0.093571	0.047512	NaN
std	21072.134879	NaN	22.519649	0.291235	0.212733	NaN
min	1.000000	NaN	0.080000	0.000000	0.000000	NaN
25%	18038.500000	NaN	24.000000	0.000000	0.000000	NaN
50%	36351.500000	NaN	44.000000	0.000000	0.000000	NaN
75%	54514.250000	NaN	60.000000	0.000000	0.000000	NaN
max	72943.000000	NaN	82.000000	1.000000	1.000000	NaN



In [136...

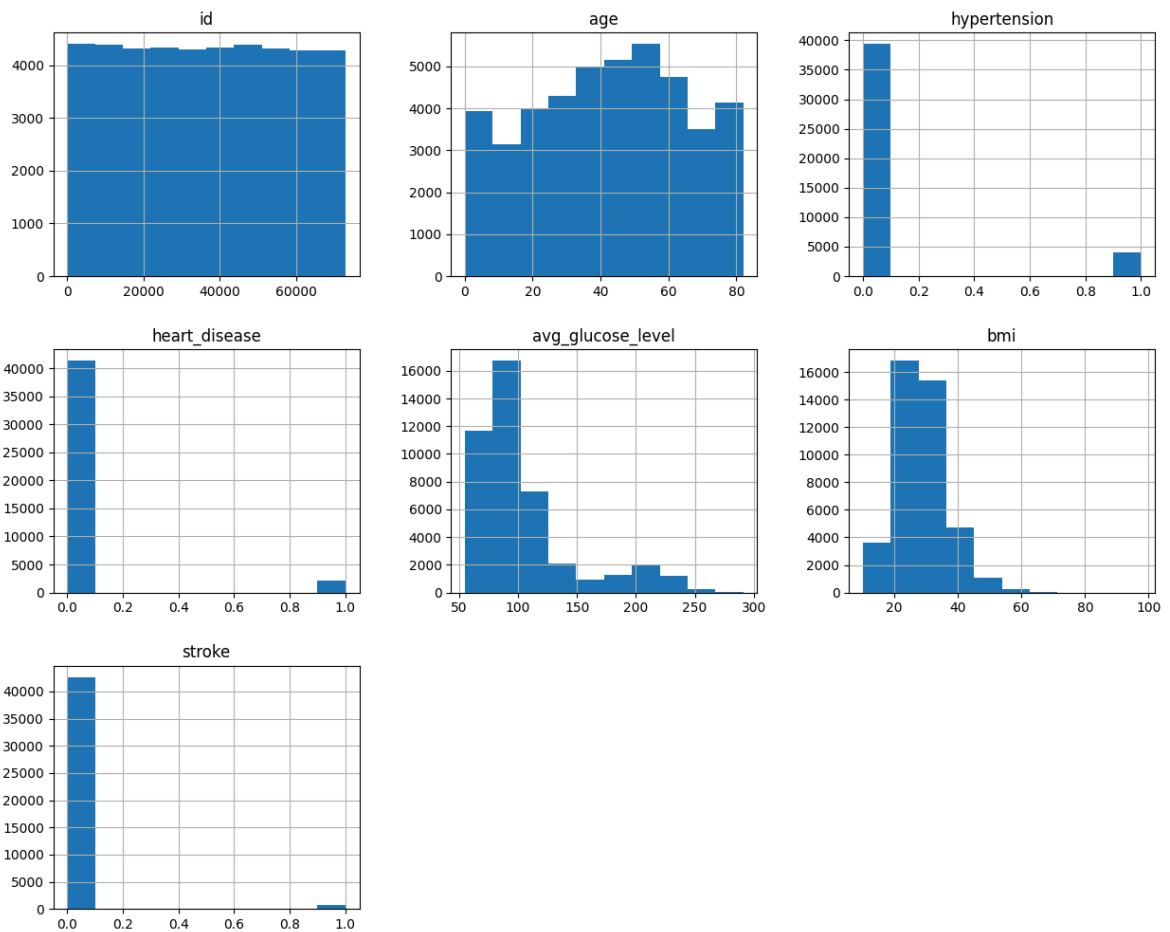
df.isnull().sum()

```
Out[136...  id                0
           gender    0
           age        0
           hypertension 0
           heart_disease 0
           ever_married 0
           work_type  0
           Residence_type 0
           avg_glucose_level 0
           bmi        1462
           smoking_status 13292
           stroke      0
           dtype: int64
```

3.perform Exploratory Data Analysis(EDA)

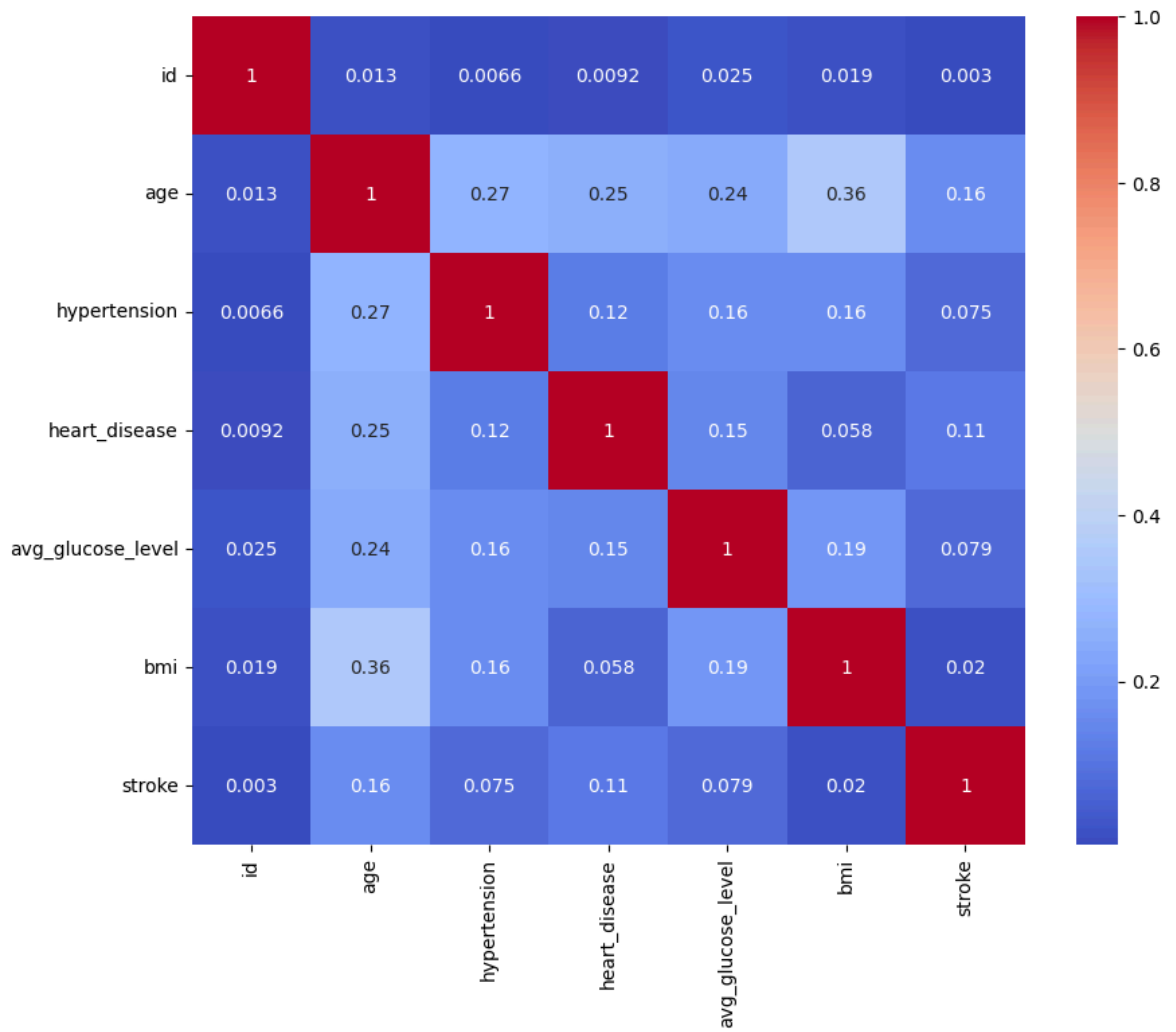
```
In [138... import matplotlib.pyplot as plt
import seaborn as sns

#1.Histogram
df.hist(figsize=(15,12))
plt.show()
```



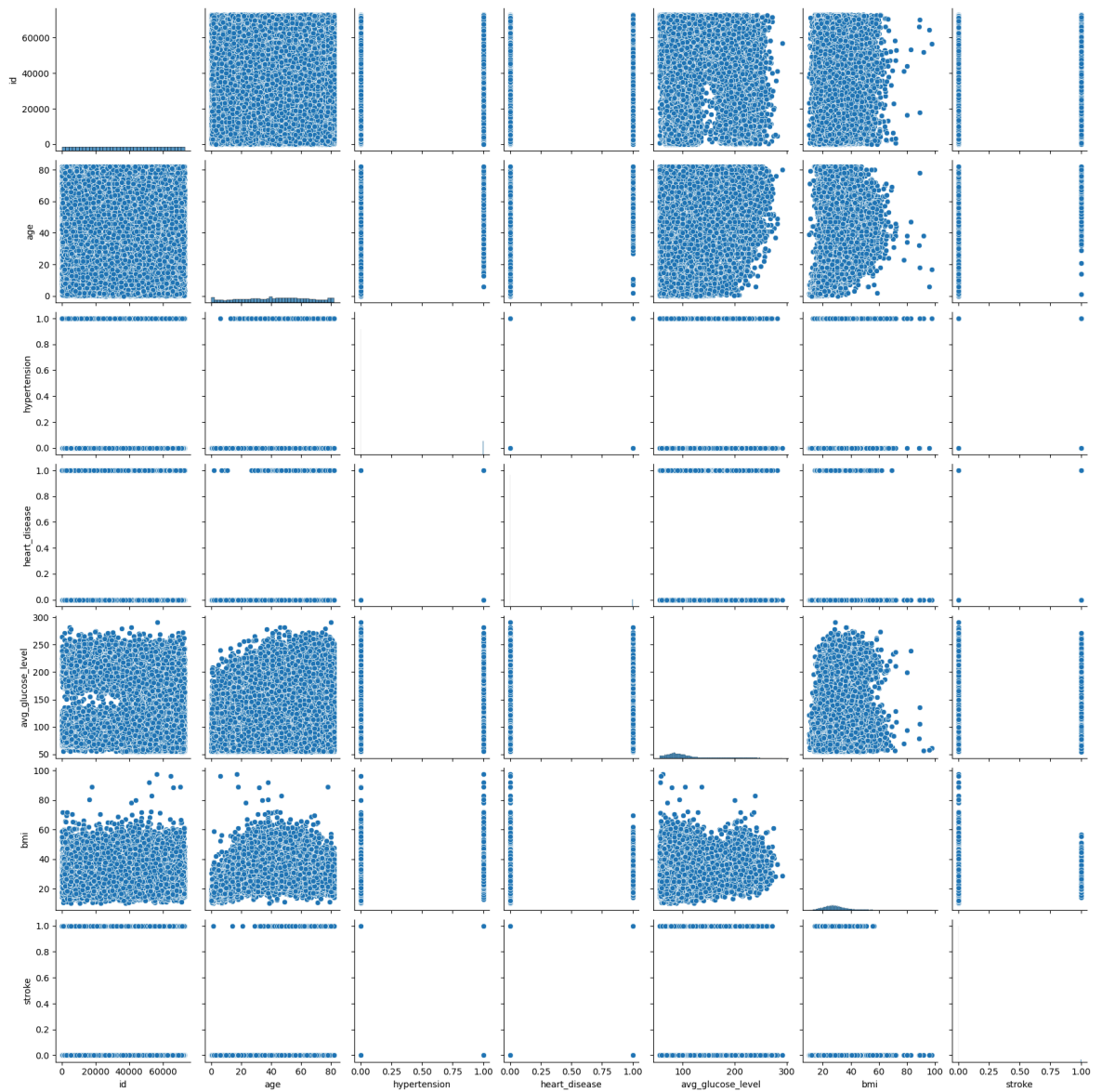
```
In [139... #2.Correlation Heatmap

plt.figure(figsize=(10,8))
sns.heatmap(df.corr(numeric_only=True),annot=True,cmap="coolwarm")
plt.show()
```



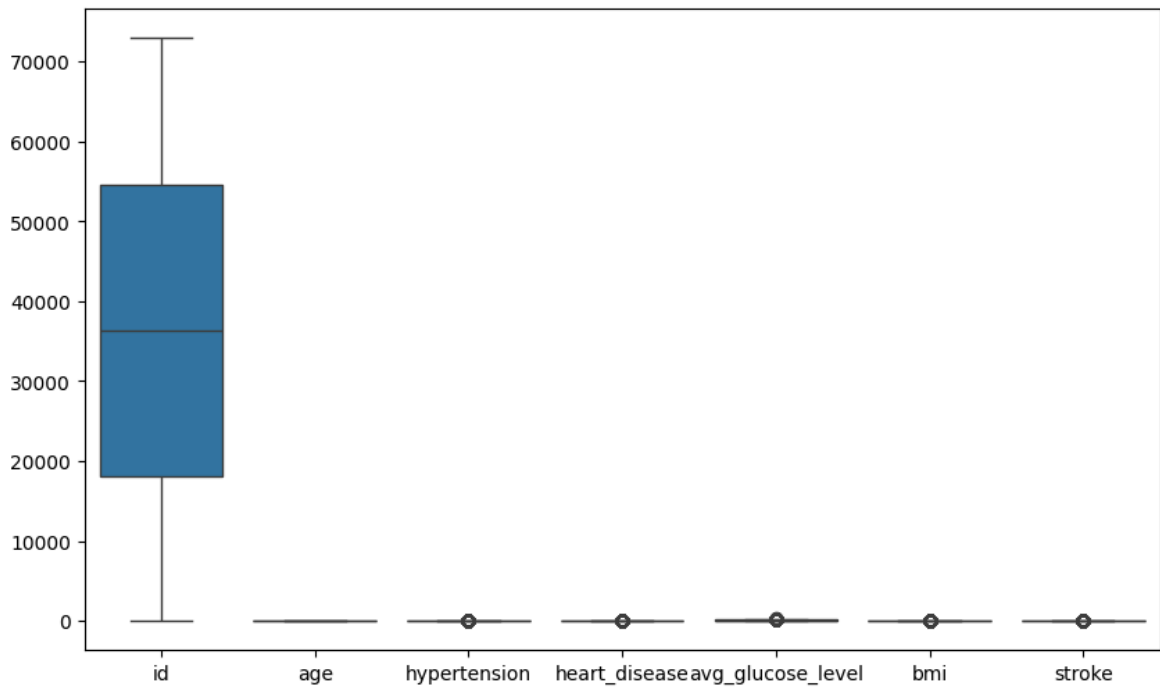
In [141...

```
#3.pair plot
sns.pairplot(df)
plt.show()
```



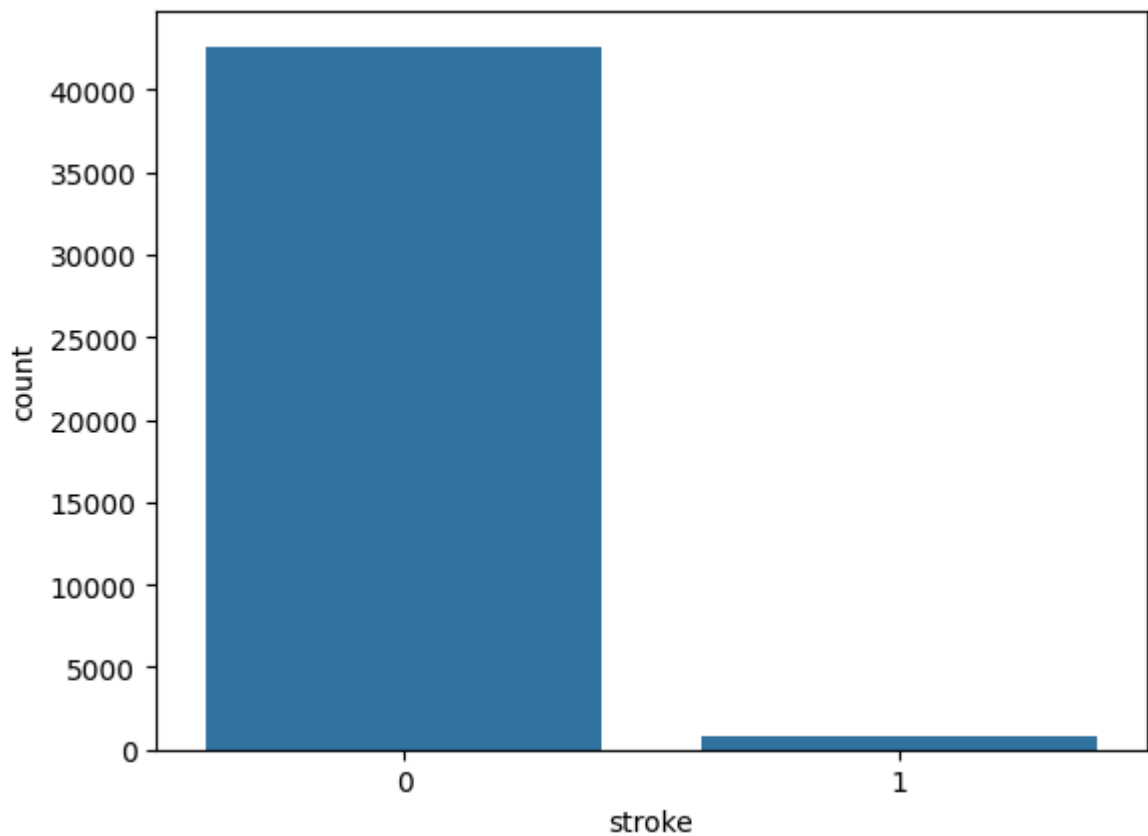
In [142...

```
#4.Box plot
plt.figure(figsize=(10,6))
sns.boxplot(data=df.select_dtypes(include=['number']))
plt.show()
```



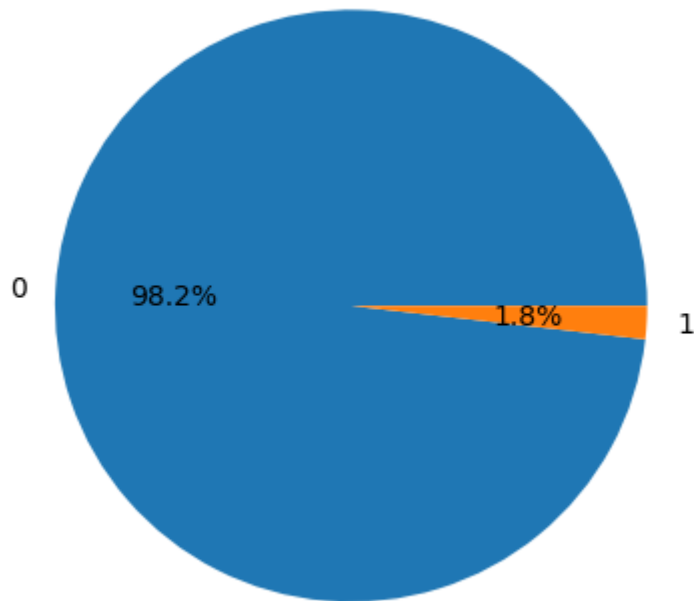
In [144...

```
#5. count plot  
sns.countplot(x="stroke", data=df)  
plt.show()
```



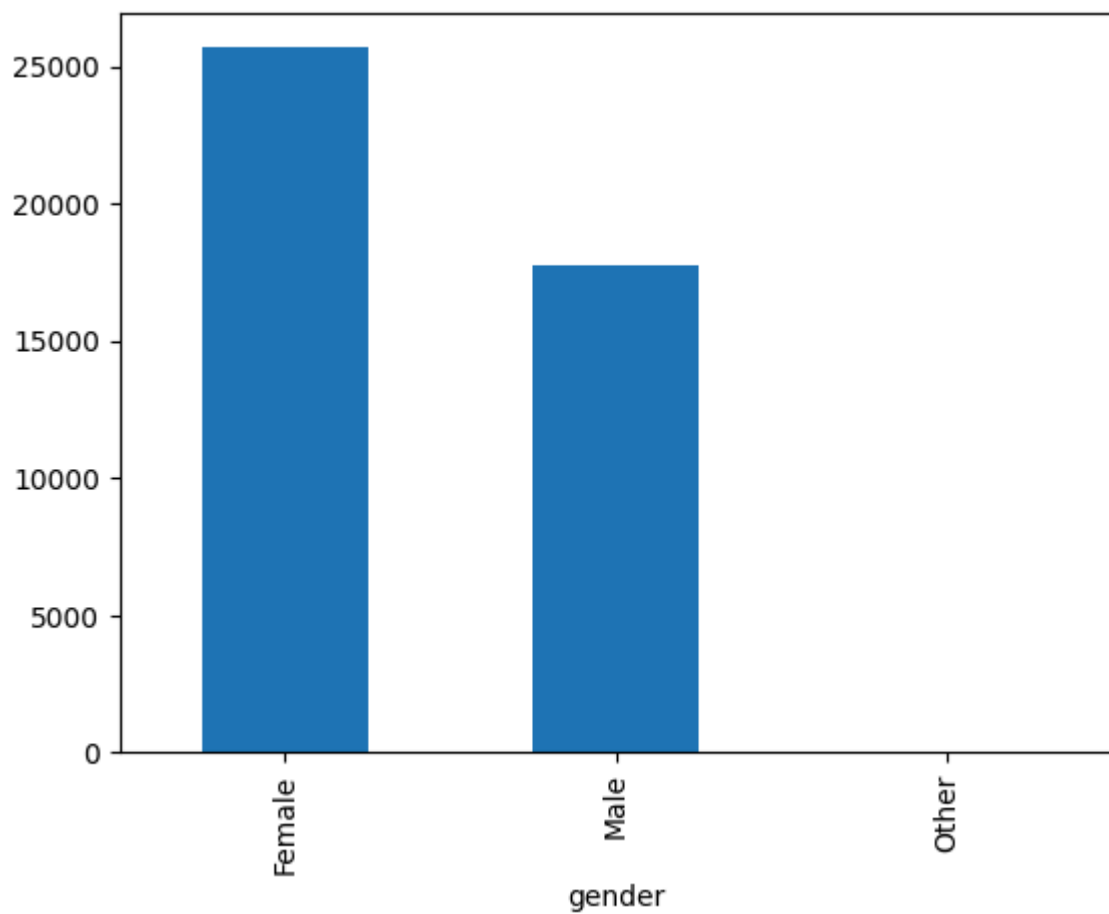
In [145...

```
#6. pie Chart  
df["stroke"].value_counts().plot(kind="pie", autopct="%1.1f%%")  
plt.ylabel("")  
plt.show()
```



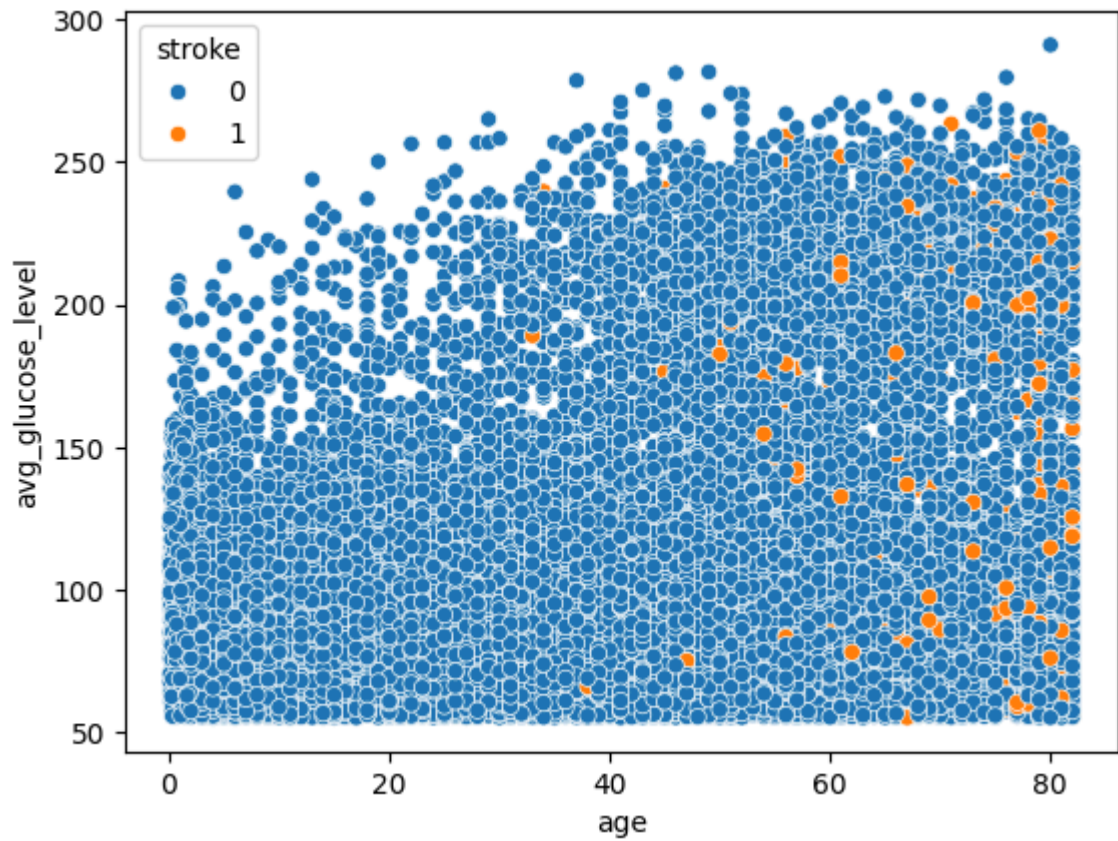
In [146...

```
#7.Bar Plot  
df["gender"].value_counts().plot(kind="bar")  
plt.show()
```



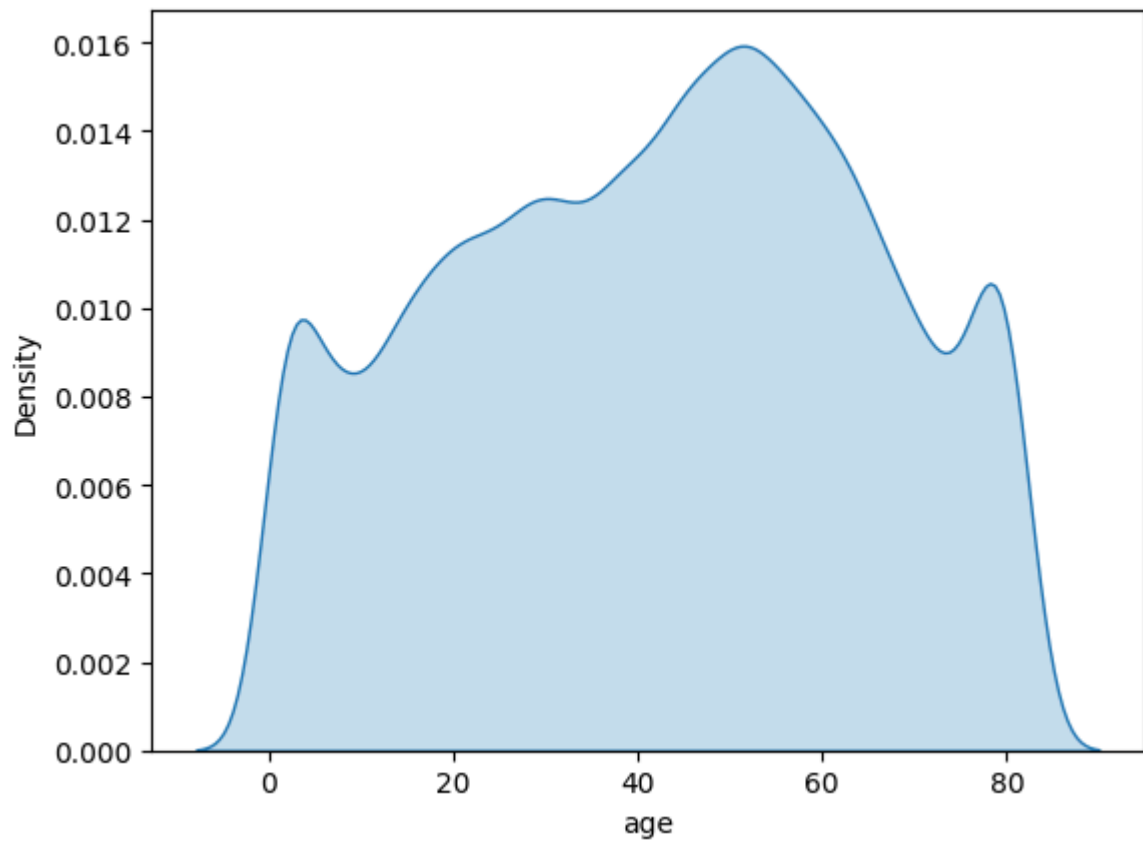
In [147...

```
#8.Scatter Plot  
sns.scatterplot(x="age",y="avg_glucose_level",hue="stroke",data=df)  
plt.show()
```

In [148...

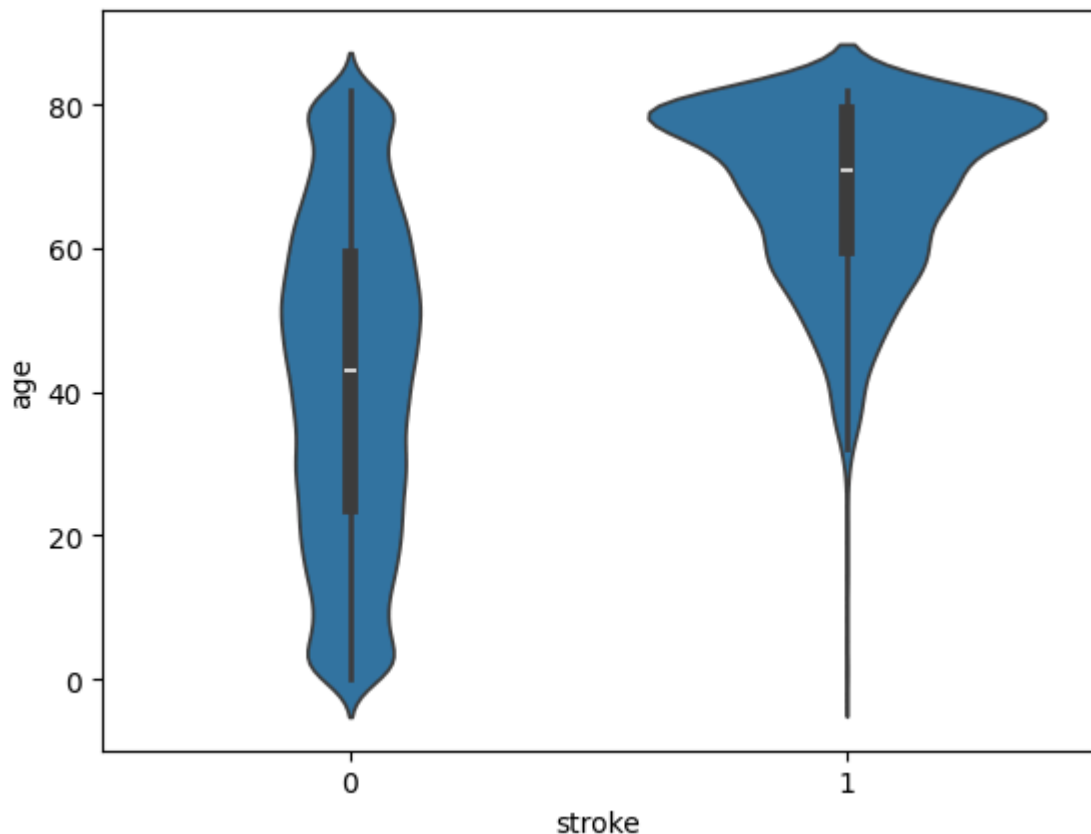
```
#9.KDE plot  
sns.kdeplot(data=df,x="age",fill=True)  
plt.show()
```



In [149...

```
#10.Violin plot
```

```
sns.violinplot(x="stroke",y="age",data=df)
plt.show()
```



4.perform Feature Engineering

Encoding categorical features (if any)

```
In [150... from sklearn.preprocessing import LabelEncoder

le=LabelEncoder()

for col in df.select_dtypes(include=['object']).columns:
    df[col]=le.fit_transform(df[col])

df.head()
```

```
Out[150...]
   id  gender  age  hypertension  heart_disease  ever_married  work_type  Residence
0  30669     1   3.0           0             0           0           4
1  30468     1  58.0           1             0           1           2
2  16523     0   8.0           0             0           0           2
3  56543     0  70.0           0             0           1           2
4  46136     1  14.0           0             0           0           1
```

Feature Scaling

```
In [151... from sklearn.preprocessing import StandardScaler

scaler=StandardScaler()

numeric_cols=["age","avg_glucose_level","bmi"]

df[numeric_cols]=scaler.fit_transform(df[numeric_cols])
df.head()
```

```
Out[151...      id  gender      age  hypertension  heart_disease  ever_married  work_type  Res
```

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Res
0	30669	1	-1.741517	0	0	0	4	
1	30468	1	0.700823	1	0	1	2	
2	16523	0	-1.519486	0	0	0	2	
3	56543	0	1.233697	0	0	1	2	
4	46136	1	-1.253049	0	0	0	1	

Create New Feature

```
In [152... df["health_score"]=df["avg_glucose_level"]+df["bmi"]
df.head()
```

```
Out[152...      id  gender      age  hypertension  heart_disease  ever_married  work_type  Res
```

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Res
0	30669	1	-1.741517	0	0	0	4	
1	30468	1	0.700823	1	0	1	2	
2	16523	0	-1.519486	0	0	0	2	
3	56543	0	1.233697	0	0	1	2	
4	46136	1	-1.253049	0	0	0	1	

5.Traning & Testing

```
In [153... from sklearn.model_selection import train_test_split

x=df.drop("stroke",axis=1)
y=df["stroke"]

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)

print("x_train",x_train.shape)
print("x_test",x_test.shape)
```

```
print("y_train",y_train.shape)
print("y_test",y_test.shape)
```

```
x_train (34720, 12)
x_test (8680, 12)
y_train (34720,)
y_test (8680,)
```

Titanic

1.Load using pandas

In [154...

```
import pandas as pd

df=pd.read_csv("D:/AI Notes/All Datasets (1)/titanic.csv")
df.head()
```

Out[154...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.25
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.2833
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	53.1
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.05



2.perform Data Cleaning

In [178...

```
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    int64
4   Sex          891 non-null    int64
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    int64
9   Fare         891 non-null    float64
10  Cabin        891 non-null    int64
11  Embarked     891 non-null    int64
12  FamilySize   891 non-null    int64
13  IsAlone      891 non-null    int64
dtypes: float64(2), int64(12)
memory usage: 97.6 KB

```

In [156... df.describe(include='all')

Out[156...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	
count	891.000000	891.000000	891.000000	891	891	714.000000	891.000000	89
unique	NaN	NaN	NaN	891	2	NaN	NaN	
top	NaN	NaN	NaN	Braund, Mr. Owen Harris	male	NaN	NaN	
freq	NaN	NaN	NaN	1	577	NaN	NaN	
mean	446.000000	0.383838	2.308642	NaN	NaN	29.699118	0.523008	
std	257.353842	0.486592	0.836071	NaN	NaN	14.526497	1.102743	
min	1.000000	0.000000	1.000000	NaN	NaN	0.420000	0.000000	
25%	223.500000	0.000000	2.000000	NaN	NaN	20.125000	0.000000	
50%	446.000000	0.000000	3.000000	NaN	NaN	28.000000	0.000000	
75%	668.500000	1.000000	3.000000	NaN	NaN	38.000000	1.000000	
max	891.000000	1.000000	3.000000	NaN	NaN	80.000000	8.000000	

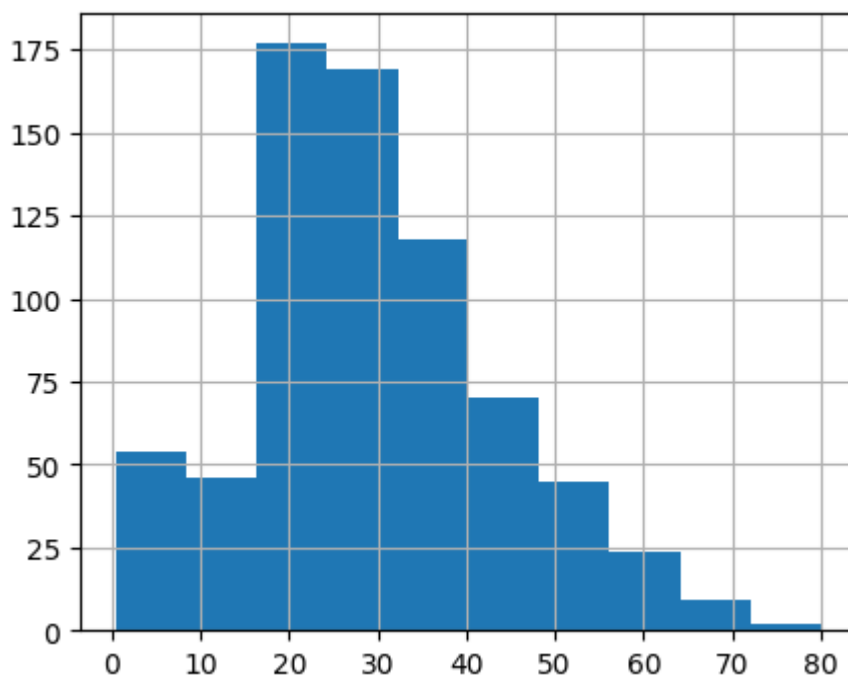
In [157... df.isnull().sum()

```
Out[157... PassengerId      0
Survived      0
Pclass        0
Name          0
Sex           0
Age          177
SibSp         0
Parch         0
Ticket        0
Fare          0
Cabin        687
Embarked      2
dtype: int64
```

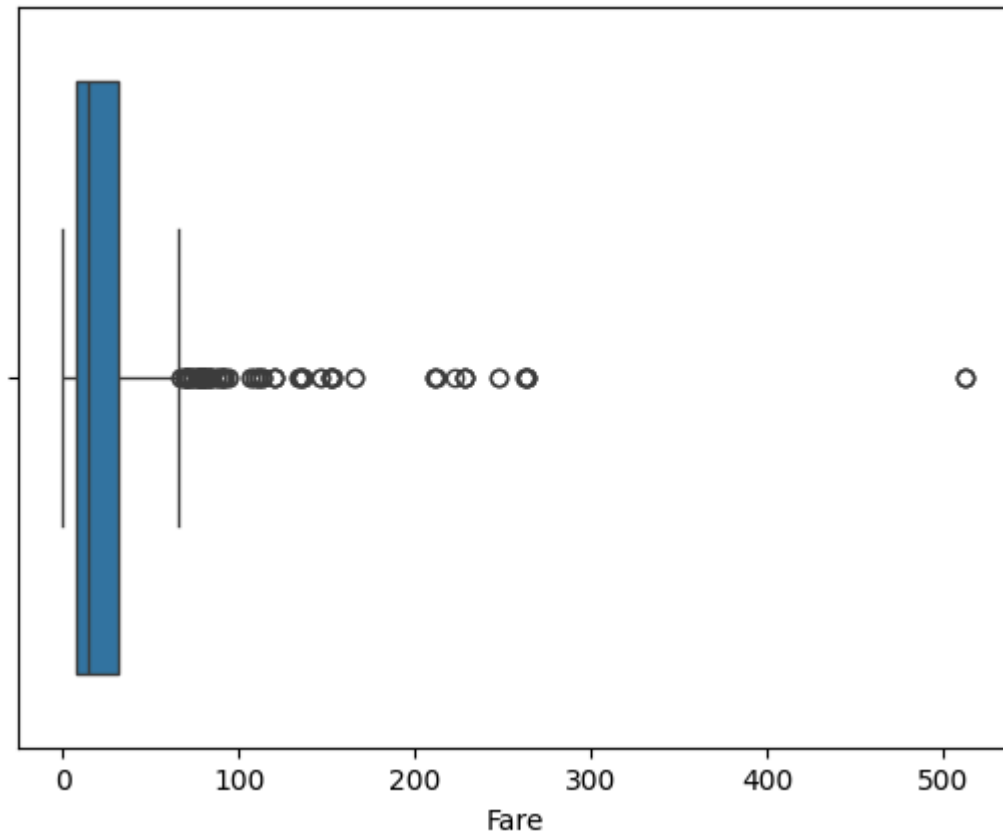
3.perform Exploratory Data Analysis(EDA)

```
In [159... import matplotlib.pyplot as plt
import seaborn as sns

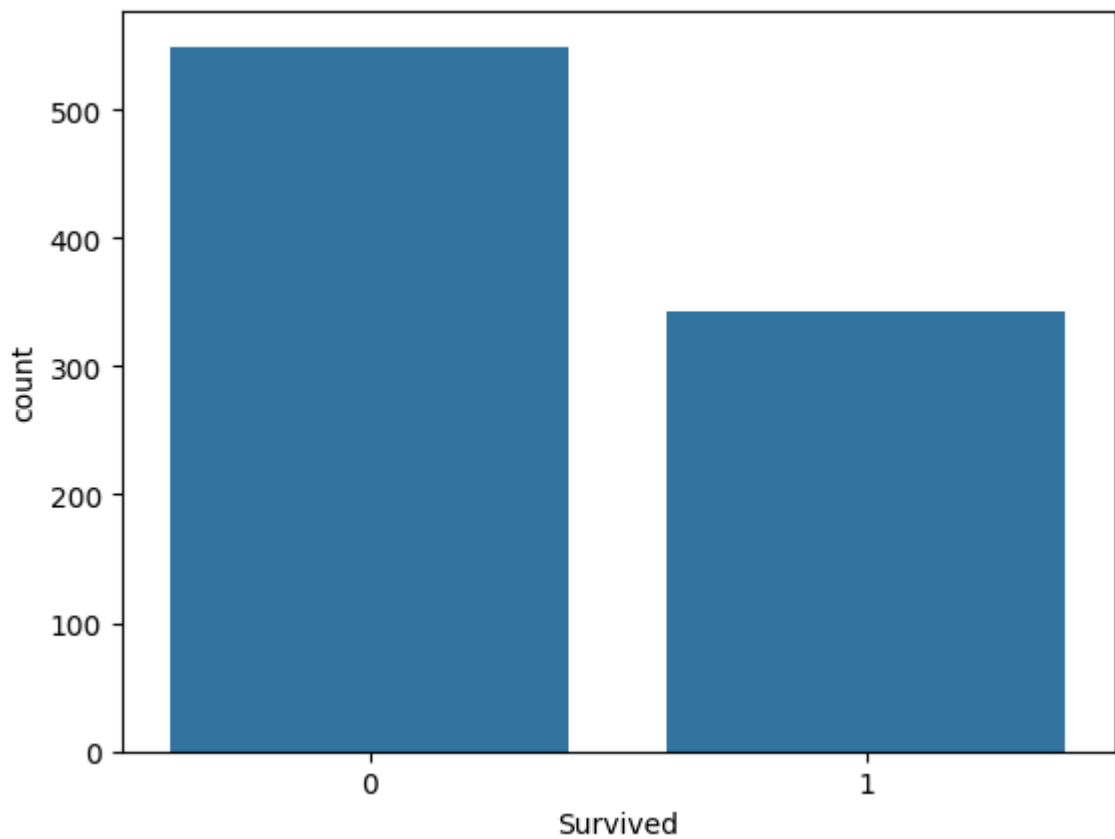
#1.Histogram
df["Age"].hist(figsize=(5,4))
plt.show()
```



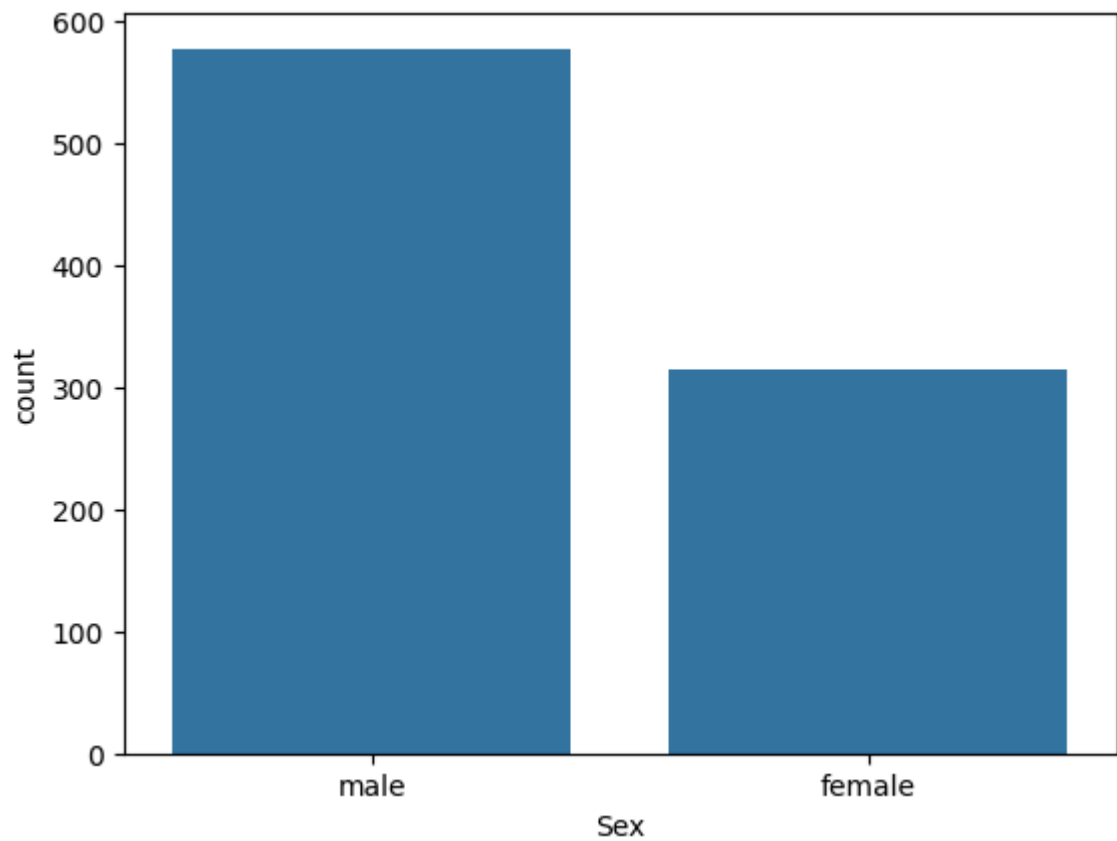
```
In [160... #2.Boxplot
sns.boxplot(x=df["Fare"])
plt.show()
```



```
In [161... #3.Countplot  
sns.countplot(x="Survived",data=df)  
plt.show()
```

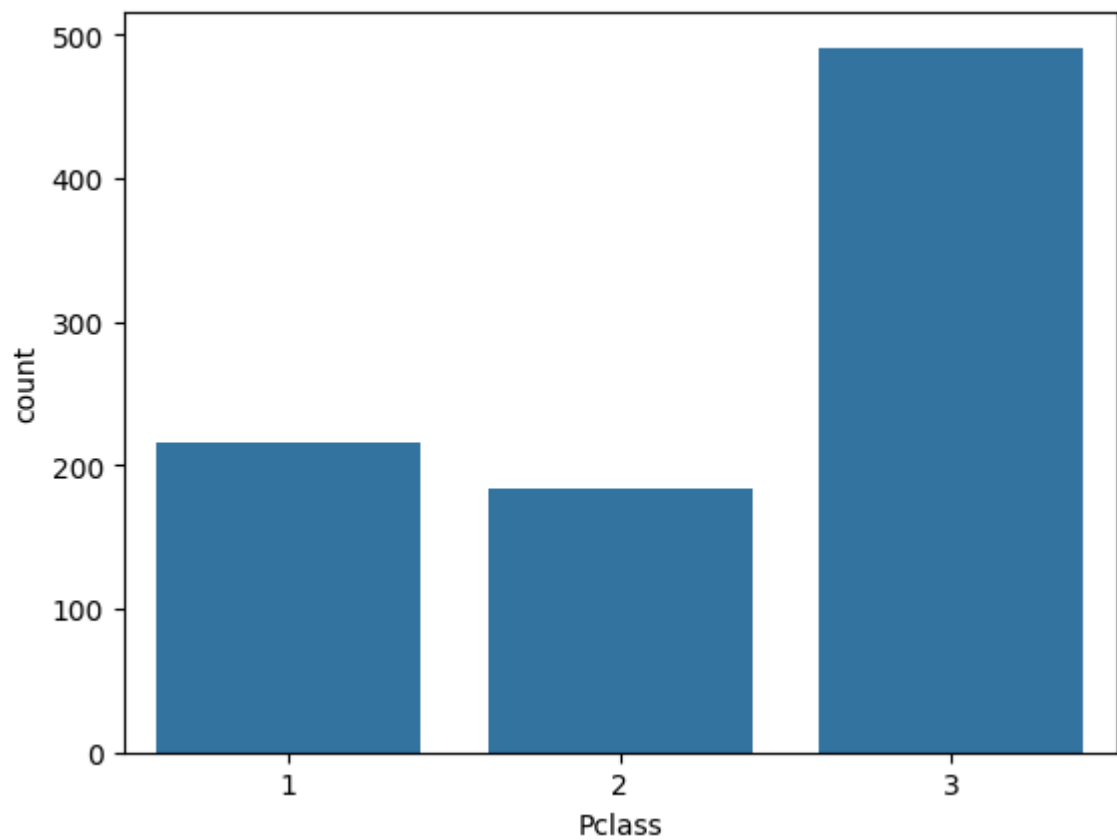


```
In [162... #4.Countplot  
sns.countplot(x="Sex",data=df)  
plt.show()
```



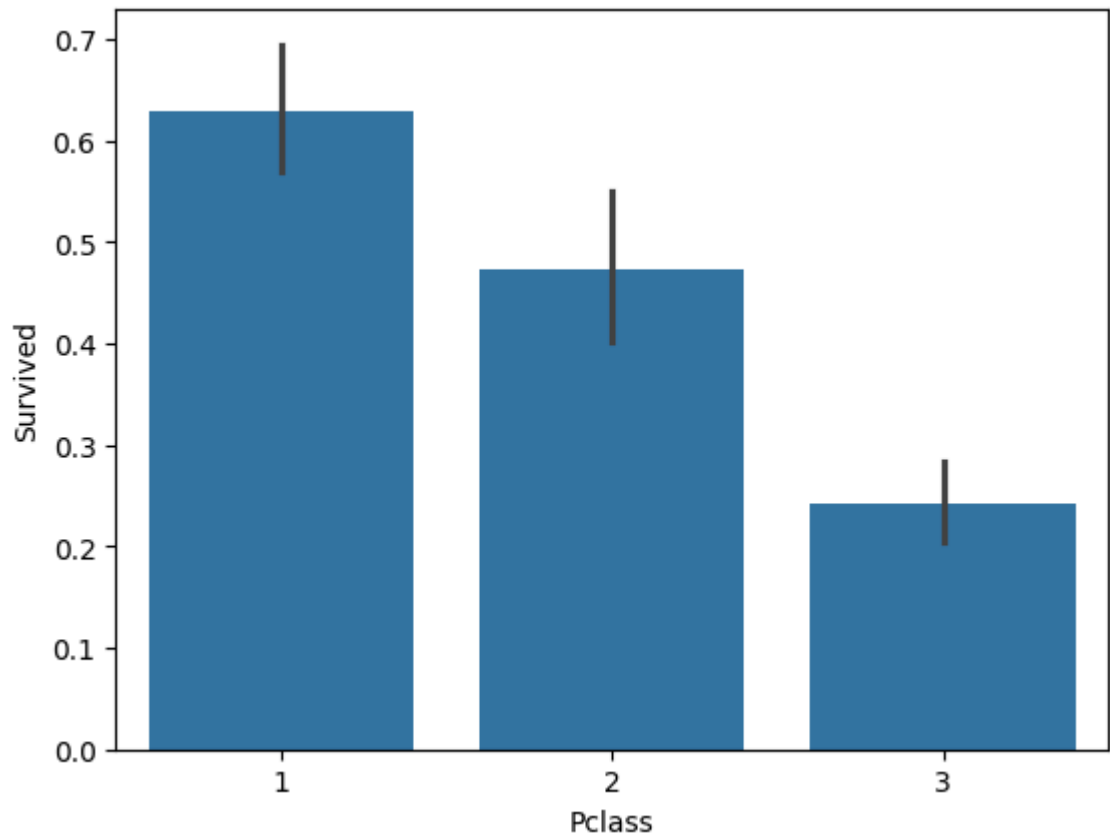
In [163...

```
#5.Countplot  
sns.countplot(x="Pclass",data=df)  
plt.show()
```



In [164...

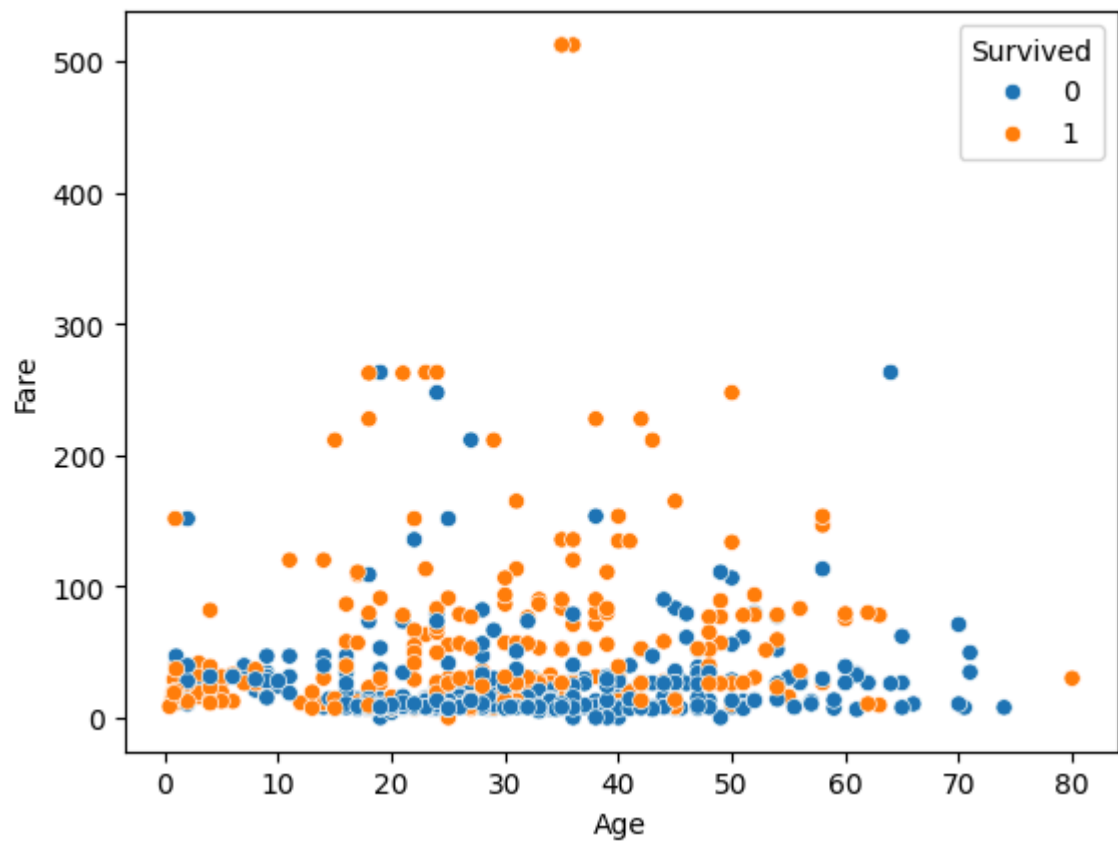
```
#6.Barplot  
sns.barplot(x="Pclass",y="Survived",data=df)  
plt.show()
```

In [165...

#7.Scatterplot

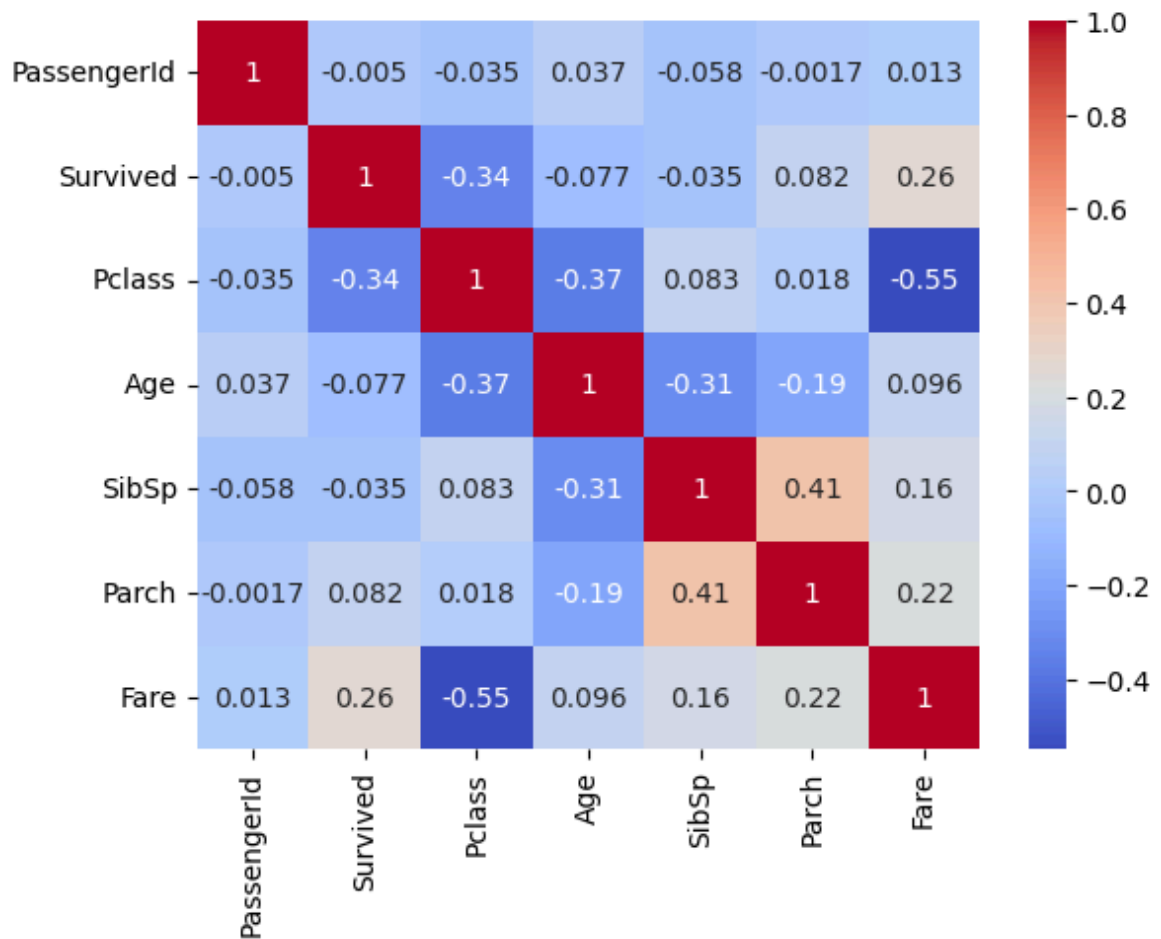
```
sns.scatterplot(x="Age",y="Fare",hue="Survived",data=df)
plt.show()
```



In [167...

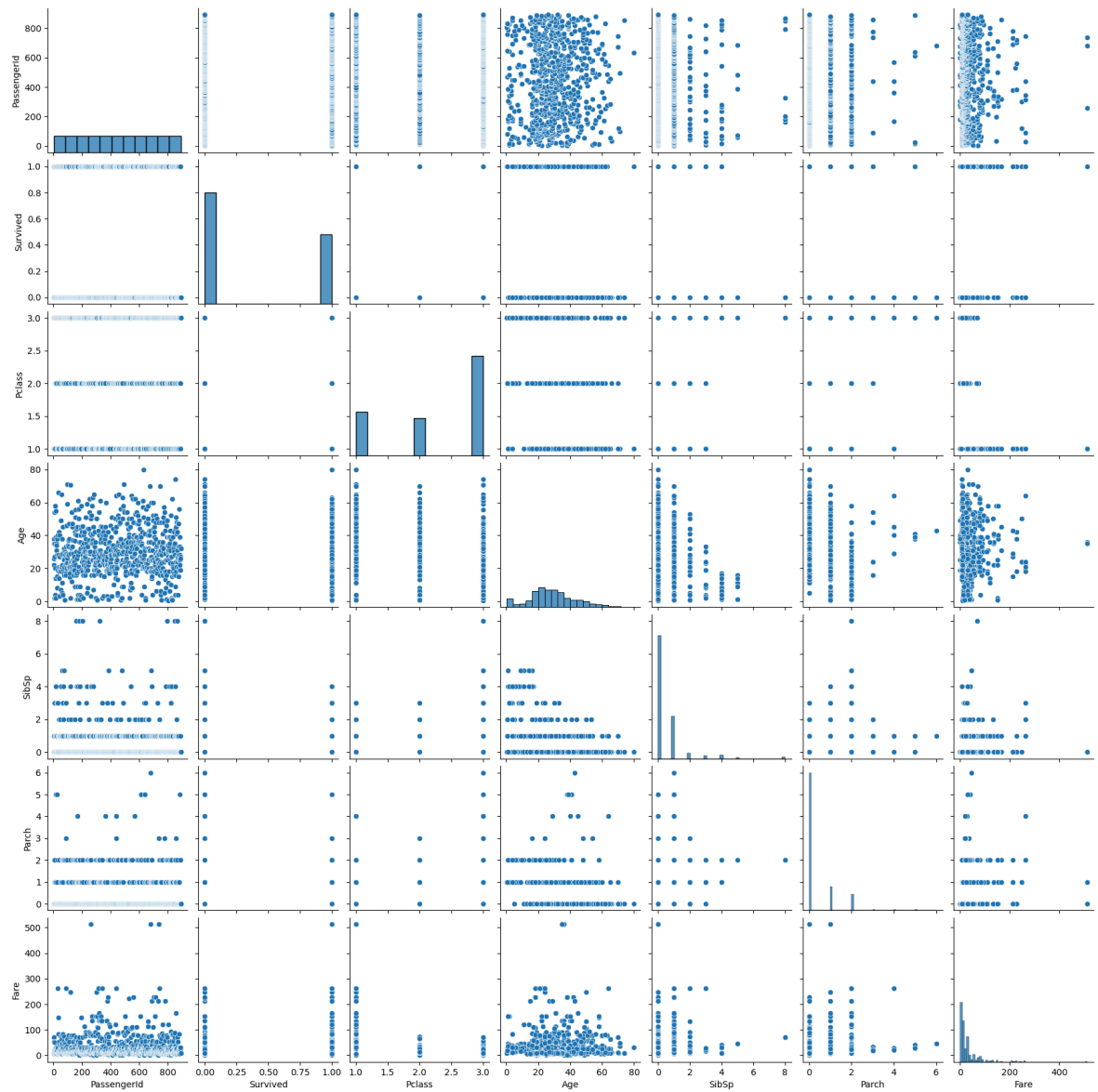
#8.Heatmap

```
sns.heatmap(df.select_dtypes(include="number").corr(),annot=True,cmap="coolwarm")
plt.show()
```



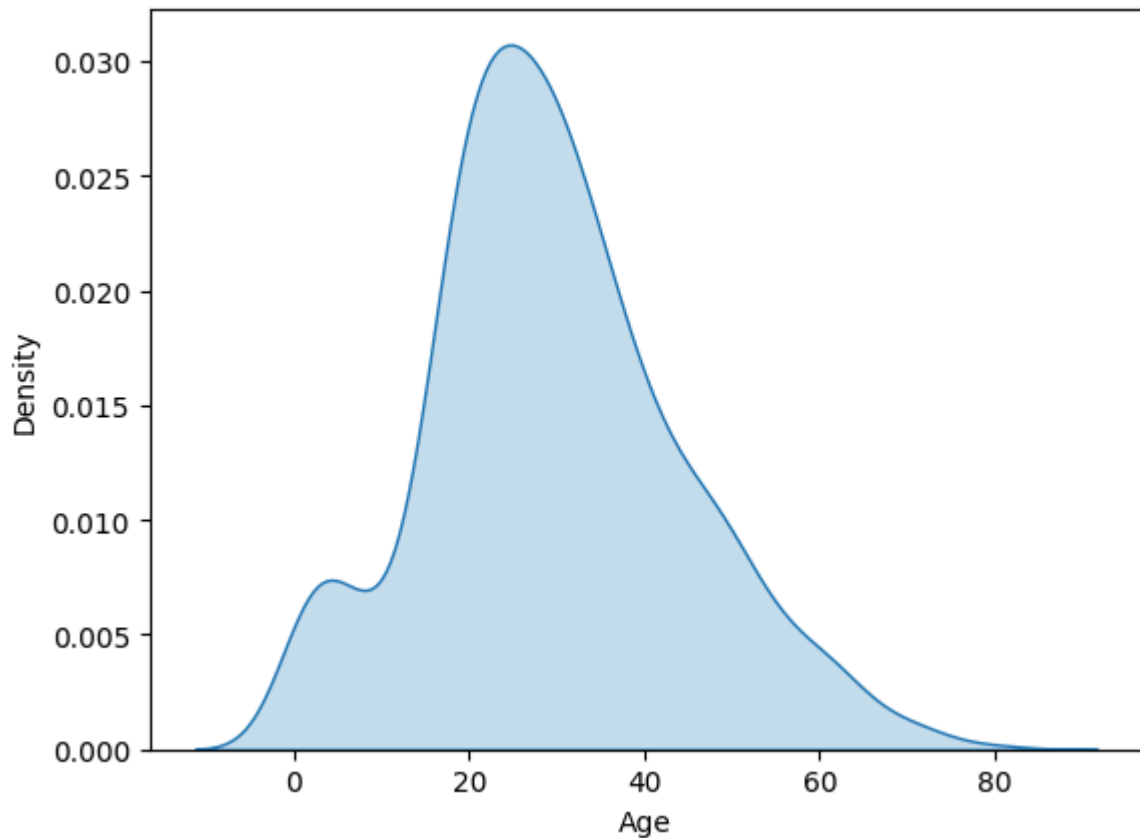
In [168...

```
#9.Pair Plot
sns.pairplot(df.select_dtypes(include="number"))
plt.show()
```



In [169...

```
#10.KDE plot
sns.kdeplot(df["Age"],fill=True)
plt.show()
```



4.perform Feature Engineering

Encoding categorical features (if any)

```
In [170... from sklearn.preprocessing import LabelEncoder

le=LabelEncoder()

for col in df.select_dtypes(include=['object']).columns:
    df[col]=le.fit_transform(df[col])
print(df.head())
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	\
0	1	0	3	108	1	22.0	1	0	523	
1	2	1	1	190	0	38.0	1	0	596	
2	3	1	3	353	0	26.0	0	0	669	
3	4	1	1	272	0	35.0	1	0	49	
4	5	0	3	15	1	35.0	0	0	472	

	Fare	Cabin	Embarked
0	7.2500	147	2
1	71.2833	81	0
2	7.9250	147	2
3	53.1000	55	2
4	8.0500	147	2

Feature Scaling

In [174...

```
from sklearn.preprocessing import StandardScaler

scaler=StandardScaler()

numeric_cols=df.select_dtypes(include=["int64","float64"]).columns
scaler.fit_transform(df[numeric_cols])
df.head()
```

Out[174...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin
0	1	0	3	108	1	22.0	1	0	523	7.2500	14
1	2	1	1	190	0	38.0	1	0	596	71.2833	8
2	3	1	3	353	0	26.0	0	0	669	7.9250	14
3	4	1	1	272	0	35.0	1	0	49	53.1000	5
4	5	0	3	15	1	35.0	0	0	472	8.0500	14

Create New Feature

In [175...

```
df["FamilySize"]=df["SibSp"]+df["Parch"]+1
df["IsAlone"]=(df["FamilySize"]==1).astype(int)
df.head()
```

Out[175...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin
0	1	0	3	108	1	22.0	1	0	523	7.2500	14
1	2	1	1	190	0	38.0	1	0	596	71.2833	8
2	3	1	3	353	0	26.0	0	0	669	7.9250	14
3	4	1	1	272	0	35.0	1	0	49	53.1000	5
4	5	0	3	15	1	35.0	0	0	472	8.0500	14

5.Traning & Testing

In [176...

```
from sklearn.model_selection import train_test_split

x=df.drop("Survived",axis=1)
y=df["Survived"]

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)

print("x_train",x_train.shape)
print("x_test",x_test.shape)
print("y_train",y_train.shape)
print("y_test",y_test.shape)
```

```
x_train (712, 13)
x_test (179, 13)
y_train (712,)
y_test (179,)
```